

1. Folder Structure

```
Project/
  .gitignore
  EcoBeacon_AI_Documentation.html
  README.md
  documentation.txt
  remove_blank_pdf_pages.py
  backend/
    .env
    .env.example
    package.json
    server.js
    config/
      db.js
      sequelize.js
    controllers/
      activityController.js
      aiController.js
      authController.js
      emissionController.js
    middleware/
      authMiddleware.js
    models/
      Activity.js
      User.js
    routes/
      activityRoutes.js
      aiRoutes.js
      authRoutes.js
      emissionRoutes.js
    seed/
      seedData.js
    services/
      aiService.js
      emissionService.js
    utils/
      ecoScore.js
  frontend/
    index.html
    package.json
    postcss.config.js
    tailwind.config.js
    vite.config.js
    public/
    src/
      App.jsx
      index.css
      main.jsx
      components/
        AIChat.jsx
        ActivityBreakdown.jsx
        AutoTracker.jsx
        BadgesPanel.jsx
        EcoScoreGauge.jsx
        EmissionChart.jsx
        ForecastPanel.jsx
        ImpactContext.jsx
```

```
    LoadingSpinner.jsx
    NLPActivityParser.jsx
    Navbar.jsx
    StatsCard.jsx
    SuggestionsPanel.jsx
context/
  AuthContext.jsx
hooks/
  useGeoTracker.js
pages/
  Dashboard.jsx
  LogActivity.jsx
  Login.jsx
  Profile.jsx
  Register.jsx
services/
  api.js
utils/
  geoTracking.js
  helpers.js
```

1. .gitignore

[Binary or unsupported file type:] - content not embedded.

2. EcoBeacon_AI_Documentation.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EcoBeacon AI ? Feature Documentation v2.0</title>
  <style>
    /* === CodeMie Design System ? inlined for portability === */
    @import url('https://fonts.googleapis.com/css2?family=Inter:ital,wght@0,400;0,500;0,600;0,700;1,400&family=JetBrainsMono:wght@400;500&display=swap');

    /* tokens.css */
    :root {
      --font-sans: 'Inter', 'Geist', -apple-system, BlinkMacSystemFont, 'Segoe UI', Arial, sans-serif;
      --font-mono: 'JetBrains Mono', 'GeistMono', 'Fira Code', 'Courier New', monospace;
      --text-xs-1: 0.625rem; --text-xs: 0.75rem; --text-sm-1: 0.813rem; --text-sm: 0.875rem;
      --text-base: 1rem; --text-h1: 2rem; --text-h2: 1.5rem; --text-h3: 1rem; --text-h4: 0.875rem;
      --text-h5: 0.75rem;
      --lh-h1: 2rem; --lh-h2: 1.5rem; --lh-h3: 1.3125rem; --lh-h4: 1.125rem; --lh-h5: 1rem;
      --radius-sm: 4px; --radius-md: 6px; --radius-lg: 8px; --radius-xl: 12px; --radius-2xl: 16px;
      --radius-full: 9999px;
      --space-0-5: 0.125rem; --space-1: 0.25rem; --space-1-5: 0.375rem; --space-2: 0.5rem; --space-2-5: 0.625rem;
      --space-3: 0.75rem; --space-4: 1rem; --space-5: 1.25rem; --space-6: 1.5rem; --space-8: 2rem;
      --shadow-sm: 0 1px 2px rgba(0,0,0,.4); --shadow-md: 0 4px 6px -1px rgba(0,0,0,.5), 0 2px 4px -2px rgba(0,0,0,.3);
      --shadow-lg: 0 10px 15px -3px rgba(0,0,0,.6), 0 4px 6px -4px rgba(0,0,0,.4);
      --transition-fast: 120ms ease; --transition-base: 200ms ease; --transition-slow: 300ms ease;
      --z-dropdown: 30; --z-sticky: 40; --z-modal: 50; --z-toast: 60;
    }
    :root, :root.dark {
      --color-bg-page: #1A1A1A; --color-bg-sidebar: #151515; --color-bg-card: #151515;
      --color-bg-nav: #000000; --color-bg-elevated: #2E2E2E; --color-bg-secondary: #212224;
      --color-bg-tertiary: #2E3033; --color-bg-quaternary: #333436; --color-bg-hover: #212224;
      --color-bg-hover-strong: #333436; --color-bg-input: #1A1A1A; --color-bg-input-prefix: #333436;
      --color-bg-btn-primary: #20222E; --color-bg-btn-primary-h: #262941; --color-bg-pagination-active: #212224;
      --color-text-primary: #FFFFFF; --color-text-secondary: #F2F0EF; --color-text-tertiary: #CCCCCC;
      --color-text-muted: #BBBBBB; --color-text-placeholder: #CCCCCC; --color-text-link: #F2F0EF;
      --color-text-link-hover: #F2F0EF; --color-text-inverse: #FFFFFF; --color-text-heading: #BBBBBB;
      --color-text-nav: #CCCCCC;
      --color-border-primary: #333436; --color-border-secondary: #47484A; --color-border-structural: #333436;
      --color-border-subtle: #4C4C4C; --color-border-accent: #FFFFFF; --color-border-focus: #FFFFFF;
      --color-border-error: #F9303C; --color-border-panel: #333436;
      --color-border-btn-secondary: #333436; --color-border-btn-secondary-h: transparent;
      --color-icon-primary: #FFFFFF; --color-icon-secondary: #CCCCCC; --color-icon-tertiary: #999999;
      --color-icon-error: #F9303C;
      --color-success: #259F4C; --color-success-bg: #1B271F; --color-success-border: #259F4C; --color-success-text: #259F4C;
      --color-error: #F9303C; --color-error-bg: #262121; --color-error-border: #FE3B4C; --color-error-text: #FE3B4C;
      --color-warning: #F5A534; --color-warning-bg: #492B00; --color-warning-border: #663B00; --color-warning-text: #F5A534;
      --color-info: #2297F6; --color-info-bg: #002442; --color-info-border: #003A69; --color-info-text: #2297F6;
      --color-purple: #C084FC; --color-purple-bg: #2D1B3D; --color-purple-border: #2D1B3D; --co

```

```

lor-purple-text: #F3E8FF;
  --color-cyan: #06B6D4; --color-cyan-bg: #003942; --color-cyan-border: #005866;
  --status-not-started-text: #A0A0A0; --status-not-started-bg: #333333; --status-not-started-border: #4C4C4C;
  --status-in-progress-text: #2297F6; --status-in-progress-bg: #002442; --status-in-progress-border: #003A69;
  --status-pending-text: #06B6D4; --status-pending-bg: #003942; --status-pending-border: #005866;
  --status-success-text: #259F4C; --status-success-bg: #1B271F; --status-success-border: #259F4C;
  --status-error-text: #FE3B4C; --status-error-bg: #262121; --status-error-border: #FE3B4C;
  --status-warning-text: #F5A534; --status-warning-bg: #492B00; --status-warning-border: #63B000;
  --status-advanced-text: #C084FC; --status-advanced-bg: #2D1B3D; --status-advanced-border: #2D1B3D;
  --gradient-primary-btn: linear-gradient(90deg,#672D92,#547CCC);
  --gradient-brand: linear-gradient(152deg,#0078C2,#0047FF,#8453D2);
  --gradient-magical: linear-gradient(90deg,#672D92,#5677C8);
}

/* base.css */
*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
html { font-size: 16px; scroll-behavior: smooth; }
body { font-family: var(--font-sans); font-size: var(--text-sm); line-height: 1.5; color: var(--color-text-primary); background-color: var(--color-bg-page); -webkit-font-smoothing: antialiased; }
a { color: var(--color-text-link); text-decoration: none; transition: color var(--transition-fast); }
a:hover { text-decoration: underline; }
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: var(--color-border-subtle); border-radius: var(--radius-full); }
::selection { background: rgba(34,151,246,.3); color: var(--color-text-primary); }
img, svg, video { display: block; max-width: 100%; }
ul, ol { list-style: none; }
code, pre, kbd { font-family: var(--font-mono); }
code { font-size: .875em; background: var(--color-bg-quaternary); color: var(--color-text-secondary); padding: .1em .4em; border-radius: var(--radius-sm); border: 1px solid var(--color-border-primary); }
pre { background: var(--color-bg-quaternary); border: 1px solid var(--color-border-primary); border-radius: var(--radius-lg); padding: var(--space-4); overflow-x: auto; font-size: var(--text-xs); line-height: 1.6; color: var(--color-text-secondary); }
pre code { background: transparent; border: none; padding: 0; }
hr, .divider { border: none; border-top: 1px solid var(--color-border-structural); margin: var(--space-4) 0; }
.container { width: 100%; max-width: 1100px; margin: 0 auto; padding: 0 var(--space-4); }

/* typography.css */
h1,.h1 { font-size: var(--text-h1); line-height: var(--lh-h1); font-weight: 700; color: var(--color-text-primary); letter-spacing: -.02em; }
h2,.h2 { font-size: var(--text-h2); line-height: var(--lh-h2); font-weight: 600; color: var(--color-text-primary); }
h3,.h3 { font-size: var(--text-h3); line-height: var(--lh-h3); font-weight: 600; color: var(--color-text-primary); }
h4,.h4 { font-size: var(--text-h4); line-height: var(--lh-h4); font-weight: 600; color: var(--color-text-primary); }
h5,.h5 { font-size: var(--text-h5); line-height: var(--lh-h5); font-weight: 600; color: var(--color-text-primary); }
h6,.h6 { font-size: var(--text-xs); line-height: 1.4; font-weight: 600; color: var(--color-text-muted); text-transform: uppercase; letter-spacing: .05em; }
.section-label { font-size: var(--text-xs); font-weight: 600; color: var(--color-text-heading); text-transform: uppercase; letter-spacing: .05em; margin-bottom: var(--space-2); }
.text-2xl { font-size: 1.5rem; } .text-xl { font-size: 1.25rem; } .text-lg { font-size: 1.125rem; }
.text-base { font-size: var(--text-base); } .text-sm { font-size: var(--text-sm); } .text-xs { font-size: var(--text-xs); }

```

```

    .font-normal { font-weight: 400; } .font-medium { font-weight: 500; } .font-semibold { font
-weight: 600; } .font-bold { font-weight: 700; }
    .text-primary { color: var(--color-text-primary); } .text-secondary { color: var(--color-te
xt-secondary); }
    .text-tertiary { color: var(--color-text-tertiary); } .text-muted { color: var(--color-text
-muted); }
    .text-success { color: var(--color-success-text); } .text-error { color: var(--color-error-
text); }
    .text-warning { color: var(--color-warning-text); } .text-info { color: var(--color-info-te
xt); }
    .uppercase { text-transform: uppercase; } .capitalize { text-transform: capitalize; }
    .truncate { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
    .nowrap { white-space: nowrap; } .text-left { text-align: left; } .text-center { text-align
: center; } .text-right { text-align: right; }
    p { color: var(--color-text-secondary); font-size: var(--text-sm); line-height: 1.6; }
    p + p { margin-top: var(--space-3); }
    .caption { font-size: var(--text-xs); color: var(--color-text-muted); line-height: 1.4; }
    .mono { font-family: var(--font-mono); font-size: .9em; }

/* buttons.css */
    .btn { display: inline-flex; align-items: center; justify-content: center; gap: var(--space
-1-5); font-family: var(--font-sans); font-weight: 600; white-space: nowrap; border-radius: var
(--radius-lg); border: 1px solid transparent; cursor: pointer; transition: background-color var
(--transition-base),border-color var(--transition-base),color var(--transition-base); text-deco
ration: none; user-select: none; }
    .btn-sm { height: 24px; padding: 0 var(--space-1-5); font-size: var(--text-xs); }
    .btn-md { height: 28px; padding: 0 var(--space-2); font-size: var(--text-xs); }
    .btn-lg { height: 44px; padding: 0 var(--space-4); font-size: var(--text-sm); }
    .btn-primary { background-color: var(--color-bg-btn-primary); color: var(--color-text-secon
dary); background-image: linear-gradient(var(--color-bg-btn-primary),var(--color-bg-btn-primary
)),var(--gradient-primary-btn); background-origin: border-box; background-clip: padding-box,bor
der-box; }
    .btn-secondary { background-color: var(--color-bg-secondary); color: var(--color-text-secon
dary); border-color: var(--color-border-btn-secondary); }
    .btn-magical { background: var(--gradient-magical); color: var(--color-text-inverse); borde
r-color: var(--color-border-structural); }

/* components.css */
    .card { display: flex; flex-direction: column; background-color: var(--color-bg-card); bord
er: 1px solid var(--color-border-structural); border-radius: var(--radius-xl); overflow: hidden
; transition: border-color var(--transition-base); }
    .card:hover { border-color: var(--color-border-secondary); }
    .card-header { display: flex; align-items: center; justify-content: space-between; padding:
var(--space-4); border-bottom: 1px solid var(--color-border-structural); }
    .card-body { flex: 1; padding: var(--space-4); }
    .card-title { font-size: var(--text-base); font-weight: 600; color: var(--color-text-primar
y); }
    .badge { display: inline-flex; align-items: center; gap: var(--space-1-5); padding: 0 var(-
space-2); height: 17px; border-radius: var(--radius-full); border: 1px solid; font-size: var(-
text-xs-1); font-weight: 700; text-transform: uppercase; letter-spacing: .04em; white-space: n
owrap; }
    .badge-dot { width: 7px; height: 7px; border-radius: var(--radius-full); display: inline-bl
ock; }
    .badge-success { background-color: var(--status-success-bg); color: var(--status-success-te
xt); border-color: var(--status-success-border); }
    .badge-success .badge-dot { background-color: var(--status-success-text); }
    .badge-warning { background-color: var(--status-warning-bg); color: var(--status-warning-te
xt); border-color: var(--status-warning-border); }
    .badge-warning .badge-dot { background-color: var(--status-warning-text); }
    .badge-error { background-color: var(--status-error-bg); color: var(--status-error-text); b
order-color: var(--status-error-border); }
    .badge-error .badge-dot { background-color: var(--status-error-text); }
    .badge-in-progress { background-color: var(--status-in-progress-bg); color: var(--status-in
-progress-text); border-color: var(--status-in-progress-border); }
    .badge-in-progress .badge-dot { background-color: var(--status-in-progress-text); animation
: badge-pulse 2s cubic-bezier(.4,0,.6,1) infinite; }
    .badge-advanced { background-color: var(--status-advanced-bg); color: var(--status-advanced

```

```

-text); border-color: var(--status-advanced-border); }
    .badge-advanced .badge-dot { background-color: var(--status-advanced-text); }
    .badge-not-started { background-color: var(--status-not-started-bg); color: var(--status-not-started-text); border-color: var(--status-not-started-border); }
    @keyframes badge-pulse { 0%,100%{opacity:1} 50%{opacity:.4} }
    .tag { display: inline-flex; align-items: center; gap: var(--space-1); padding: var(--space-1) var(--space-2); border-radius: var(--radius-lg); border: 1px solid var(--color-border-secondary); background-color: var(--color-bg-secondary); font-size: var(--text-xs); font-weight: 600; color: var(--color-text-tertiary); white-space: nowrap; }
    .tag-green { background-color: var(--color-success-bg); border-color: var(--color-success-border); color: var(--color-success-text); }
    .tag-blue { background-color: var(--color-info-bg); border-color: var(--color-info-border); color: var(--color-info-text); }
    .tag-purple { background-color: var(--color-purple-bg); border-color: var(--color-purple-border); color: var(--color-purple-text); }
    .tag-yellow { background-color: var(--color-warning-bg); border-color: var(--color-warning-border); color: var(--color-warning-text); }
    .tag-red { background-color: var(--color-error-bg); border-color: var(--color-error-border); color: var(--color-error-text); }
    .alert { display: flex; align-items: flex-start; gap: var(--space-2); padding: var(--space-2) var(--space-3); border-radius: var(--radius-md); border: 1px solid; font-size: var(--text-xs); line-height: 1.5; }
    .alert-info { background-color: var(--color-info-bg); border-color: var(--color-info-border); color: var(--color-info-text); }
    .alert-success { background-color: var(--color-success-bg); border-color: var(--color-success-border); color: var(--color-success-text); }
    .alert-warning { background-color: var(--color-warning-bg); border-color: var(--color-warning-border); color: var(--color-warning-text); }
    .stat-card { display: flex; flex-direction: column; gap: var(--space-1); padding: var(--space-3) var(--space-4); background-color: var(--color-bg-card); border: 1px solid var(--color-border-structural); border-radius: var(--radius-xl); }
    .stat-card-label { font-size: var(--text-xs-1); font-weight: 700; text-transform: uppercase; letter-spacing: .04em; color: var(--color-text-muted); }
    .stat-card-value { font-size: var(--text-h2); font-weight: 700; line-height: 1.2; color: var(--color-text-primary); }
    .stat-card-desc { font-size: var(--text-xs); color: var(--color-text-muted); }
    .stat-grid { display: grid; grid-template-columns: repeat(auto-fill,minmax(200px,1fr)); gap: var(--space-3); }
    .dl-grid { display: grid; grid-template-columns: max-content 1fr; gap: var(--space-2) var(--space-4); font-size: var(--text-sm); }
    .dl-grid dt { color: var(--color-text-muted); font-weight: 500; white-space: nowrap; }
    .dl-grid dd { color: var(--color-text-primary); }

    /* layout.css */
    .table-wrapper { width: 100%; overflow-x: auto; }
    table.table, .table { width: 100%; border-collapse: separate; border-spacing: 0; font-size: var(--text-xs); line-height: 1.3; }
    .table thead tr th { text-align: left; padding: var(--space-2-5) var(--space-4); font-weight: 600; color: var(--color-text-primary); background-color: var(--color-bg-quaternary); border-top: 1px solid var(--color-border-structural); border-bottom: 1px solid var(--color-border-structural); white-space: nowrap; }
    .table thead tr th:first-child { border-left: 1px solid var(--color-border-structural); border-radius: var(--radius-lg) 0 0 0; }
    .table thead tr th:last-child { border-right: 1px solid var(--color-border-structural); border-radius: 0 var(--radius-lg) 0 0; }
    .table tbody tr td { padding: var(--space-3) var(--space-4); color: var(--color-text-secondary); border-bottom: 1px solid var(--color-border-structural); vertical-align: middle; }
    .table tbody tr:hover td { background-color: var(--color-bg-hover); }
    .table tbody tr:last-child td { border-bottom: none; }
    .td-number { text-align: right; font-variant-numeric: tabular-nums; }
    .tabs { display: flex; flex-direction: column; }
    .tabs-list { display: flex; align-items: stretch; border-bottom: 1px solid var(--color-border-panel); }
    .tab-item { display: inline-flex; align-items: center; padding: var(--space-2) var(--space-2); padding-bottom: var(--space-4); font-size: var(--text-sm); color: var(--color-text-primary); cursor: pointer; border-bottom: 2px solid transparent; transition: border-color var(--transition-base); background: transparent; border-top: none; border-left: none; border-right: none; fo

```

```

nt-family: var(--font-sans); white-space: nowrap; }
.tab-item.active { border-bottom-color: var(--color-text-primary); font-weight: 600; }
.tabs-panel { flex: 1; padding-top: var(--space-4); }

/* utilities.css */
.hidden { display: none !important; } .block { display: block; } .flex { display: flex; } .
inline-flex { display: inline-flex; } .grid { display: grid; }
.flex-col { flex-direction: column; } .flex-1 { flex: 1; } .flex-shrink-0 { flex-shrink: 0;
}
.items-start { align-items: flex-start; } .items-center { align-items: center; } .items-end
{ align-items: flex-end; }
.justify-start { justify-content: flex-start; } .justify-center { justify-content: center;
} .justify-end { justify-content: flex-end; } .justify-between { justify-content: space-between
; }
.grid-2 { grid-template-columns: repeat(2,minmax(0,1fr)); } .grid-3 { grid-template-columns
: repeat(3,minmax(0,1fr)); } .grid-4 { grid-template-columns: repeat(4,minmax(0,1fr)); }
.gap-1{gap:var(--space-1)} .gap-2{gap:var(--space-2)} .gap-3{gap:var(--space-3)} .gap-4{gap
:var(--space-4)} .gap-6{gap:var(--space-6)} .gap-8{gap:var(--space-8)}
.p-1{padding:var(--space-1)} .p-2{padding:var(--space-2)} .p-3{padding:var(--space-3)} .p-4
{padding:var(--space-4)} .p-6{padding:var(--space-6)} .p-8{padding:var(--space-8)}
.px-2{padding-left:var(--space-2);padding-right:var(--space-2)} .px-3{padding-left:var(--sp
ace-3);padding-right:var(--space-3)} .px-4{padding-left:var(--space-4);padding-right:var(--spac
e-4)}
.py-1{padding-top:var(--space-1);padding-bottom:var(--space-1)} .py-2{padding-top:var(--spa
ce-2);padding-bottom:var(--space-2)} .py-3{padding-top:var(--space-3);padding-bottom:var(--spac
e-3)} .py-4{padding-top:var(--space-4);padding-bottom:var(--space-4)}
.pt-2{padding-top:var(--space-2)} .pt-4{padding-top:var(--space-4)} .pt-8{padding-top:var(-
space-8)} .pb-2{padding-bottom:var(--space-2)} .pb-4{padding-bottom:var(--space-4)}
.mx-auto{margin-left:auto;margin-right:auto} .ml-auto{margin-left:auto}
.mt-1{margin-top:var(--space-1)} .mt-2{margin-top:var(--space-2)} .mt-3{margin-top:var(--sp
ace-3)} .mt-4{margin-top:var(--space-4)} .mt-6{margin-top:var(--space-6)} .mt-8{margin-top:var(
--space-8)}
.mb-1{margin-bottom:var(--space-1)} .mb-2{margin-bottom:var(--space-2)} .mb-3{margin-bottom
:var(--space-3)} .mb-4{margin-bottom:var(--space-4)} .mb-6{margin-bottom:var(--space-6)}
.w-full{width:100%} .min-w-0{min-width:0} .max-w-full{max-width:100%}
.overflow-hidden{overflow:hidden} .overflow-x-auto{overflow-x:auto}
.relative{position:relative} .absolute{position:absolute}
.rounded-sm{border-radius:var(--radius-sm)} .rounded-md{border-radius:var(--radius-md)} .ro
unded-lg{border-radius:var(--radius-lg)} .rounded-xl{border-radius:var(--radius-xl)} .rounded-f
ull{border-radius:var(--radius-full)}
.bg-card{background-color:var(--color-bg-card)} .bg-secondary{background-color:var(--color-
bg-secondary)} .bg-elevated{background-color:var(--color-bg-elevated)}
.border{border:1px solid var(--color-border-structural)} .border-top{border-top:1px solid v
ar(--color-border-structural)} .border-bottom{border-bottom:1px solid var(--color-border-struct
ural)}
.shadow-sm{box-shadow:var(--shadow-sm)} .shadow-md{box-shadow:var(--shadow-md)}
.opacity-60{opacity:.6}

/* ===== Page-specific styles ===== */
body { padding: var(--space-6); }

/* Cover */
.cover {
background: linear-gradient(135deg, #0d1117 0%, #161b22 40%, #1e2a3a 100%);
border: 1px solid var(--color-border-structural);
border-radius: var(--radius-2xl);
padding: 64px 48px;
margin-bottom: var(--space-8);
position: relative;
overflow: hidden;
}
.cover::before {
content: '';
position: absolute;
top: -80px; right: -80px;
width: 300px; height: 300px;
background: radial-gradient(circle, rgba(100,190,100,0.12) 0%, transparent 70%);

```

```

    border-radius: 50%;
}
.cover-logo {
  width: 56px; height: 56px;
  background: linear-gradient(135deg, #1a7a3a, #2d9e5a);
  border-radius: var(--radius-xl);
  display: flex; align-items: center; justify-content: center;
  font-size: 28px; margin-bottom: var(--space-6);
}
.cover-title {
  font-size: 2.75rem; font-weight: 700; letter-spacing: -0.03em;
  background: linear-gradient(90deg, #4ade80, #22d3ee, #818cf8);
  -webkit-background-clip: text; -webkit-text-fill-color: transparent;
  background-clip: text; margin-bottom: var(--space-3);
}
.cover-subtitle { font-size: var(--text-base); color: var(--color-text-tertiary); margin-bottom: var(--space-6); }
.cover-meta { display: flex; gap: var(--space-4); flex-wrap: wrap; }
.cover-chip {
  display: inline-flex; align-items: center; gap: var(--space-2);
  padding: var(--space-1-5) var(--space-3);
  background: rgba(255,255,255,0.07); border: 1px solid rgba(255,255,255,0.12);
  border-radius: var(--radius-full); font-size: var(--text-xs); color: var(--color-text-tertiary);
}

/* TOC */
.toc-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: var(--space-3); }
.toc-item {
  display: flex; align-items: center; gap: var(--space-3);
  padding: var(--space-3) var(--space-4);
  background: var(--color-bg-secondary);
  border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-lg); cursor: pointer;
  transition: border-color var(--transition-base), background var(--transition-base);
  text-decoration: none;
}
.toc-item:hover { border-color: var(--color-border-secondary); background: var(--color-bg-hover); text-decoration: none; }
.toc-num { font-size: var(--text-xs); color: var(--color-text-muted); font-weight: 600; min-width: 20px; }
.toc-label { font-size: var(--text-sm); color: var(--color-text-primary); font-weight: 500; }

/* Section header */
.section-header {
  display: flex; align-items: center; gap: var(--space-3);
  padding: var(--space-4) 0; margin-bottom: var(--space-4);
  border-bottom: 1px solid var(--color-border-structural);
}
.section-icon {
  width: 40px; height: 40px; border-radius: var(--radius-lg);
  display: flex; align-items: center; justify-content: center; font-size: 20px;
  flex-shrink: 0;
}
.icon-green { background: rgba(37,159,76,0.15); }
.icon-blue { background: rgba(34,151,246,0.15); }
.icon-purple { background: rgba(192,132,252,0.15); }
.icon-orange { background: rgba(245,165,52,0.15); }
.icon-cyan { background: rgba(6,182,212,0.15); }

/* AI feature card */
.ai-feature-card {
  background: var(--color-bg-secondary);
  border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-xl);
  padding: var(--space-4);

```

```

    transition: border-color var(--transition-base);
  }
  .ai-feature-card:hover { border-color: var(--color-border-secondary); }
  .ai-feature-title { display: flex; align-items: center; gap: var(--space-2); margin-bottom:
var(--space-2); }
  .ai-feature-icon { font-size: 20px; }
  .ai-badge {
    font-size: var(--text-xs-1); font-weight: 700; text-transform: uppercase;
    padding: 2px 8px; border-radius: var(--radius-full);
    background: rgba(192,132,252,0.15); color: var(--color-purple);
    border: 1px solid var(--color-purple-border);
  }
  .example-block {
    margin-top: var(--space-3); padding: var(--space-3);
    background: var(--color-bg-tertiary);
    border: 1px solid var(--color-border-structural);
    border-radius: var(--radius-lg);
  }
  .example-label { font-size: var(--text-xs); color: var(--color-text-muted); margin-bottom:
var(--space-2); font-weight: 600; text-transform: uppercase; letter-spacing: .05em; }
  .example-pills { display: flex; flex-wrap: wrap; gap: var(--space-2); }
  .example-pill {
    font-size: var(--text-xs); padding: var(--space-1) var(--space-2-5);
    background: rgba(34,151,246,0.1); border: 1px solid var(--color-info-border);
    border-radius: var(--radius-full); color: var(--color-info-text);
    font-family: var(--font-mono);
  }
}

/* Step cards */
.steps { display: flex; flex-direction: column; gap: var(--space-3); }
.step-card {
  display: flex; gap: var(--space-4); padding: var(--space-4);
  background: var(--color-bg-secondary); border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-xl);
}
.step-num {
  width: 32px; height: 32px; border-radius: var(--radius-full);
  background: var(--gradient-magical); color: white;
  display: flex; align-items: center; justify-content: center;
  font-weight: 700; font-size: var(--text-sm); flex-shrink: 0;
}
.step-content h4 { margin-bottom: var(--space-1); }
.step-content p { margin-top: 0; }

/* Flow diagram */
.flow-diagram {
  display: flex; align-items: center; gap: var(--space-3);
  flex-wrap: wrap; padding: var(--space-4);
  background: var(--color-bg-secondary);
  border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-xl); margin-top: var(--space-4);
}
.flow-node {
  text-align: center; padding: var(--space-3) var(--space-4);
  background: var(--color-bg-elevated);
  border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-lg); min-width: 100px;
}
.flow-node-icon { font-size: 20px; margin-bottom: var(--space-1); }
.flow-node-label { font-size: var(--text-xs); font-weight: 600; color: var(--color-text-pri
mary); }
.flow-node-sub { font-size: var(--text-xs-1); color: var(--color-text-muted); margin-top: 2
px; }
.flow-arrow { color: var(--color-text-muted); font-size: 18px; flex-shrink: 0; }

/* Speed table */
.speed-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: var(--space-2); ma

```

```

margin-top: var(--space-3); }
.speed-item {
  text-align: center; padding: var(--space-3);
  background: var(--color-bg-elevated); border: 1px solid var(--color-border-structural);
  border-radius: var(--radius-lg);
}
.speed-icon { font-size: 24px; margin-bottom: var(--space-1); }
.speed-label { font-size: var(--text-xs); font-weight: 600; color: var(--color-text-primary); }
); }
.speed-range { font-size: var(--text-xs-1); color: var(--color-text-muted); }

/* Budget bar */
.budget-bar-track {
  height: 8px; background: var(--color-bg-tertiary);
  border-radius: var(--radius-full); overflow: hidden; margin-top: var(--space-1);
}
.budget-bar-fill { height: 100%; border-radius: var(--radius-full); }

/* Print support */
@media print {
  body { padding: 16px; background: white; color: #111; }
  .cover { background: #f3f4f6 !important; }
  .cover-title { -webkit-text-fill-color: #111 !important; background: none; color: #111; }
  .card, .ai-feature-card, .step-card { break-inside: avoid; }
  .section-header { break-after: avoid; }
}
</style>
</head>
<body>
<div class="container">

  <!-- ?? COVER ?????????????????????????????????????????????????????????????? -->
  <div class="cover">
    <div class="cover-logo">?</div>
    <div class="cover-title">EcoBeacon AI</div>
    <p class="cover-subtitle">AI-Powered Carbon Footprint Tracker ? Feature Documentation &
    Demo Guide</p>
    <div class="cover-meta">
      <span class="cover-chip">? Version 2.0</span>
      <span class="cover-chip">? April 2026</span>
      <span class="cover-chip">?? React 18 + Node.js</span>
      <span class="cover-chip">? 9 AI Features</span>
      <span class="cover-chip">? JWT Auth</span>
    </div>
  </div>

  <!-- ?? TABLE OF CONTENTS ?????????????????????????????????????????????????????????????? -->
  <div class="card mt-4 mb-8">
    <div class="card-header">
      <div class="card-title">? Table of Contents</div>
    </div>
    <div class="card-body">
      <div class="toc-grid">
        <a href="#overview" class="toc-item"><span class="toc-num">01</span><span class="toc-label">Application Overview</span></a>
        <a href="#stack" class="toc-item"><span class="toc-num">02</span><span class="toc-label">Tech Stack</span></a>
        <a href="#core" class="toc-item"><span class="toc-num">03</span><span class="toc-label">Core Features</span></a>
        <a href="#ai" class="toc-item"><span class="toc-num">04</span><span class="toc-label">Advanced AI Features</span></a>
        <a href="#dataflow" class="toc-item"><span class="toc-num">05</span><span class="toc-label">Data Flow</span></a>
        <a href="#demo" class="toc-item"><span class="toc-num">06</span><span class="toc-label">Demo Walkthrough</span></a>
        <a href="#api" class="toc-item"><span class="toc-num">07</span><span class="toc-label">API Reference</span></a>

```

```

    <a href="#emissions" class="toc-item"><span class="toc-num">08</span><span class="toc-
label">Emission Factors</span></a>
    <a href="#scoring" class="toc-item"><span class="toc-num">09</span><span class="toc-
label">Eco Score & Badges</span></a>
  </div>
</div>
</div>

<!-- ?? 01 OVERVIEW ?????????????????????????????????????????????? -->
<div id="overview" class="section-header">
  <div class="section-icon icon-green">?</div>
  <div>
    <h2>Application Overview</h2>
    <p class="caption">What EcoBeacon AI is and what it does</p>
  </div>
</div>

<div class="alert alert-info mb-4">
  EcoBeacon AI is a full-stack personal carbon footprint tracking web application. Users log
  daily activities across four categories, receive real-time emission calculations, earn gamifica
  tion rewards, and are guided by an AI engine to reduce their carbon impact.
</div>

<div class="stat-grid mb-6">
  <div class="stat-card">
    <span class="stat-card-label">Activity Categories</span>
    <span class="stat-card-value">4</span>
    <span class="stat-card-desc">Transport · Electricity · Food · Shopping</span>
  </div>
  <div class="stat-card">
    <span class="stat-card-label">AI-Powered Features</span>
    <span class="stat-card-value">9</span>
    <span class="stat-card-desc">NLP, Forecast, Chat, Anomaly Detection?</span>
  </div>
  <div class="stat-card">
    <span class="stat-card-label">Eco Score Range</span>
    <span class="stat-card-value">0?100</span>
    <span class="stat-card-desc">Dynamic sustainability score</span>
  </div>
  <div class="stat-card">
    <span class="stat-card-label">API Endpoints</span>
    <span class="stat-card-value">11</span>
    <span class="stat-card-desc">Auth, Activity, Emissions, AI</span>
  </div>
</div>

<!-- ?? 02 TECH STACK ?????????????????????????????????????????????? -->
<div id="stack" class="section-header mt-8">
  <div class="section-icon icon-blue">??</div>
  <div>
    <h2>Tech Stack</h2>
    <p class="caption">Technologies powering EcoBeacon AI</p>
  </div>
</div>

<div class="grid grid-2 gap-4 mb-6">
  <div class="card">
    <div class="card-header"><div class="card-title">?? Frontend</div></div>
    <div class="card-body">
      <dl class="dl-grid">
        <dt>Framework</dt><dd><code>React 18</code> with hooks & context</dd>
        <dt>Router</dt><dd><code>React Router v6</code></dd>
        <dt>HTTP</dt><dd><code>Axios</code> with interceptors</dd>
        <dt>Charts</dt><dd><code>Recharts</code> (AreaChart, PieChart)</dd>
        <dt>Styling</dt><dd><code>Tailwind CSS</code> custom config</dd>
        <dt>Build</dt><dd><code>Vite</code> with API proxy</dd>
      </dl>
    </div>
  </div>

```

```

</div>
<div class="card">
  <div class="card-header"><div class="card-title">?? Backend</div></div>
  <div class="card-body">
    <dl class="dl-grid">
      <dt>Runtime</dt><dd><code>Node.js</code> + <code>Express 4</code></dd>
      <dt>Database</dt><dd><code>SQLite 3</code> via Sequelize ORM</dd>
      <dt>Auth</dt><dd><code>JWT</code> (jsonwebtoken) + bcryptjs</dd>
      <dt>AI Engine</dt><dd>Custom rule-based service (no external API)</dd>
      <dt>Dev</dt><dd><code>nodemon</code> auto-reload</dd>
      <dt>GPS</dt><dd>Browser Geolocation API (client-side)</dd>
    </dl>
  </div>
</div>
</div>

<!-- ?? 03 CORE FEATURES ?????????????????????????????????????????????????????????????? -->
<div id="core" class="section-header mt-8">
  <div class="section-icon icon-orange">?</div>
  <div>
    <h2>Core Features</h2>
    <p class="caption">Foundational functionality available to all users</p>
  </div>
</div>

<div class="grid grid-2 gap-4 mb-4">
  <div class="card">
    <div class="card-header"><div class="card-title">? User Authentication</div></div>
    <div class="card-body">
      <p>Secure register/login flow using <strong>JWT tokens</strong>. Tokens are stored in <code>localStorage</code> under key <code>eco_token</code>. All protected routes require a valid Bearer token. Axios interceptors automatically inject the token on every request and redirect to <code>/login</code> on 401 responses.</p>
      <div class="flex gap-2 mt-3 flex-wrap">
        <span class="tag">Register</span>
        <span class="tag">Login</span>
        <span class="tag tag-green">JWT Protected</span>
        <span class="tag">Profile Update</span>
      </div>
    </div>
  </div>
  <div class="card">
    <div class="card-header"><div class="card-title">? Activity Logging</div></div>
    <div class="card-body">
      <p>Log activities across 4 categories. Each submission calculates emissions server-side, updates the Eco Score, checks for new badges, and updates the daily streak. A real-time emission preview is shown client-side as the user types ? no API call needed.</p>
      <div class="flex gap-2 mt-3 flex-wrap">
        <span class="tag tag-blue">? Transportation</span>
        <span class="tag tag-yellow">? Electricity</span>
        <span class="tag tag-green">?? Food</span>
        <span class="tag tag-purple">?? Shopping</span>
      </div>
    </div>
  </div>
</div>

<div class="grid grid-2 gap-4 mb-4">
  <div class="card">
    <div class="card-header"><div class="card-title">? Emission Dashboard</div></div>
    <div class="card-body">
      <p>The Dashboard displays four KPI stat cards (today, week, month, eco score), an <strong>Area Chart</strong> with toggle between stacked/total views, a <strong>Donut Chart</strong> for category breakdown, and an SVG arc <strong>Eco Score Gauge</strong>. Data is fetched in parallel via <code>Promise.all</code>.</p>
    </div>
  </div>

```

```

<div class="card">
  <div class="card-header"><div class="card-title">? Gamification</div></div>
  <div class="card-body">
    <p><strong>9 Badges</strong> are awarded for milestones: first log, activity count (10,
50), eco score thresholds (60, 80), streak length (7, 30 days), and eco-friendly choices. A <strong>Daily Streak</strong> counter increments when the user logs on consecutive days and resets if they miss a day.</p>
  </div>
</div>
</div>

<!-- ?? 04 AI FEATURES ?????????????????????????????????????????????????????????????? -->
<div id="ai" class="section-header mt-8">
  <div class="section-icon icon-purple">?</div>
  <div>
    <h2>Advanced AI Features</h2>
    <p class="caption">9 intelligent features added in v2.0 ? all client-side or rule-based,
no external ML API required</p>
  </div>
</div>

<div class="alert alert-success mb-4">
  <span>?</span>
  <span>All AI features are fully integrated and available in the application. No API keys or
external services required beyond the local Node.js backend.</span>
</div>

<!-- AI Feature 1 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">AI Natural Language Logger</h3>
    <span class="ai-badge">NLP</span>
  </div>
  <p>Located at the top of the <strong>Log Activity page</strong>. Users describe their activity in plain English ? a regex + keyword rule engine parses the text and auto-fills the activity type, category, and value fields.</p>
  <div class="example-block">
    <div class="example-label">Example inputs parsed correctly</div>
    <div class="example-pills">
      <span class="example-pill">drove 25km to work</span>
      <span class="example-pill">ate a beef burger for lunch</span>
      <span class="example-pill">used 8 kWh today</span>
      <span class="example-pill">cycled 10km to school</span>
      <span class="example-pill">flew 500km for business</span>
      <span class="example-pill">took the train 30km</span>
      <span class="example-pill">had a vegan dinner</span>
      <span class="example-pill">bought 2 items online</span>
    </div>
  </div>
  <p class="caption mt-3">File: <code>frontend/src/components/NLPActivityParser.jsx</code> .
35+ parsing rules across all 4 activity types</p>
</div>

<!-- AI Feature 2 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">AI Impact Contextualizer</h3>
    <span class="ai-badge">Analysis</span>
  </div>
  <p>Shown below the emission preview in the Log Activity form. Converts estimated CO2 into <strong>real-world equivalents</strong> that users intuitively understand, plus dual carbon budget progress bars.</p>
  <div class="grid grid-2 gap-3 mt-3">
    <div class="example-block">
      <div class="example-label">Real-world equivalents</div>

```

```

<dl class="dl-grid">
  <dt>? Trees</dt><dd>Days a tree needs to absorb this CO? (21 kg/yr)</dd>
  <dt>? Phones</dt><dd>Number of smartphone charges (0.008 kg each)</dd>
  <dt>? TV Hours</dt><dd>Hours of LED TV watching (0.1 kg/hr)</dd>
  <dt>? Showers</dt><dd>10-minute hot showers (0.5 kg each)</dd>
</dl>
</div>
<div class="example-block">
  <div class="example-label">Carbon budget bars</div>
  <p style="font-size:var(--text-xs);color:var(--color-text-muted);margin-bottom:8px">World average: 7.5 kg CO?/day</p>
  <div class="budget-bar-track"><div class="budget-bar-fill" style="width:35%;background:var(--color-success)"></div></div>
  <p style="font-size:var(--text-xs);color:var(--color-text-muted);margin-top:8px;margin-bottom:4px">Paris Agreement target: 2.5 kg CO?/day</p>
  <div class="budget-bar-track"><div class="budget-bar-fill" style="width:80%;background:var(--color-warning)"></div></div>
</div>
</div>
<p class="caption mt-3">File: <code>frontend/src/components/ImpactContext.jsx</code></p>
</div>

<!-- AI Feature 3 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">Smart Pattern Recognition</h3>
    <span class="ai-badge">Pattern AI</span>
  </div>
  <p>While filling the Log Activity form, the app fetches the user's recent history for the same activity type + category and <strong>compares the current input to their personal average</strong>. A contextual tip appears showing % above or below normal.</p>
  <div class="grid grid-2 gap-2 mt-3">
    <div class="alert alert-warning"><span>?</span><span><strong>Above average:</strong> "28% more than your usual 18.5 for this category."</span></div>
    <div class="alert alert-success"><span>?</span><span><strong>Below average:</strong> "15% less than your usual 18.5 ? great job!"</span></div>
  </div>
  <p class="caption mt-3">Logic: inline hook <code>useSmartTip()</code> in <code>LogActivity.jsx</code></p>
</div>

<!-- AI Feature 4 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">Carbon Budget Advisor</h3>
    <span class="ai-badge">Real-time</span>
  </div>
  <p>Displayed inline next to the emission preview. Shows the <strong>percentage of the Paris Agreement daily target (2.5 kg CO?)</strong> that the current activity would consume. Color-coded: green (<math>\leq 30\%</math>), amber (<math>30\% < 70\%</math>), red (<math>\geq 70\%</math>).</p>
  <p class="caption mt-2">Component: <code>CarbonBudgetBadge</code> inline in <code>LogActivity.jsx</code></p>
</div>

<!-- AI Feature 5 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">GPS Transport Mode Classifier</h3>
    <span class="ai-badge">Geolocation AI</span>
  </div>
  <p>Available in the <strong>Transportation tab</strong> of Log Activity. Uses the browser's <code>Geolocation.watchPosition()</code> API to track movement in real time. A weighted rolling average over the last 8 speed readings classifies the transport mode automatically.</p>
  <div class="speed-grid mt-3">

```

```

    <div class="speed-item"><div class="speed-icon">?</div><div class="speed-label">Walking</div><div class="speed-range">&lt; 7 km/h</div></div>
    <div class="speed-item"><div class="speed-icon">?</div><div class="speed-label">Cycling</div><div class="speed-range">&lt; 25 km/h</div></div>
    <div class="speed-item"><div class="speed-icon">?</div><div class="speed-label">Bus / City</div><div class="speed-range">&lt; 60 km/h</div></div>
    <div class="speed-item"><div class="speed-icon">?</div><div class="speed-label">Car</div><div class="speed-range">&lt; 130 km/h</div></div>
    <div class="speed-item"><div class="speed-icon">?</div><div class="speed-label">Train</div><div class="speed-range">&lt; 350 km/h</div></div>
    <div class="speed-item"><div class="speed-icon">??</div><div class="speed-label">Flight</div><div class="speed-range">? 350 km/h</div></div>
  </div>
  <p class="caption mt-3">Files: <code>frontend/src/utis/geoTracking.js</code>, <code>hooks/useGeoTracker.js</code>, <code>components/AutoTracker.jsx</code></p>
</div>

```

```

<!-- AI Feature 6 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">AI Emission Forecast</h3>
    <span class="ai-badge">Predictive</span>
  </div>
  <p>Visible on the <strong>Dashboard</strong> as the "AI Emission Forecast" panel. Analyses the user's last 30 days of activity to predict daily emissions for the next 7 days using a <strong>rolling average + linear trend</strong> algorithm. Displayed as an area chart with confidence level.</p>
  <div class="grid grid-2 gap-3 mt-3">
    <div>
      <div class="example-label">Algorithm</div>
      <ol style="list-style:decimal;padding-left:16px;font-size:var(--text-xs);color:var(--color-text-secondary);line-height:2">
        <li>Build date ? daily emission map</li>
        <li>Calculate overall mean</li>
        <li>Split history into first/second halves</li>
        <li>Compute trend delta (slope)</li>
        <li>Project: mean + trend × day offset</li>
      </ol>
    </div>
    <div>
      <div class="example-label">Confidence levels</div>
      <dl class="dl-grid">
        <dt><span class="badge badge-success">High</span></dt><dd>? 14 days of history</dd>
        <dt><span class="badge badge-warning">Medium</span></dt><dd>7?13 days of history</dd>
        <dt><span class="badge badge-not-started">Low</span></dt><dd>&lt; 7 days of history</dd>
      </dl>
    </div>
  </div>
  <p class="caption mt-3">Backend: <code>backend/services/aiService.js ? forecastEmissions()</code> · Frontend: <code>components/ForecastPanel.jsx</code></p>
</div>

```

```

<!-- AI Feature 7 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">AI Weekly Goals Generator</h3>
    <span class="ai-badge">Personalised</span>
  </div>
  <p>Shown below the Forecast chart on the Dashboard. The AI analyses usage patterns across the last 30 days and generates up to <strong>3 personalised weekly reduction goals</strong> with specific, measurable targets and estimated CO2 savings.</p>
  <div class="table-wrapper mt-3">
    <table class="table">
      <thead><tr><th>Goal Type</th><th>Trigger Condition</th><th>Example Target</th></tr></thead>

```

```

ead>
    <tbody>
      <tr><td>? Reduce Car Distance</td><td>User has car trips logged</td><td>"Drive max 20
0 km this week (was 250)"</td></tr>
      <tr><td>? Meat-Free Days</td><td>User has meat meals logged</td><td>"3 fewer meat mea
ls this week"</td></tr>
      <tr><td>? Cut Electricity Use</td><td>Electricity logs exist</td><td>"Use max 45 kWh
this week (was 50)"</td></tr>
      <tr><td>? Boost Eco Score</td><td>Eco score < 60</td><td>"Target score of 70"</td>
</tr>
    </tbody>
  </table>
</div>
<p class="caption mt-3">Backend: <code>backend/services/aiService.js ? generateWeeklyGoals(
)</code></p>
</div>

<!-- AI Feature 8 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">??</span>
    <h3 style="flex:1">Anomaly Detection</h3>
    <span class="ai-badge">Statistical</span>
  </div>
  <p>Available via <code>GET /api/ai/anomalies/:userId</code>. Scans the last 30 days of acti
vities per category and uses a <strong>z-score statistical method</strong> to flag any activity
with emission > 2x the category average and above the mean + 2? threshold.</p>
  <div class="example-block mt-3">
    <div class="example-label">Example anomaly alert</div>
    <div class="alert alert-warning" style="margin-top:8px">
      <span>??</span>
      <span>"This transportation activity emitted <strong>12.75 kg CO</strong> ? <strong>3.2
x</strong> your usual average of 3.98 kg."</span>
    </div>
  </div>
  <p class="caption mt-3">Backend: <code>backend/services/aiService.js ? detectAnomalies()</c
ode></p>
</div>

<!-- AI Feature 9 -->
<div class="ai-feature-card mb-3">
  <div class="ai-feature-title">
    <span class="ai-feature-icon">?</span>
    <h3 style="flex:1">AI Chat Assistant</h3>
    <span class="ai-badge">Conversational</span>
  </div>
  <p>A <strong>floating chat button</strong> (bottom-right corner) available on all authentic
ated pages. Rule-based NLP matches keyword patterns in user messages and returns contextual sus
tainability guidance. No external AI API required.</p>
  <div class="grid grid-2 gap-3 mt-3">
    <div>
      <div class="example-label">Quick-question shortcuts</div>
      <ul style="list-style:disc;padding-left:16px;font-size:var(--text-xs);color:var(--color
-text-secondary);line-height:2">
        <li>What is my biggest emission source?</li>
        <li>How can I reduce car emissions?</li>
        <li>What is the Paris Agreement target?</li>
        <li>How do I improve my eco score?</li>
        <li>Tips for sustainable food choices</li>
        <li>How do badges work?</li>
      </ul>
    </div>
    <div>
      <div class="example-label">Topics covered</div>
      <div class="flex gap-2 flex-wrap">
        <span class="tag">Transport</span>
        <span class="tag">Food & Diet</span>
      </div>
    </div>
  </div>

```

```

        <span class="tag">Electricity</span>
        <span class="tag">Shopping</span>
        <span class="tag">Eco Score</span>
        <span class="tag">Badges</span>
        <span class="tag">Paris Targets</span>
        <span class="tag">Aviation</span>
    </div>
</div>
</div>
<p class="caption mt-3">Files: <code>frontend/src/components/AIChat.jsx</code> · <code>back
end/controllers/aiController.js ? chatResponse()</code></p>
</div>

<!-- ?? 05 DATA FLOW ????????????????????????????????????????????????????????? -->
<div id="dataflow" class="section-header mt-8">
    <div class="section-icon icon-cyan">?</div>
    <div>
        <h2>How It Works ? Data Flow</h2>
        <p class="caption">End-to-end request pipeline from UI to database</p>
    </div>
</div>

<div class="card mb-4">
    <div class="card-header"><div class="card-title">? Request Pipeline</div></div>
    <div class="card-body">
        <div class="flow-diagram">
            <div class="flow-node"><div class="flow-node-icon">??</div><div class="flow-node-label">
>React UI</div><div class="flow-node-sub">Vite :3000</div></div>
            <div class="flow-arrow">?</div>
            <div class="flow-node"><div class="flow-node-icon">?</div><div class="flow-node-label">
Axios</div><div class="flow-node-sub">+ JWT header</div></div>
            <div class="flow-arrow">?</div>
            <div class="flow-node"><div class="flow-node-icon">?</div><div class="flow-node-label">
Express API</div><div class="flow-node-sub">Node :5000</div></div>
            <div class="flow-arrow">?</div>
            <div class="flow-node"><div class="flow-node-icon">?</div><div class="flow-node-label">
Auth Guard</div><div class="flow-node-sub">JWT verify</div></div>
            <div class="flow-arrow">?</div>
            <div class="flow-node"><div class="flow-node-icon">??</div><div class="flow-node-label">
>Controller</div><div class="flow-node-sub">Business logic</div></div>
            <div class="flow-arrow">?</div>
            <div class="flow-node"><div class="flow-node-icon">??</div><div class="flow-node-label">
>SQLite</div><div class="flow-node-sub">Sequelize ORM</div></div>
        </div>
    </div>
</div>

<div class="grid grid-2 gap-4 mb-6">
    <div class="card">
        <div class="card-header"><div class="card-title">? Auth Flow</div></div>
        <div class="card-body">
            <ol style="list-style:decimal;padding-left:16px;font-size:var(--text-xs);color:var(--co
lor-text-secondary);line-height:2.2">
                <li>User submits credentials to <code>POST /api/auth/login</code></li>
                <li>Server verifies password with bcryptjs</li>
                <li>JWT signed with <code>process.env.JWT_SECRET</code> (30d expiry)</li>
                <li>Token stored in <code>localStorage</code> as <code>eco_token</code></li>
                <li>Axios injects <code>Authorization: Bearer &lt;token></code> on every request</
li>
                <li>authMiddleware.js decodes token, attaches user to <code>req.user</code></li>
                <li>401 response ? Axios interceptor clears token, redirects to <code>/login</code></
li>
            </ol>
        </div>
    </div>
</div>
<div class="card">
    <div class="card-header"><div class="card-title">? AI Pipeline</div></div>

```

```

<div class="card-body">
  <ol style="list-style:decimal;padding-left:16px;font-size:var(--text-xs);color:var(--color-text-secondary);line-height:2.2">
    <li>Dashboard / LogActivity page loads</li>
    <li>Frontend calls <code>GET /api/ai/suggestions/:userId</code></li>
    <li>Controller fetches last 30 days of activities from SQLite</li>
    <li><code>aiService.js</code> analyses by type, generates prioritised suggestions</li>
  </ol>
</div>
<div class="card-body">
  <ol style="list-style:decimal;padding-left:16px;font-size:var(--text-xs);color:var(--color-text-secondary);line-height:2.2">
    <li>Forecast: applies rolling average + linear trend, returns 7-day prediction</li>
    <li>Goals: pattern-analyses usage, generates 3 personalised targets</li>
    <li>Chat: keyword matching on <code>POST /api/ai/chat</code>, returns contextual reply</li>
  </ol>
</div>
</div>

<!-- ?? 06 DEMO WALKTHROUGH ?????????????????????????????????????????????????????????????? -->
<div id="demo" class="section-header mt-8">
  <div class="section-icon icon-green">?</div>
  <div>
    <h2>Demo Walkthrough</h2>
    <p class="caption">Step-by-step guide for a live product demonstration</p>
  </div>
</div>

<div class="alert alert-info mb-4">
  <span>??</span>
  <span>Before demo: start backend (<code>cd backend &amp;&amp; npm start</code>) and frontend (<code>cd frontend &amp;&amp; npm run dev</code>). Default URLs: Frontend <strong>http://localhost:3000</strong> · API <strong>http://localhost:5000</strong></span>
</div>

<div class="steps">
  <div class="step-card">
    <div class="step-num">1</div>
    <div class="step-content">
      <h4>Register a new account</h4>
      <p>Navigate to <code>/register</code>. Fill in name, email, and password. Show that the account is created instantly with an Eco Score of 50. Login redirects to the Dashboard.</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">2</div>
    <div class="step-content">
      <h4>Log a car trip using NLP</h4>
      <p>Go to <code>/log</code>. In the blue "AI Natural Language Logger" box at the top, type <em>"drove 25km to work"</em> and press Parse. Watch as the form auto-selects Transportation ? Car ? 25 km. Click Apply, then Log Activity.</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">3</div>
    <div class="step-content">
      <h4>Show impact analysis</h4>
      <p>After setting a value, click "Show AI impact analysis" below the emission preview. The Impact Contextualizer appears showing trees needed, phone charges, TV hours, and carbon budget bars for the Paris target and world average.</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">4</div>
    <div class="step-content">
      <h4>Log 4?5 more activities</h4>
      <p>Log: beef meal (Food ? Beef), 10 kWh electricity (Electricity ? Standard), a flight (Transportation ? Flight, NLP: "flew 300km"), a vegan meal. This gives the AI service enough data to generate meaningful analysis.</p>
    </div>
  </div>
</div>

```

```

    </div>
  </div>
  <div class="step-card">
    <div class="step-num">5</div>
    <div class="step-content">
      <h4>View the Dashboard AI Forecast</h4>
      <p>Navigate to Dashboard. Scroll past the charts to the "AI Emission Forecast" panel. Show the 7-day prediction area chart, trend direction (increasing/decreasing/stable), confidence level, and the personalised weekly goals below it.</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">6</div>
    <div class="step-content">
      <h4>Demonstrate AI Suggestions</h4>
      <p>In the SuggestionsPanel on the Dashboard, expand the high-priority suggestions. Show how suggestions are generated specifically for the activities logged (e.g., "Switch to Public Transport" if car trips were logged).</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">7</div>
    <div class="step-content">
      <h4>Chat with AI Assistant</h4>
      <p>Click the ? floating button (bottom-right). Ask: <em>"What is my biggest emission source?"</em> ? show the contextual response. Try a quick-question shortcut. Ask about the Paris Agreement target to demonstrate climate knowledge integration.</p>
    </div>
  </div>
  <div class="step-card">
    <div class="step-num">8</div>
    <div class="step-content">
      <h4>Show Badges & Streak</h4>
      <p>Scroll to the BadgesPanel on Dashboard. Show which badges have been earned (First Log, etc.). Navigate to Profile (<code>/profile</code>) to show the paginated activity history with delete functionality and the full badges grid.</p>
    </div>
  </div>
</div>

<!-- ?? 07 API REFERENCE ????????????????????????????????????????????????????????????? -->
<div id="api" class="section-header mt-8">
  <div class="section-icon icon-blue">?</div>
  <div>
    <h2>API Reference</h2>
    <p class="caption">All available REST endpoints ? base URL: <code>http://localhost:5000/api</code></p>
  </div>
</div>

<div class="table-wrapper mb-6">
  <table class="table">
    <thead>
      <tr>
        <th>Method</th>
        <th>Endpoint</th>
        <th>Auth</th>
        <th>Description</th>
        <th>Response</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td><span class="badge badge-success">POST</span></td>
        <td><code>/auth/register</code></td>
        <td>No</td>
        <td>Register new user (name, email, password)</td>
        <td>user + token</td>
      </tr>
      <tr>
        <td><span class="badge badge-success">POST</span></td>
        <td><code>/auth/login</code></td>
        <td>No</td>
        <td>Login with email + password</td>
        <td>user + token</td>
      </tr>
      <tr>
        <td><span class="badge badge-in-progress">GET</span></td>
        <td><code>/auth/profile</code></td>
        <td></td>
        <td></td>
        <td></td>
      </tr>
    </tbody>
  </table>
</div>

```

```

ode></td><td>? JWT</td><td>Get current user's profile</td><td>user object</td></tr>
    <tr><td><span class="badge badge-warning">PUT</span></td><td><code>/auth/profile</code>
</td><td>? JWT</td><td>Update user name</td><td>updated user</td></tr>
    <tr><td><span class="badge badge-success">POST</span></td><td><code>/activity/add</code>
></td><td>? JWT</td><td>Log a new activity (type, category, value, notes, date)</td><td>activit
y + stats</td></tr>
    <tr><td><span class="badge badge-in-progress">GET</span></td><td><code>/activity/user/:
id</code></td><td>? JWT</td><td>Get user's activity history (supports <code>?limit=N</code>)</t
d><td>activities[]</td></tr>
    <tr><td><span class="badge badge-error">DELETE</span></td><td><code>/activity/:id</code>
></td><td>? JWT</td><td>Delete a specific activity</td><td>success message</td></tr>
    <tr><td><span class="badge badge-in-progress">GET</span></td><td><code>/emissions/summa
ry/:id</code></td><td>? JWT</td><td>Get daily/weekly/monthly summaries + breakdowns</td><td>sum
mary object</td></tr>
    <tr><td><span class="badge badge-in-progress">GET</span></td><td><code>/ai/suggestions/
:userId</code></td><td>? JWT</td><td>Get rule-based AI recommendations (last 30 days)</td><td>s
uggestions[]</td></tr>
    <tr><td><span class="badge badge-in-progress">GET</span></td><td><code>/ai/forecast/:us
erId</code></td><td>? JWT</td><td>7-day emission forecast + weekly goals</td><td>forecast[] + g
oals[]</td></tr>
    <tr><td><span class="badge badge-in-progress">GET</span></td><td><code>/ai/anomalies/:u
serId</code></td><td>? JWT</td><td>Detect unusual emission spikes (z-score method)</td><td>anom
alies[]</td></tr>
    <tr><td><span class="badge badge-success">POST</span></td><td><code>/ai/chat</code></td>
><td>? JWT</td><td>AI chat ? send message, receive sustainability reply</td><td>{ reply }</td><
/tr>
  </tbody>
</table>
</div>

<!-- ?? 08 EMISSION FACTORS ?????????????????????????????????????????????? -->
<div id="emissions" class="section-header mt-8">
  <div class="section-icon icon-orange">??</div>
  <div>
    <h2>Emission Factors Reference</h2>
    <p class="caption">CO2 factors used for all calculations (kg CO2 per unit)</p>
  </div>
</div>

<div class="grid grid-2 gap-4 mb-6">
  <div class="card">
    <div class="card-header"><div class="card-title">? Transportation (kg CO2/km)</div></div>
    <div class="card-body p-0">
      <table class="table">
        <thead><tr><th>Category</th><th class="td-number">Factor</th></tr></thead>
        <tbody>
          <tr><td>Car</td><td class="td-number text-error">0.210</td></tr>
          <tr><td>Motorcycle</td><td class="td-number text-warning">0.103</td></tr>
          <tr><td>Bus</td><td class="td-number text-warning">0.089</td></tr>
          <tr><td>Flight</td><td class="td-number text-error">0.255</td></tr>
          <tr><td>Train</td><td class="td-number text-success">0.041</td></tr>
          <tr><td>Electric Car</td><td class="td-number text-success">0.053</td></tr>
          <tr><td>Bicycle</td><td class="td-number text-success">0.005</td></tr>
          <tr><td>Walking</td><td class="td-number text-success">0.000</td></tr>
        </tbody>
      </table>
    </div>
  </div>
</div>
<div>
  <div class="card mb-3">
    <div class="card-header"><div class="card-title">? Electricity (kg CO2/kwh)</div></div>
    <div class="card-body p-0">
      <table class="table">
        <thead><tr><th>Category</th><th class="td-number">Factor</th></tr></thead>
        <tbody>
          <tr><td>Standard Grid</td><td class="td-number text-error">0.850</td></tr>
          <tr><td>Solar</td><td class="td-number text-success">0.041</td></tr>

```

```

        <tr><td>Wind Energy</td><td class="td-number text-success">0.011</td></tr>
    </tbody>
</table>
</div>
</div>
<div class="card">
    <div class="card-header"><div class="card-title">?? Shopping (kg CO?/item)</div></div>
    <div class="card-body p-0">
        <table class="table">
            <thead><tr><th>Category</th><th class="td-number">Factor</th></tr></thead>
            <tbody>
                <tr><td>High Impact</td><td class="td-number text-error">5.00</td></tr>
                <tr><td>Normal</td><td class="td-number text-warning">2.00</td></tr>
                <tr><td>Eco-Friendly</td><td class="td-number text-success">0.50</td></tr>
                <tr><td>Secondhand</td><td class="td-number text-success">0.20</td></tr>
            </tbody>
        </table>
    </div>
</div>
</div>
</div>
<div class="card mb-6">
    <div class="card-header"><div class="card-title">?? Food (kg CO?/meal)</div></div>
    <div class="card-body p-0">
        <table class="table">
            <thead><tr><th>Category</th><th class="td-number">Factor</th><th>Notes</th></tr></thead>
            <tbody>
                <tr><td>? Beef Meal</td><td class="td-number text-error">6.61</td><td>Highest-impact food category</td></tr>
                <tr><td>? Meat Meal</td><td class="td-number text-error">5.00</td><td>Chicken, pork, lamb etc.</td></tr>
                <tr><td>? Fast Food</td><td class="td-number text-warning">3.50</td><td>Includes packaging emissions</td></tr>
                <tr><td>? Fish Meal</td><td class="td-number text-warning">2.80</td><td>Varies by species</td></tr>
                <tr><td>? Dairy-based</td><td class="td-number text-warning">2.10</td><td>Cheese, milk products</td></tr>
                <tr><td>? Vegetarian</td><td class="td-number text-success">1.50</td><td>Plant-heavy, no meat</td></tr>
                <tr><td>? Vegan</td><td class="td-number text-success">0.90</td><td>Lowest-impact food option</td></tr>
            </tbody>
        </table>
    </div>
</div>

<!-- ?? 09 SCORING ?????????????????????????????????????????????????????????????? -->
<div id="scoring" class="section-header mt-8">
    <div class="section-icon icon-green">?</div>
    <div>
        <h2>Eco Score & Badges System</h2>
        <p class="caption">How the Eco Score is calculated and what triggers each badge</p>
    </div>
</div>

<div class="grid grid-2 gap-4 mb-6">
    <div class="card">
        <div class="card-header"><div class="card-title">? Eco Score Calculation (0?100)</div></div>
        <div class="card-body">
            <p class="mb-3">Based on daily average emission over the last 30 days:</p>
            <table class="table">
                <thead><tr><th>Daily Average</th><th class="td-number">Base Score</th></tr></thead>
                <tbody>
                    <tr><td>? 2 kg/day</td><td class="td-number text-success">95</td></tr>
                </tbody>
            </table>
        </div>
    </div>
</div>

```

```

        <tr><td>? 5 kg/day</td><td class="td-number text-success">80</td></tr>
        <tr><td>? 10 kg/day</td><td class="td-number text-success">65</td></tr>
        <tr><td>? 15 kg/day</td><td class="td-number text-warning">50</td></tr>
        <tr><td>? 20 kg/day</td><td class="td-number text-warning">35</td></tr>
        <tr><td>? 30 kg/day</td><td class="td-number text-error">20</td></tr>
        <tr><td>&gt; 30 kg/day</td><td class="td-number text-error">10</td></tr>
    </tbody>
</table>
<p class="caption mt-3">+0 to +10 bonus points for eco-friendly activity ratio (walking
, cycling, vegan, etc.)</p>
</div>
</div>
<div class="card">
    <div class="card-header"><div class="card-title">?? Badge Criteria</div></div>
    <div class="card-body">
        <table class="table">
            <thead><tr><th>Badge</th><th>Trigger</th></tr></thead>
            <tbody>
                <tr><td>? First Step</td><td>First activity logged</td></tr>
                <tr><td>? Logger</td><td>10 activities logged</td></tr>
                <tr><td>? Active Tracker</td><td>50 activities logged</td></tr>
                <tr><td>? Eco Starter</td><td>Eco score ? 60</td></tr>
                <tr><td>? Eco Champion</td><td>Eco score ? 80</td></tr>
                <tr><td>? Streak Starter</td><td>7-day streak</td></tr>
                <tr><td>?? Streak Master</td><td>30-day streak</td></tr>
                <tr><td>? Green Traveller</td><td>5+ walking/cycling trips</td></tr>
                <tr><td>? Plant Lover</td><td>5+ veg/vegan meals</td></tr>
            </tbody>
        </table>
    </div>
</div>
</div>
<!-- ?? FOOTER ????????????????????????????????????????????????????????????? -->
<div class="border-top mt-8 pt-4 mb-8 flex justify-between items-center">
    <p class="text-muted caption">EcoBeacon AI v2.0 · Documentation generated April 2026</p>
    <div class="flex gap-3">
        <span class="tag tag-green">React 18</span>
        <span class="tag tag-blue">Node.js</span>
        <span class="tag tag-purple">9 AI Features</span>
    </div>
</div>
</div>
</body>
</html>

```

3. README.md

? EcoBeacon AI

AI-powered sustainability platform that tracks, analyzes, and reduces your carbon footprint.

Quick Start

Prerequisites

- Node.js 18+
- MongoDB (local or MongoDB Atlas)

Backend Setup

```
```bash
cd backend
npm install
cp .env.example .env
Edit .env with your MongoDB URI and JWT secret
npm run dev
```
```

Frontend Setup

```
```bash
cd frontend
npm install
npm run dev
```
```

Open <http://localhost:5173>

Seed Demo Data (optional)

```
```bash
cd backend
npm run seed
Login: alex@ecobeacon.com / password123
```
```

Tech Stack

- **Frontend**: React 18, Tailwind CSS, Recharts, React Router v6, Axios
- **Backend**: Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs
- **AI**: Rule-based recommendation engine

API Endpoints

| Method | Endpoint | Description |
|--------|-----------------------------|---------------------------|
| POST | /api/auth/register | Register user |
| POST | /api/auth/login | Login |
| GET | /api/auth/profile | Get profile (auth) |
| POST | /api/activity/add | Log activity (auth) |
| GET | /api/activity/user/:id | Get activities (auth) |
| DELETE | /api/activity/:id | Delete activity (auth) |
| GET | /api/emissions/summary/:id | Emission summary (auth) |
| GET | /api/ai/suggestions/:userId | AI recommendations (auth) |

Emission Factors

| Category | Factor |
|----------|------------------------------|
| Car | 0.21 kg CO ₂ /km |
| Bus | 0.089 kg CO ₂ /km |
| Train | 0.041 kg CO ₂ /km |
| Flight | 0.255 kg CO ₂ /km |

```
Electricity	0.85 kg CO2/kWh
Meat meal	5.0 kg CO2
Veg meal	1.5 kg CO2
Vegan meal	0.9 kg CO2
```

4. backend/.env

[Binary or unsupported file type:] - content not embedded.

5. backend/.env.example

[Binary or unsupported file type: .example] - content not embedded.

6. backend/config/db.js

```
const sequelize = require('./sequelize');

const connectDB = async () => {
  try {
    await sequelize.authenticate();
    require('../models/User');
    require('../models/Activity');
    await sequelize.sync();
    console.log('? SQLite Connected: data/ecobeacon.sqlite');
  } catch (error) {
    console.error(`? Database connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

7. backend/config/sequelize.js

```
const { Sequelize } = require('sequelize');
const path = require('path');
const fs = require('fs');

const dbDir = path.join(__dirname, '../data');
if (!fs.existsSync(dbDir)) fs.mkdirSync(dbDir, { recursive: true });

const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: path.join(dbDir, 'ecobeacon.sqlite'),
  logging: false
});

module.exports = sequelize;
```

8. backend/controllers/activityController.js

```

const { Op } = require('sequelize');
const Activity = require('../models/Activity');
const User = require('../models/User');
const { calculateEmission } = require('../services/emissionService');
const { calculateEcoScore, getBadges, updateStreak } = require('../utils/ecoScore');

const addActivity = async (req, res) => {
  try {
    const { type, category, value, notes, date } = req.body;
    const userId = req.user.id;

    if (!type || !category || value === undefined) {
      return res.status(400).json({ message: 'Type, category, and value are required' });
    }
    if (parseFloat(value) < 0) {
      return res.status(400).json({ message: 'Value cannot be negative' });
    }

    let emission;
    try {
      emission = calculateEmission(type, category, parseFloat(value));
    } catch (err) {
      return res.status(400).json({ message: err.message });
    }

    const activity = await Activity.create({
      userId,
      type,
      category,
      value: parseFloat(value),
      emission,
      notes: notes || '',
      date: date ? new Date(date) : new Date()
    });

    const allActivities = await Activity.findAll({ where: { userId } });
    const newEcoScore = calculateEcoScore(allActivities);
    const badges = getBadges(newEcoScore, req.user.streak, allActivities.length);
    const streakUpdate = updateStreak(req.user);
    const totalEmission = allActivities.reduce((sum, a) => sum + a.emission, 0);

    await User.update({
      ecoScore: newEcoScore,
      totalEmission: parseFloat(totalEmission.toFixed(2)),
      badges: badges.map(b => b.name),
      streak: streakUpdate.streak,
      lastLogDate: streakUpdate.lastLogDate
    }, { where: { id: userId } });

    res.status(201).json({
      success: true,
      message: 'Activity logged successfully',
      activity,
      stats: {
        emission,
        newEcoScore,
        streak: streakUpdate.streak,
        badgesEarned: badges.map(b => b.name)
      }
    });
  } catch (error) {
    console.error('Add activity error:', error);
    res.status(500).json({ message: 'Error adding activity', error: error.message });
  }
};

```

```

const getUserActivities = async (req, res) => {
  try {
    const userId = parseInt(req.params.id);

    if (req.user.id !== userId) {
      return res.status(403).json({ message: 'Not authorized to access this data' });
    }

    const { limit = 50, page = 1, type, startDate, endDate } = req.query;
    const skip = (parseInt(page) - 1) * parseInt(limit);

    const where = { userId };
    if (type) where.type = type;
    if (startDate || endDate) {
      where.date = {};
      if (startDate) where.date[Op.gte] = new Date(startDate);
      if (endDate) where.date[Op.lte] = new Date(endDate);
    }

    const [activities, total] = await Promise.all([
      Activity.findAll({
        where,
        order: [['date', 'DESC']],
        offset: skip,
        limit: parseInt(limit)
      }),
      Activity.count({ where })
    ]);

    res.json({
      success: true,
      activities,
      pagination: {
        total,
        page: parseInt(page),
        limit: parseInt(limit),
        pages: Math.ceil(total / parseInt(limit))
      }
    });
  } catch (error) {
    res.status(500).json({ message: 'Error fetching activities', error: error.message });
  }
};

const deleteActivity = async (req, res) => {
  try {
    const activity = await Activity.findByPk(req.params.id);

    if (!activity) return res.status(404).json({ message: 'Activity not found' });
    if (activity.userId !== req.user.id) {
      return res.status(403).json({ message: 'Not authorized to delete this activity' });
    }

    await activity.destroy();

    const allActivities = await Activity.findAll({ where: { userId: req.user.id } });
    const newEcoScore = calculateEcoScore(allActivities);
    const totalEmission = allActivities.reduce((sum, a) => sum + a.emission, 0);

    await User.update({
      ecoScore: newEcoScore,
      totalEmission: parseFloat(totalEmission.toFixed(2))
    }, { where: { id: req.user.id } });

    res.json({ success: true, message: 'Activity deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Error deleting activity', error: error.message });
  }
}

```

```
};  
  
module.exports = { addActivity, getUserActivities, deleteActivity };
```

9. backend/controllers/aiController.js

```

const { Op } = require('sequelize');
const Activity = require('../models/Activity');
const {
  generateRecommendations,
  forecastEmissions,
  detectAnomalies,
  generateWeeklyGoals,
} = require('../services/aiService');

const getSuggestions = async (req, res) => {
  try {
    const userId = parseInt(req.params.userId);
    if (req.user.id !== userId) return res.status(403).json({ message: 'Not authorized' });

    const thirtyDaysAgo = new Date();
    thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

    const activities = await Activity.findAll({
      where: { userId, date: { [Op.gte]: thirtyDaysAgo } },
      order: [['date', 'DESC']],
    });

    const suggestions = generateRecommendations(activities, req.user.ecoScore || 50);

    res.json({ success: true, suggestions, analysisDate: new Date(), activitiesAnalyzed: activities.length });
  } catch (error) {
    console.error('AI suggestions error:', error);
    res.status(500).json({ message: 'Error generating suggestions', error: error.message });
  }
};

const getForecast = async (req, res) => {
  try {
    const userId = parseInt(req.params.userId);
    if (req.user.id !== userId) return res.status(403).json({ message: 'Not authorized' });

    const thirtyDaysAgo = new Date();
    thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

    const activities = await Activity.findAll({
      where: { userId, date: { [Op.gte]: thirtyDaysAgo } },
      order: [['date', 'ASC']],
    });

    const forecast = forecastEmissions(activities, 7);
    const goals = generateWeeklyGoals(activities, req.user.ecoScore || 50);

    res.json({ success: true, forecast, goals, generatedAt: new Date() });
  } catch (error) {
    console.error('Forecast error:', error);
    res.status(500).json({ message: 'Error generating forecast', error: error.message });
  }
};

const getAnomalies = async (req, res) => {
  try {
    const userId = parseInt(req.params.userId);
    if (req.user.id !== userId) return res.status(403).json({ message: 'Not authorized' });

    const thirtyDaysAgo = new Date();
    thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

    const activities = await Activity.findAll({
      where: { userId, date: { [Op.gte]: thirtyDaysAgo } },

```

```

    order: [['date', 'DESC']],
  });

  const anomalies = detectAnomalies(activities);

  res.json({ success: true, anomalies, scanned: activities.length });
} catch (error) {
  console.error('Anomaly detection error:', error);
  res.status(500).json({ message: 'Error detecting anomalies', error: error.message });
}
};

const chatResponse = (req, res) => {
  const { message } = req.body;
  if (!message) return res.status(400).json({ message: 'Message required' });

  const msg = message.toLowerCase();
  let reply = '';

  if (/biggest|largest|most emission/.test(msg)) {
    reply = "Transportation is typically the largest source of personal emissions (avg 45%), followed by food (25%), electricity (20%), and shopping (10%). Check your Activity Breakdown chart on the Dashboard for your personal split.";
  } else if (/reduce|lower|cut|improve/.test(msg) && /transport|car|driv/.test(msg)) {
    reply = "To cut transport emissions: switch to public transit or cycling for trips under 10 km, carpool when driving is unavoidable, and consider an electric vehicle for longer commutes. This alone can save 1-3 kg CO2 per day.";
  } else if (/reduce|lower|cut/.test(msg) && /food|eat|diet/.test(msg)) {
    reply = "A plant-based diet is one of the most impactful changes you can make. Replacing 3 beef meals/week with vegetarian options saves ~15 kg CO2 per month. Try 'Meatless Mondays' as a start!";
  } else if (/electricity|energy|power/.test(msg)) {
    reply = "Shift heavy appliances (washing machine, dishwasher) to off-peak hours (10 pm-7 am). Switch to LED bulbs, unplug standby devices, and consider a green energy tariff. Solar panels can cut home electricity emissions by up to 90%.";
  } else if (/score|eco score/.test(msg)) {
    reply = "Your Eco Score (0-100) reflects your emission efficiency. A score above 80 = Excellent, 50-79 = Moderate, below 50 = Needs Improvement. Log consistently and focus on high-impact changes to raise your score.";
  } else if (/badge|streak|achievement/.test(msg)) {
    reply = "Badges are awarded for milestones: first log, activity count, eco score thresholds, and streak length. Maintain daily logging streaks to unlock 'Eco Warrior' and 'Streak Master' badges!";
  } else if (/paris|climate|global|target/.test(msg)) {
    reply = "The Paris Agreement targets limiting warming to 1.5°C. This requires each person to emit no more than ~2.5 kg CO2/day by 2030 compared to the current world average of ~7.5 kg/day. You can track your progress against this target in the Impact Analysis panel.";
  } else if (/hello|hi|hey|help/.test(msg)) {
    reply = "Hello! I'm your EcoBeacon AI assistant. I can answer questions about reducing your carbon footprint, understanding your eco score, climate targets, and more. What would you like to know?";
  } else if (/flight|fly|travel|air/.test(msg)) {
    reply = "Aviation is one of the most carbon-intensive activities. A single long-haul flight can emit more CO2 than months of driving. Consider trains for trips under 500 km - they're up to 80% cleaner. If you must fly, choose direct routes and economy class.";
  } else if (/shopping|buy|purchase/.test(msg)) {
    reply = "Manufacturing goods accounts for a major share of global emissions. Tips: buy secondhand (90% less CO2 per item), choose quality over quantity, repair before replacing, and favor brands with sustainability certifications.";
  } else {
    reply = "Great question! I can help with topics like: reducing transport emissions, sustainable food choices, energy saving at home, understanding your eco score, climate targets, and more. Try asking me something specific!";
  }

  res.json({ success: true, reply, timestamp: new Date() });
};

```

```
module.exports = { getSuggestions, getForecast, getAnomalies, chatResponse };
```

10. backend/controllers/authController.js

```

const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const generateToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '30d' });

const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Please provide all required fields' });
    }
    if (password.length < 6) {
      return res.status(400).json({ message: 'Password must be at least 6 characters' });
    }

    const existing = await User.findOne({ where: { email: email.toLowerCase() } });
    if (existing) {
      return res.status(400).json({ message: 'An account with this email already exists' });
    }

    const salt = await bcrypt.genSalt(12);
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = await User.create({ name, email: email.toLowerCase(), password: hashedPassword });

    res.status(201).json({
      success: true,
      message: 'Account created successfully',
      user: {
        _id: user.id,
        name: user.name,
        email: user.email,
        ecoScore: user.ecoScore,
        totalEmission: user.totalEmission,
        badges: user.badges,
        streak: user.streak
      },
      token: generateToken(user.id)
    });
  } catch (error) {
    if (error.name === 'SequelizeUniqueConstraintError') {
      return res.status(400).json({ message: 'Email already in use' });
    }
    res.status(500).json({ message: 'Server error during registration', error: error.message });
  }
};

const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Please provide email and password' });
    }

    const user = await User.findOne({ where: { email: email.toLowerCase() } });
    if (!user) {
      return res.status(401).json({ message: 'Invalid email or password' });
    }

    const isMatch = await bcrypt.compare(password, user.password);
  }
};

```

```

    if (!isMatch) {
      return res.status(401).json({ message: 'Invalid email or password' });
    }

    res.json({
      success: true,
      message: 'Login successful',
      user: {
        _id: user.id,
        name: user.name,
        email: user.email,
        ecoScore: user.ecoScore,
        totalEmission: user.totalEmission,
        badges: user.badges,
        streak: user.streak
      },
      token: generateToken(user.id)
    });
  } catch (error) {
    res.status(500).json({ message: 'Server error during login', error: error.message });
  }
};

const getProfile = async (req, res) => {
  try {
    const user = await User.findByPk(req.user.id, { attributes: { exclude: ['password'] } });
    if (!user) return res.status(404).json({ message: 'User not found' });
    res.json({ success: true, user });
  } catch (error) {
    res.status(500).json({ message: 'Error fetching profile', error: error.message });
  }
};

const updateProfile = async (req, res) => {
  try {
    const { name } = req.body;
    const updates = {};
    if (name) updates.name = name;

    const user = await User.findByPk(req.user.id);
    if (!user) return res.status(404).json({ message: 'User not found' });

    await user.update(updates);

    const updated = await User.findByPk(req.user.id, { attributes: { exclude: ['password'] } });
    ;
    res.json({ success: true, user: updated });
  } catch (error) {
    res.status(500).json({ message: 'Error updating profile', error: error.message });
  }
};

module.exports = { register, login, getProfile, updateProfile };

```

11. backend/controllers/emissionController.js

```

const { Op } = require('sequelize');
const Activity = require('../models/Activity');

const getEmissionSummary = async (req, res) => {
  try {
    const userId = parseInt(req.params.id);

    if (req.user.id !== userId) {
      return res.status(403).json({ message: 'Not authorized' });
    }

    const now = new Date();
    const startOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate());
    const startOfWeek = new Date(now);
    startOfWeek.setDate(now.getDate() - 7);
    const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);

    const [dailyActs, weeklyActs, monthlyActs, allActs] = await Promise.all([
      Activity.findAll({ where: { userId, date: { [Op.gte]: startOfDay } } }),
      Activity.findAll({ where: { userId, date: { [Op.gte]: startOfWeek } } }),
      Activity.findAll({ where: { userId, date: { [Op.gte]: startOfMonth } } }),
      Activity.findAll({ where: { userId }, order: [['date', 'DESC']] })
    ]);

    const sumEmissions = (acts) => parseFloat(acts.reduce((s, a) => s + a.emission, 0).toFixed(
2));

    res.json({
      success: true,
      summary: {
        daily: sumEmissions(dailyActs),
        weekly: sumEmissions(weeklyActs),
        monthly: sumEmissions(monthlyActs),
        total: sumEmissions(allActs)
      },
      dailyBreakdown: getDailyBreakdown(weeklyActs, 7),
      monthlyBreakdown: getDailyBreakdown(monthlyActs, 30),
      categoryBreakdown: getCategoryBreakdown(monthlyActs),
      activityCount: {
        daily: dailyActs.length,
        weekly: weeklyActs.length,
        monthly: monthlyActs.length,
        total: allActs.length
      }
    });
  } catch (error) {
    res.status(500).json({ message: 'Error fetching emission summary', error: error.message });
  }
};

const getDailyBreakdown = (activities, days) => {
  const result = [];

  for (let i = days - 1; i >= 0; i--) {
    const date = new Date();
    date.setDate(date.getDate() - i);
    const dayStart = new Date(date.getFullYear(), date.getMonth(), date.getDate());
    const dayEnd = new Date(dayStart);
    dayEnd.setDate(dayEnd.getDate() + 1);

    const dayActs = activities.filter(a => {
      const d = new Date(a.date);
      return d >= dayStart && d < dayEnd;
    });
  }
};

```

```

const byType = (type) => parseFloat(
  dayActs.filter(a => a.type === type).reduce((s, a) => s + a.emission, 0).toFixed(2)
);

result.push({
  date: date.toISOString().split('T')[0],
  label: date.toLocaleDateString('en-US', { weekday: 'short', month: 'short', day: 'numeric'
' })),
  total: parseFloat(dayActs.reduce((s, a) => s + a.emission, 0).toFixed(2)),
  transportation: byType('transportation'),
  electricity: byType('electricity'),
  food: byType('food'),
  shopping: byType('shopping')
});
}

return result;
};

const getCategoryBreakdown = (activities) => {
  const totals = {};
  let grandTotal = 0;

  activities.forEach(a => {
    totals[a.type] = (totals[a.type] || 0) + a.emission;
    grandTotal += a.emission;
  });

  return Object.entries(totals).map(([name, value]) => ({
    name,
    value: parseFloat(value.toFixed(2)),
    percentage: grandTotal > 0 ? parseFloat(((value / grandTotal) * 100).toFixed(1)) : 0
  }));
};

module.exports = { getEmissionSummary };

```

12. backend/middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({ message: 'Not authorized, no token provided' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id, { attributes: { exclude: ['password'] } });

    if (!req.user) {
      return res.status(401).json({ message: 'User no longer exists' });
    }

    next();
  } catch (error) {
    return res.status(401).json({ message: 'Not authorized, invalid token' });
  }
};

module.exports = { protect };
```

13. backend/models/Activity.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/sequelize');

const Activity = sequelize.define('Activity', {
  userId: {
    type: DataTypes.INTEGER,
    allowNull: false
  },
  type: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      isIn: [['transportation', 'electricity', 'food', 'shopping']]
    }
  },
  category: {
    type: DataTypes.STRING,
    allowNull: false
  },
  value: {
    type: DataTypes.FLOAT,
    allowNull: false
  },
  unit: {
    type: DataTypes.STRING,
    defaultValue: ''
  },
  emission: {
    type: DataTypes.FLOAT,
    allowNull: false
  },
  notes: {
    type: DataTypes.STRING(200),
    defaultValue: ''
  },
  date: {
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW
  },
  _id: {
    type: DataTypes.VIRTUAL,
    get() { return this.id; }
  }
}, { timestamps: true });

module.exports = Activity;
```

14. backend/models/User.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/sequelize');

const User = sequelize.define('User', {
  name: {
    type: DataTypes.STRING(50),
    allowNull: false
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false
  },
  ecoScore: {
    type: DataTypes.FLOAT,
    defaultValue: 50
  },
  totalEmission: {
    type: DataTypes.FLOAT,
    defaultValue: 0
  },
  badges: {
    type: DataTypes.TEXT,
    defaultValue: '[]',
    get() {
      const val = this.getDataValue('badges');
      try { return JSON.parse(val || '[]'); } catch { return []; }
    },
    set(val) {
      this.setDataValue('badges', JSON.stringify(Array.isArray(val) ? val : []));
    }
  },
  streak: {
    type: DataTypes.INTEGER,
    defaultValue: 0
  },
  lastLogDate: {
    type: DataTypes.DATE,
    allowNull: true,
    defaultValue: null
  },
  _id: {
    type: DataTypes.VIRTUAL,
    get() { return this.id; }
  }
}, { timestamps: true });

module.exports = User;
```

15. backend/package.json

```
{
  "name": "ecobeacon-backend",
  "version": "1.0.0",
  "description": "EcoBeacon AI Backend - Carbon Footprint Tracker",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node seed/seedData.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.2",
    "sequelize": "^6.37.8",
    "sqlite3": "^5.1.7"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

16. backend/routes/activityRoutes.js

```
const express = require('express');
const router = express.Router();
const { addActivity, getUserActivities, deleteActivity } = require('../controllers/activityController');
const { protect } = require('../middleware/authMiddleware');

router.post('/add', protect, addActivity);
router.get('/user/:id', protect, getUserActivities);
router.delete('/:id', protect, deleteActivity);

module.exports = router;
```

17. backend/routes/aiRoutes.js

```
const express = require('express');
const router = express.Router();
const { getSuggestions, getForecast, getAnomalies, chatResponse } = require('../controllers/aiController');
const { protect } = require('../middleware/authMiddleware');

router.get('/suggestions/:userId', protect, getSuggestions);
router.get('/forecast/:userId', protect, getForecast);
router.get('/anomalies/:userId', protect, getAnomalies);
router.post('/chat', protect, chatResponse);

module.exports = router;
```

18. backend/routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const { register, login, getProfile, updateProfile } = require('../controllers/authController')
;
const { protect } = require('../middleware/authMiddleware');

router.post('/register', register);
router.post('/login', login);
router.get('/profile', protect, getProfile);
router.put('/profile', protect, updateProfile);

module.exports = router;
```

19. backend/routes/emissionRoutes.js

```
const express = require('express');
const router = express.Router();
const { getEmissionSummary } = require('../controllers/emissionController');
const { protect } = require('../middleware/authMiddleware');

router.get('/summary/:id', protect, getEmissionSummary);

module.exports = router;
```

20. backend/seed/seedData.js

```

require('dotenv').config({ path: '../.env' });
const bcrypt = require('bcryptjs');
const sequelize = require('../config/sequelize');
const User = require('../models/User');
const Activity = require('../models/Activity');

const getRandomDate = (daysBack) => {
  const date = new Date();
  date.setDate(date.getDate() - Math.floor(Math.random() * daysBack));
  return date;
};

const seedData = async () => {
  try {
    await sequelize.authenticate();
    await sequelize.sync({ force: true });
    console.log('? Connected to SQLite, tables reset');

    const salt = await bcrypt.genSalt(12);
    const hashedPassword = await bcrypt.hash('password123', salt);

    const users = await User.bulkCreate([
      { name: 'Alex Green', email: 'alex@ecobeacon.com', password: hashedPassword, ecoScore: 72, streak: 5 },
      { name: 'Sam Earth', email: 'sam@ecobeacon.com', password: hashedPassword, ecoScore: 85, streak: 12 },
      { name: 'Jordan Blue', email: 'jordan@ecobeacon.com', password: hashedPassword, ecoScore: 43, streak: 2 }
    ]);

    console.log(`? Created ${users.length} users`);

    const activityTemplates = [
      { type: 'transportation', category: 'car', value: 25, emission: 5.25, unit: 'km' },
      { type: 'transportation', category: 'bus', value: 15, emission: 1.335, unit: 'km' },
      { type: 'transportation', category: 'train', value: 40, emission: 1.64, unit: 'km' },
      { type: 'transportation', category: 'bike', value: 8, emission: 0.04, unit: 'km' },
      { type: 'transportation', category: 'walking', value: 3, emission: 0, unit: 'km' },
      { type: 'transportation', category: 'flight', value: 800, emission: 204, unit: 'km' },
      { type: 'electricity', category: 'standard', value: 8, emission: 6.8, unit: 'kWh' },
      { type: 'electricity', category: 'standard', value: 12, emission: 10.2, unit: 'kWh' },
      { type: 'electricity', category: 'solar', value: 10, emission: 0.41, unit: 'kWh' },
      { type: 'food', category: 'meat', value: 1, emission: 5.0, unit: 'meal' },
      { type: 'food', category: 'veg', value: 1, emission: 1.5, unit: 'meal' },
      { type: 'food', category: 'fastfood', value: 1, emission: 3.5, unit: 'meal' },
      { type: 'food', category: 'vegan', value: 1, emission: 0.9, unit: 'meal' },
      { type: 'food', category: 'beef', value: 1, emission: 6.61, unit: 'meal' },
      { type: 'shopping', category: 'normal', value: 2, emission: 4.0, unit: 'item' },
      { type: 'shopping', category: 'eco-friendly', value: 3, emission: 1.5, unit: 'item' }
    ];
  }
};

```

```

    tem' },
    { type: 'shopping',      category: 'high-impact', value: 1,  emission: 5.0,  unit: 'i
    tem' },
    { type: 'shopping',      category: 'secondhand',  value: 2,  emission: 0.4,  unit: 'i
    tem' }
  ];

  const activities = [];
  users.forEach(user => {
    for (let i = 0; i < 30; i++) {
      const template = activityTemplates[Math.floor(Math.random() * activityTemplates.length)
];
      activities.push({
        userId: user.id,
        ...template,
        notes: '',
        date: getRandomDate(30)
      });
    }
  });

  await Activity.bulkCreate(activities);
  console.log(`? Created ${activities.length} activities`);

  console.log('\n? Seed complete! Test credentials:');
  console.log('  Email: alex@ecobeacon.com | Password: password123');
  console.log('  Email: sam@ecobeacon.com  | Password: password123');

  process.exit(0);
} catch (error) {
  console.error('? Seed error:', error);
  process.exit(1);
}
};

seedData();

```

21. backend/server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();

const startServer = async () => {
  await connectDB();

  const app = express();

  app.use(cors({
    origin: process.env.CLIENT_URL || 'http://localhost:5173',
    credentials: true
  }));
  app.use(express.json({ limit: '10mb' }));
  app.use(express.urlencoded({ extended: true }));

  app.use('/api/auth', require('./routes/authRoutes'));
  app.use('/api/activity', require('./routes/activityRoutes'));
  app.use('/api/emissions', require('./routes/emissionRoutes'));
  app.use('/api/ai', require('./routes/aiRoutes'));

  app.get('/api/health', (req, res) => {
    res.json({ status: 'ok', message: 'EcoBeacon API is running', timestamp: new Date() });
  });

  app.use((req, res) => {
    res.status(404).json({ message: 'Route not found' });
  });

  app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ message: 'Server error', error: err.message });
  });

  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`? EcoBeacon server running on port ${PORT}`);
    console.log(`? API available at http://localhost:${PORT}/api`);
  });
};

startServer();
```

22. backend/services/aiService.js

```

const generateRecommendations = (activities, ecoScore) => {
  if (!activities || activities.length === 0) return getDefaultRecommendations();

  const byType = groupByType(activities);
  const suggestions = [];

  // Transportation analysis
  const transport = byType.transportation || [];
  if (transport.length > 0) {
    const carTrips = transport.filter(a => a.category === 'car');
    const flights = transport.filter(a => a.category === 'flight');
    const ecoTrips = transport.filter(a => ['walking', 'bike', 'train'].includes(a.category));

    if (carTrips.length >= 3) {
      const potentialSaving = (carTrips.reduce((s, a) => s + a.emission, 0) * 0.6).toFixed(1);
      suggestions.push({
        id: 'transport_car',
        type: 'transportation',
        priority: 'high',
        title: 'Switch to Public Transport',
        description: `You've taken ${carTrips.length} car trips recently. Using public transport or carpooling 3 days/week could save ~${potentialSaving} kg CO2.`,
        impact: 'high',
        icon: '?',
        savings: potentialSaving
      });
    }

    if (flights.length > 0) {
      const flightEmission = flights.reduce((s, a) => s + a.emission, 0);
      suggestions.push({
        id: 'transport_flight',
        type: 'transportation',
        priority: 'high',
        title: 'Reduce Air Travel',
        description: `Flights generated ${flightEmission.toFixed(1)} kg CO2. Consider trains for trips under 500km ? they can be 80% less polluting.`,
        impact: 'high',
        icon: '?',
        savings: (flightEmission * 0.8).toFixed(1)
      });
    }

    if (ecoTrips.length === 0 && transport.length >= 2) {
      suggestions.push({
        id: 'transport_eco',
        type: 'transportation',
        priority: 'medium',
        title: 'Try Walking or Cycling',
        description: "For trips under 5km, walking or cycling generates zero emissions and improves your health. Give it a try this week!",
        impact: 'medium',
        icon: '?',
        savings: '2-5'
      });
    }
  }

  // Electricity analysis
  const electricity = byType.electricity || [];
  if (electricity.length > 0) {
    const totalKwh = electricity.reduce((s, a) => s + a.value, 0);
    const totalEmission = electricity.reduce((s, a) => s + a.emission, 0);

    if (totalKwh > 15) {

```

```

    suggestions.push({
      id: 'electricity_peak',
      type: 'electricity',
      priority: 'high',
      title: 'Reduce Peak Hour Usage',
      description: `You've used ${totalKwh.toFixed(1)} kwh recently. Shifting energy-heavy tasks (laundry, dishwasher) to off-peak hours (10pm-7am) can reduce grid strain.`,
      impact: 'medium',
      icon: '?',
      savings: (totalEmission * 0.12).toFixed(1)
    });
  }

  suggestions.push({
    id: 'electricity_solar',
    type: 'electricity',
    priority: 'medium',
    title: 'Explore Renewable Energy',
    description: 'Switching to a green energy tariff or installing solar panels can cut electricity emissions by up to 90%.',
    impact: 'high',
    icon: '??',
    savings: (totalEmission * 0.9).toFixed(1)
  });
}

// Food analysis
const food = byType.food || [];
if (food.length > 0) {
  const meatMeals = food.filter(a => ['meat', 'beef', 'fastfood'].includes(a.category));
  const vegMeals = food.filter(a => ['veg', 'vegan'].includes(a.category));

  if (meatMeals.length > vegMeals.length) {
    const potentialSaving = (meatMeals.reduce((s, a) => s + a.emission, 0) * 0.65).toFixed(1);
    ;
    suggestions.push({
      id: 'food_meat',
      type: 'food',
      priority: 'high',
      title: 'Reduce Meat Consumption',
      description: `${meatMeals.length} meat meals logged. Replacing 3 meat meals per week with plant-based options could save ~${potentialSaving} kg CO2.`,
      impact: 'high',
      icon: '?',
      savings: potentialSaving
    });
  }

  const fastFoods = food.filter(a => a.category === 'fastfood');
  if (fastFoods.length >= 3) {
    suggestions.push({
      id: 'food_cooking',
      type: 'food',
      priority: 'medium',
      title: 'Cook More at Home',
      description: "Home-cooked vegetarian meals can have 60% lower emissions than fast food. Try batch cooking on Sundays!",
      impact: 'medium',
      icon: '?',
      savings: (fastFoods.reduce((s, a) => s + a.emission * 0.6, 0)).toFixed(1)
    });
  }
}

// Shopping analysis
const shopping = byType.shopping || [];
if (shopping.length > 0) {
  const highImpact = shopping.filter(a => a.category === 'high-impact');

```

```

const normalItems = shopping.filter(a => a.category === 'normal');

if (highImpact.length > 0) {
  suggestions.push({
    id: 'shopping_eco',
    type: 'shopping',
    priority: 'medium',
    title: 'Choose Eco-Friendly Products',
    description: `${highImpact.length} high-impact purchases detected. Choosing eco-friendly or secondhand alternatives can cut shopping emissions by 70%.`,
    impact: 'medium',
    icon: '??',
    savings: (highImpact.reduce((s, a) => s + a.emission * 0.7, 0)).toFixed(1)
  });
}

if (normalItems.length >= 3) {
  suggestions.push({
    id: 'shopping_secondhand',
    type: 'shopping',
    priority: 'low',
    title: 'Try Secondhand Shopping',
    description: "Buying secondhand reduces emissions by ~90% per item. Check local thrift stores, Vinted, or eBay for great finds.",
    impact: 'high',
    icon: '??',
    savings: (normalItems.reduce((s, a) => s + a.emission * 0.8, 0)).toFixed(1)
  });
}

// Score-based general advice
if (ecoScore < 40) {
  suggestions.unshift({
    id: 'general_improve',
    type: 'general',
    priority: 'high',
    title: 'Start Small ? Big Impact',
    description: "Your score has room to grow! Focus on your single biggest emission source first. Small, consistent changes create lasting habits.",
    impact: 'high',
    icon: '?',
    savings: '0'
  });
}

if (ecoScore >= 80) {
  suggestions.push({
    id: 'general_inspire',
    type: 'general',
    priority: 'low',
    title: 'Inspire Your Community',
    description: "You're leading by example! Share your eco journey with friends and family to multiply your positive impact.",
    impact: 'community',
    icon: '?',
    savings: '?'
  });
}

const priorityOrder = { high: 0, medium: 1, low: 2 };
return suggestions
  .sort((a, b) => (priorityOrder[a.priority] ?? 3) - (priorityOrder[b.priority] ?? 3))
  .slice(0, 6);
};

const groupByType = (activities) => {
  return activities.reduce((acc, a) => {

```

```

    if (!acc[a.type]) acc[a.type] = [];
    acc[a.type].push(a);
    return acc;
  }, {}));
};

const getDefaultRecommendations = () => [
  {
    id: 'default_transport',
    type: 'transportation',
    priority: 'high',
    title: 'Log Your Commute',
    description: "Transportation is typically the largest source of personal emissions. Start b
y logging your daily commute to understand your impact.",
    impact: 'high',
    icon: '?',
    savings: '0'
  },
  {
    id: 'default_food',
    type: 'food',
    priority: 'high',
    title: 'Track Your Meals',
    description: "Food choices account for ~25% of personal emissions. Log your meals daily to
identify opportunities to eat more sustainably.",
    impact: 'high',
    icon: '??',
    savings: '0'
  },
  {
    id: 'default_electricity',
    type: 'electricity',
    priority: 'medium',
    title: 'Monitor Energy Usage',
    description: "Tracking your electricity consumption helps you identify peak usage periods a
nd find ways to save both money and carbon.",
    impact: 'medium',
    icon: '?',
    savings: '0'
  }
];

// ?? Forecast: predict daily emissions for the next N days ???????????????????
const forecastEmissions = (activities, days = 7) => {
  if (!activities || activities.length === 0) {
    return Array.from({ length: days }, (_, i) => ({
      date: dayOffset(i + 1),
      predicted: 0,
      confidence: 'low',
    }));
  }
};

// Build a map of date ? total emission
const byDate = {};
activities.forEach(a => {
  const d = new Date(a.date).toISOString().split('T')[0];
  byDate[d] = (byDate[d] || 0) + (a.emission || 0);
});

const dailyValues = Object.values(byDate);
const avg = dailyValues.reduce((s, v) => s + v, 0) / dailyValues.length;

// Simple linear trend from first half ? second half
const half = Math.floor(dailyValues.length / 2);
const firstHalf = dailyValues.slice(0, half);
const secondHalf = dailyValues.slice(half);
const firstAvg = firstHalf.length ? firstHalf.reduce((s, v) => s + v, 0) / firstHalf.length :
avg;

```

```

const secondAvg = secondHalf.length ? secondHalf.reduce((s, v) => s + v, 0) / secondHalf.length : avg;
const trendDelta = (secondAvg - firstAvg) / Math.max(half, 1);

const confidence = dailyValues.length >= 14 ? 'high' : dailyValues.length >= 7 ? 'medium' : 'low';

return Array.from({ length: days }, (_, i) => ({
  date: dayOffset(i + 1),
  predicted: Math.max(0, parseFloat((avg + trendDelta * (i + 1)).toFixed(2))),
  confidence,
  trend: trendDelta > 0.1 ? 'increasing' : trendDelta < -0.1 ? 'decreasing' : 'stable',
})));
};

// ?? Anomaly detection: flag activities significantly above normal ??????????
const detectAnomalies = (activities) => {
  if (!activities || activities.length < 5) return [];

  const byType = groupByType(activities);
  const anomalies = [];

  Object.entries(byType).forEach(([type, acts]) => {
    const emissions = acts.map(a => a.emission || 0);
    const mean = emissions.reduce((s, v) => s + v, 0) / emissions.length;
    const variance = emissions.reduce((s, v) => s + Math.pow(v - mean, 2), 0) / emissions.length;
    const stdDev = Math.sqrt(variance);
    const threshold = mean + 2 * stdDev;

    acts.forEach(a => {
      if ((a.emission || 0) > threshold && (a.emission || 0) > mean * 2) {
        anomalies.push({
          activityId: a.id || a._id,
          type,
          category: a.category,
          emission: a.emission,
          average: parseFloat(mean.toFixed(2)),
          multiplier: parseFloat((a.emission / mean).toFixed(1)),
          date: a.date,
          message: `This ${type} activity emitted ${(a.emission || 0).toFixed(2)} kg CO2 ? ${(a.emission / mean).toFixed(1)}x your usual average of ${mean.toFixed(2)} kg.`
        });
      }
    });
  });

  return anomalies.slice(0, 5);
};

// ?? Weekly goal generator ?????????????????????????????????????????????????????????????
const generateWeeklyGoals = (activities, ecoScore) => {
  const byType = groupByType(activities);
  const goals = [];

  const transport = byType.transportation || [];
  const carTrips = transport.filter(a => a.category === 'car');
  if (carTrips.length > 0) {
    const weeklyCarKm = carTrips.reduce((s, a) => s + a.value, 0) / 4;
    goals.push({
      id: 'goal_car_km',
      title: 'Reduce Car Distance',
      target: Math.round(weeklyCarKm * 0.8),
      unit: 'km by car this week',
      current: Math.round(weeklyCarKm),
      saving: parseFloat((weeklyCarKm * 0.2 * 0.21).toFixed(1)),
      icon: '?',
    });
  }
};

```

```

}

const food = byType.food || [];
const meatMeals = food.filter(a => ['beef', 'meat', 'fastfood'].includes(a.category));
if (meatMeals.length > 0) {
  const weeklyMeat = Math.round(meatMeals.length / 4);
  goals.push({
    id: 'goal_meat_meals',
    title: 'Meat-Free Days',
    target: Math.min(weeklyMeat - 1, 3),
    unit: 'fewer meat meals this week',
    current: weeklyMeat,
    saving: parseFloat((meatMeals.reduce((s, a) => s + a.emission, 0) / 4 * 0.25).toFixed(1))
  },
  {
    icon: '?',
  });
}

const electricity = byType.electricity || [];
if (electricity.length > 0) {
  const weeklyKwh = electricity.reduce((s, a) => s + a.value, 0) / 4;
  goals.push({
    id: 'goal_electricity',
    title: 'Cut Electricity Use',
    target: Math.round(weeklyKwh * 0.9),
    unit: 'kWh this week',
    current: Math.round(weeklyKwh),
    saving: parseFloat((weeklyKwh * 0.1 * 0.85).toFixed(1)),
    icon: '?',
  });
}

if (ecoScore < 60) {
  goals.push({
    id: 'goal_eco_score',
    title: 'Boost Eco Score',
    target: Math.min(ecoScore + 10, 100),
    unit: 'eco score target',
    current: ecoScore,
    saving: 0,
    icon: '?',
  });
}

return goals.slice(0, 3);
};

const dayOffset = (n) => {
  const d = new Date();
  d.setDate(d.getDate() + n);
  return d.toISOString().split('T')[0];
};

module.exports = { generateRecommendations, forecastEmissions, detectAnomalies, generateWeeklyGoals };

```

23. backend/services/emissionService.js

```

const EMISSION_FACTORS = {
  transportation: {
    car: { factor: 0.21, unit: 'km', label: 'Car' },
    motorcycle: { factor: 0.103, unit: 'km', label: 'Motorcycle' },
    bus: { factor: 0.089, unit: 'km', label: 'Bus' },
    train: { factor: 0.041, unit: 'km', label: 'Train' },
    flight: { factor: 0.255, unit: 'km', label: 'Flight' },
    bike: { factor: 0.005, unit: 'km', label: 'Bicycle' },
    walking: { factor: 0, unit: 'km', label: 'Walking' },
    electric_car: { factor: 0.053, unit: 'km', label: 'Electric Car' }
  },
  electricity: {
    standard: { factor: 0.85, unit: 'kwh', label: 'Standard Grid' },
    solar: { factor: 0.041, unit: 'kwh', label: 'Solar' },
    wind: { factor: 0.011, unit: 'kwh', label: 'Wind Energy' }
  },
  food: {
    beef: { factor: 6.61, unit: 'meal', label: 'Beef Meal' },
    meat: { factor: 5.0, unit: 'meal', label: 'Meat Meal' },
    fish: { factor: 2.8, unit: 'meal', label: 'Fish Meal' },
    fastfood: { factor: 3.5, unit: 'meal', label: 'Fast Food' },
    dairy: { factor: 2.1, unit: 'meal', label: 'Dairy-based' },
    veg: { factor: 1.5, unit: 'meal', label: 'Vegetarian' },
    vegan: { factor: 0.9, unit: 'meal', label: 'Vegan' }
  },
  shopping: {
    'high-impact': { factor: 5.0, unit: 'item', label: 'High Impact' },
    normal: { factor: 2.0, unit: 'item', label: 'Normal' },
    'eco-friendly': { factor: 0.5, unit: 'item', label: 'Eco-Friendly' },
    secondhand: { factor: 0.2, unit: 'item', label: 'Secondhand' }
  }
};

const calculateEmission = (type, category, value) => {
  const typeFactors = EMISSION_FACTORS[type];
  if (!typeFactors) throw new Error(`Unknown activity type: ${type}`);

  const categoryData = typeFactors[category];
  if (!categoryData) throw new Error(`Unknown category '${category}' for type '${type}'`);

  return parseFloat((categoryData.factor * value).toFixed(4));
};

const getEmissionFactors = () => EMISSION_FACTORS;

const getCategoryInfo = (type, category) => EMISSION_FACTORS[type]?.[category] || null;

module.exports = { calculateEmission, getEmissionFactors, getCategoryInfo, EMISSION_FACTORS };

```

24. backend/utils/ecoScore.js

```

const calculateEcoScore = (activities) => {
  if (!activities || activities.length === 0) return 50;

  const totalEmission = activities.reduce((sum, a) => sum + a.emission, 0);
  const days = getUniqueDays(activities);
  const dailyAvg = totalEmission / Math.max(days, 1);

  let baseScore;
  if (dailyAvg <= 2) baseScore = 95;
  else if (dailyAvg <= 4) baseScore = 88;
  else if (dailyAvg <= 6) baseScore = 78;
  else if (dailyAvg <= 10) baseScore = 65;
  else if (dailyAvg <= 15) baseScore = 50;
  else if (dailyAvg <= 20) baseScore = 35;
  else if (dailyAvg <= 30) baseScore = 20;
  else baseScore = 10;

  const ecoFriendlyCount = activities.filter(a =>
    (a.type === 'transportation' && ['walking', 'bike', 'train'].includes(a.category)) ||
    (a.type === 'food' && ['veg', 'vegan'].includes(a.category)) ||
    (a.type === 'shopping' && ['eco-friendly', 'secondhand'].includes(a.category))
  ).length;

  const consistencyBonus = Math.min(10, Math.floor((ecoFriendlyCount / activities.length) * 15));
};

return Math.min(100, Math.max(0, Math.round(baseScore + consistencyBonus)));
};

const getUniqueDays = (activities) => {
  const days = new Set(activities.map(a => new Date(a.date).toString()));
  return days.size;
};

const getScoreLabel = (score) => {
  if (score >= 80) return { label: 'Excellent', color: '#10b981', emoji: '?' };
  if (score >= 50) return { label: 'Moderate', color: '#f59e0b', emoji: '?' };
  return { label: 'Needs Improvement', color: '#ef4444', emoji: '??' };
};

const getBadges = (ecoScore, streak, totalActivities) => {
  const badges = [];

  if (totalActivities >= 1) badges.push({ id: 'getting_started', name: 'Getting Started', icon: '?', desc: 'Logged first activity' });
  if (totalActivities >= 10) badges.push({ id: 'active_tracker', name: 'Active Tracker', icon: '?', desc: 'Logged 10+ activities' });
  if (totalActivities >= 50) badges.push({ id: 'dedicated', name: 'Dedicated Tracker', icon: '?', desc: 'Logged 50+ activities' });
  if (ecoScore >= 60) badges.push({ id: 'eco_aware', name: 'Eco Aware', icon: '?', desc: 'Score 60+' });
  if (ecoScore >= 80) badges.push({ id: 'eco_warrior', name: 'Eco Warrior', icon: '?', desc: 'Score 80+' });
  if (ecoScore >= 90) badges.push({ id: 'green_hero', name: 'Green Hero', icon: '?', desc: 'Score 90+' });
  if (streak >= 3) badges.push({ id: 'streak_3', name: '3-Day Streak', icon: '?', desc: '3 consecutive days' });
  if (streak >= 7) badges.push({ id: 'week_streak', name: 'Week Warrior', icon: '?', desc: '7-day streak' });
  if (streak >= 30) badges.push({ id: 'month_streak', name: 'Eco Champion', icon: '?', desc: '30-day streak' });

  return badges;
};

```

```
const updateStreak = (user) => {
  const now = new Date();
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());

  if (!user.lastLogDate) {
    return { streak: 1, lastLogDate: today };
  }

  const lastLog = new Date(user.lastLogDate);
  const lastLogDay = new Date(lastLog.getFullYear(), lastLog.getMonth(), lastLog.getDate());
  const diffDays = Math.floor((today - lastLogDay) / (1000 * 60 * 60 * 24));

  if (diffDays === 0) return { streak: user.streak, lastLogDate: user.lastLogDate };
  if (diffDays === 1) return { streak: user.streak + 1, lastLogDate: today };
  return { streak: 1, lastLogDate: today };
};

module.exports = { calculateEcoScore, getScoreLabel, getBadges, updateStreak };
```

25. documentation.txt

```
=====
                        ECOBEACON AI ? PROJECT DOCUMENTATION
=====
```

```
Version: 1.0
```

```
Date: 2026-04-29
```

```
-----
TABLE OF CONTENTS
```

- ```

1. Project Overview
2. Tech Stack
3. Directory Structure
4. Setup & Installation
5. Environment Variables
6. Database
7. Backend Architecture
 7.1 Entry Point (server.js)
 7.2 Configuration
 7.3 Models
 7.4 Middleware
 7.5 Routes
 7.6 Controllers
 7.7 Services
 7.8 Utilities
 7.9 Seed Data
8. Frontend Architecture
 8.1 Entry Point & App Shell
 8.2 Routing & Route Guards
 8.3 Auth Context
 8.4 API Service (Axios)
 8.5 Pages
 8.6 Components
 8.7 Utilities & Hooks
9. AI / Recommendation Engine
 9.1 Recommendation Generator
 9.2 Emission Forecaster
 9.3 Anomaly Detector
 9.4 Weekly Goal Generator
 9.5 Rule-Based Chat Bot
10. Emission Factor Reference
11. API Endpoint Reference
12. Eco Score & Gamification
13. Known Notes & Design Decisions
```

```
=====
1. PROJECT OVERVIEW
=====
```

EcoBeacon AI is a full-stack web application for personal carbon footprint tracking and reduction. Users log daily activities across four categories (transportation, electricity, food, shopping), receive precise CO2 emission calculations, and get AI-powered recommendations to reduce their environmental impact.

Key Features:

- Activity logging with real-time emission preview
- Natural language activity parser ("I drove 20km to work")
- GPS-based automatic trip tracker with transport mode detection
- Personalized AI recommendations, 7-day emission forecast, anomaly detection
- Eco Score (0-100) with streak tracking and achievement badges
- Interactive dashboard: charts, category breakdowns, forecasts
- Rule-based AI chat assistant
- Impact context (equivalencies: trees, phone charges, showers, etc.)

## 2. TECH STACK

Layer	Technology
Frontend	React 18, React Router v6, Tailwind CSS, Recharts, Axios
Backend	Node.js, Express 4, Sequelize 6, SQLite3
Auth	JWT (jsonwebtoken 30-day tokens), bcryptjs (12 salt rounds)
Build Tool	Vite 4
Dev Server	nodemon

NOTE: The README.md and .env.example reference MongoDB/Mongoose. The actual implementation uses SQLite + Sequelize. No external database server is required.

## 3. DIRECTORY STRUCTURE

```

Project/
??? README.md
??? documentation.txtx <- This file
??? EcoBeacon_AI_Documentation.html <- Pre-built HTML documentation
?
??? backend/
? ??? server.js <- Express entry point
? ??? package.json
? ??? .env / .env.example
? ?
? ??? config/
? ? ??? db.js <- Sequelize authenticate + sync
? ? ??? sequelize.js <- SQLite Sequelize instance
? ?
? ??? models/
? ? ??? User.js
? ? ??? Activity.js
? ?
? ??? controllers/
? ? ??? authController.js
? ? ??? activityController.js
? ? ??? emissionController.js
? ? ??? aiController.js
? ?
? ??? routes/
? ? ??? authRoutes.js
? ? ??? activityRoutes.js
? ? ??? emissionRoutes.js
? ? ??? aiRoutes.js
? ?
? ??? middleware/
? ? ??? authMiddleware.js <- JWT protect middleware
? ?
? ??? services/
? ? ??? emissionService.js <- Emission factor table + calculator
? ? ??? aiService.js <- Recommendation, forecast, anomaly engine
? ?
? ??? utils/
? ? ??? ecoScore.js <- Score calculation, badges, streak logic
? ?
? ??? seed/
? ? ??? seedData.js <- 3 demo users + 30 activities each
? ?
? ??? data/
? ??? ecobeacon.sqlite <- Auto-created SQLite database file
?
??? frontend/
 ??? index.html

```

```

??? package.json
??? vite.config.js <- Dev server on :5173, proxies /api ? :5000
??? tailwind.config.js
??? postcss.config.js
??? src/
 ??? main.jsx <- React 18 createRoot entry
 ??? App.jsx <- Router + route guards
 ??? index.css
 ?
 ??? context/
 ? ??? AuthContext.jsx <- Global auth state
 ?
 ??? services/
 ? ??? api.js <- Axios instance, token injection, 401 handler
 ?
 ??? hooks/
 ? ??? useGeoTracker.js <- GPS watch, distance, speed, mode detection
 ?
 ??? utils/
 ? ??? helpers.js <- Activity config, emission factors, formatters
 ? ??? geoTracking.js <- Haversine distance, transport mode classifier
 ?
 ??? pages/
 ? ??? Login.jsx
 ? ??? Register.jsx
 ? ??? Dashboard.jsx
 ? ??? LogActivity.jsx
 ? ??? Profile.jsx
 ?
 ??? components/
 ??? Navbar.jsx
 ??? LoadingSpinner.jsx
 ??? StatsCard.jsx
 ??? EcoScoreGauge.jsx
 ??? EmissionChart.jsx
 ??? ActivityBreakdown.jsx
 ??? SuggestionsPanel.jsx
 ??? ForecastPanel.jsx
 ??? BadgesPanel.jsx
 ??? AIChat.jsx
 ??? NLPActivityParser.jsx
 ??? AutoTracker.jsx
 ??? ImpactContext.jsx

```

#### 4. SETUP & INSTALLATION

##### Prerequisites

-----

- Node.js 18+
- No database server needed (SQLite is file-based)

##### Backend Setup

-----

```

cd backend
npm install
cp .env.example .env
Edit .env: set JWT_SECRET to a random secret string
npm run dev # Starts on http://localhost:5000

```

##### Frontend Setup

-----

```

cd frontend
npm install
npm run dev # Starts on http://localhost:5173

```

Seed Demo Data (optional)

```

cd backend
npm run seed

Demo accounts created:
alex@ecobeacon.com / password123
sam@ecobeacon.com / password123
jordan@ecobeacon.com / password123

WARNING: The seed script calls sequelize.sync({ force: true }), which drops
and recreates all tables. Do not run on a database with real user data.

=====
5. ENVIRONMENT VARIABLES
=====

File: backend/.env (copy from backend/.env.example)

Variable | Default | Description
-----+-----+-----
PORT | 5000 | Express server port
JWT_SECRET | (required) | Secret for signing JWTs
CLIENT_URL | http://localhost:5173 | Allowed CORS origin
MONGO_URI | (unused ? legacy leftover) | Not consumed by the app
OPENAI_API_KEY | (unused ? legacy leftover) | Not consumed; AI is rule-based

=====
6. DATABASE
=====

Engine: SQLite (file at backend/data/ecobeacon.sqlite)
ORM: Sequelize 6

Tables are auto-created via sequelize.sync() on server startup. No migrations
are required. The data directory is created automatically if it doesn't exist.

Users Table

Column | Type | Notes
-----+-----+-----
id | INTEGER | Auto-increment primary key
name | STRING | Max 50 chars, required
email | STRING | Unique, required, stored lowercase
password | STRING | bcrypt hash (12 rounds)
ecoScore | FLOAT | 0-100, default 50
totalEmission | FLOAT | Cumulative kg CO2, default 0
badges | TEXT | JSON-serialized array of badge objects
streak | INTEGER | Consecutive days logged, default 0
lastLogDate | DATE | Date of last logged activity
createdAt | DATE | Auto-managed by Sequelize
updatedAt | DATE | Auto-managed by Sequelize

Activities Table

Column | Type | Notes
-----+-----+-----
id | INTEGER | Auto-increment primary key
userId | INTEGER | References users.id (no FK constraint)
type | STRING | One of: transportation, electricity, food, shopping
category | STRING | e.g., car, bus, beef, solar, secondhand
value | FLOAT | Quantity (km, kwh, meals, items)
unit | STRING | Display unit, default ''
emission | FLOAT | Pre-calculated kg CO2
notes | STRING | Optional, max 200 chars
date | DATE | Defaults to now
createdAt | DATE | Auto-managed
updatedAt | DATE | Auto-managed

```

Both models expose a virtual `_id` field (alias for `id`) for frontend compatibility.

## 7. BACKEND ARCHITECTURE

### 7.1 ENTRY POINT ? backend/server.js

The `startServer()` async function:

1. Calls `connectDB()` ? authenticates SQLite, syncs Sequelize models
2. Creates Express app with:
  - CORS (allowed origin: `CLIENT_URL` env var)
  - JSON body parsing (10 MB limit)
  - URL-encoded body parsing
3. Mounts routes:
  - `/api/auth` ? `authRoutes`
  - `/api/activity` ? `activityRoutes`
  - `/api/emissions` ? `emissionRoutes`
  - `/api/ai` ? `aiRoutes`
4. Health check: `GET /api/health` ? { status: 'OK', timestamp }
5. 404 handler and global error handler
6. Listens on PORT (default 5000)

### 7.2 CONFIGURATION

backend/config/sequelize.js

Creates the Sequelize instance using SQLite dialect.  
Stores the database at `backend/data/ecobeacon.sqlite`.  
Ensures the `data/` directory exists using `fs.mkdirSync`.

backend/config/db.js

Calls `sequelize.authenticate()` to verify the connection.  
Requires both model files (so Sequelize registers them).  
Calls `sequelize.sync()` to create/update tables.  
Calls `process.exit(1)` on failure.

### 7.3 MODELS

backend/models/User.js

Sequelize model. Notable details:

- password is never returned in API responses (excluded in queries)
- badges field uses a custom getter (`JSON.parse`) and setter (`JSON.stringify`) so the controller always works with a plain JS array
- `_id` virtual field returns `this.id` for MongoDB-style frontend compatibility

backend/models/Activity.js

Sequelize model.

- type is validated against the enum: `[transportation, electricity, food, shopping]`
- emission is stored at creation and is not recalculated on read
- `_id` virtual field returns `this.id`

### 7.4 MIDDLEWARE

backend/middleware/authMiddleware.js ? exports `protect()`

Steps:

1. Read Authorization header, extract Bearer token
2. Return 401 if no token
3. Verify token with `jwt.verify(token, JWT_SECRET)`
4. Find user by `decoded.id`, excluding the password field
5. Return 401 if user not found or token invalid
6. Attach user to `req.user` and call `next()`

### 7.5 ROUTES

Auth Routes (backend/routes/authRoutes.js)

POST `/api/auth/register` ? `register` (public)  
POST `/api/auth/login` ? `login` (public)

```
GET /api/auth/profile ? getProfile (protected)
PUT /api/auth/profile ? updateProfile (protected)
```

#### Activity Routes (backend/routes/activityRoutes.js)

```
POST /api/activity/add ? addActivity (protected)
GET /api/activity/user/:id ? getUserActivities (protected)
DELETE /api/activity/:id ? deleteActivity (protected)
```

#### Emission Routes (backend/routes/emissionRoutes.js)

```
GET /api/emissions/summary/:id ? getEmissionSummary (protected)
```

#### AI Routes (backend/routes/aiRoutes.js)

```
GET /api/ai/suggestions/:userId ? getSuggestions (protected)
GET /api/ai/forecast/:userId ? getForecast (protected)
GET /api/ai/anomalies/:userId ? getAnomalies (protected)
POST /api/ai/chat ? chatResponse (protected)
```

### 7.6 CONTROLLERS

-----

#### authController.js

```
register(req, res)
- Validates: name, email, password all present
- Checks for duplicate email (returns 400 if taken)
- Hashes password: bcrypt.hash(password, 12)
- Creates user with ecoScore=50, streak=0, badges=[]
- Signs JWT with 30-day expiry
- Returns: { user: {...}, token }
```

```
login(req, res)
- Finds user by email (lowercase)
- Compares password with bcrypt.compare
- Signs JWT, returns { user: {...}, token }
```

```
getProfile(req, res)
- Returns req.user (already fetched by authMiddleware)
```

```
updateProfile(req, res)
- Only accepts name updates currently
- Updates and returns the refreshed user
```

#### activityController.js

```
addActivity(req, res)
- Validates type, category, value (value must be > 0)
- Calls calculateEmission(type, category, value) from emissionService
- Creates Activity record
- Fetches all user activities to recalculate stats
- Calls calculateEcoScore, getBadges, updateStreak from ecoScore utils
- Updates User record (ecoScore, totalEmission, badges, streak, lastLogDate)
- Returns: { activity, ecoScore, totalEmission, streak, badges }
```

```
getUserActivities(req, res)
- Authorization: req.user.id must match req.params.id
- Query params: page (default 1), limit (default 20), type, startDate, endDate
- Returns: { activities, pagination: { total, page, pages } }
```

```
deleteActivity(req, res)
- Deletes activity by id
- Recalculates and saves new ecoScore and totalEmission for the user
- Returns: { message, ecoScore, totalEmission }
```

#### emissionController.js

```
getEmissionSummary(req, res)
- Authorization check: user can only fetch their own summary
- Runs 4 parallel queries: today / last 7 days / current month / all time
- Returns:
 summary: { daily, weekly, monthly, total } in kg CO2
 dailyBreakdown: 7-day array, each with total + per-type breakdown
 monthlyBreakdown: 30-day array (same shape)
```

```
categoryBreakdown: per-type totals with percentage (current month)
activityCount: { today, week, month, total }
```

#### aiController.js

```
getSuggestions(req, res)
- Fetches activities from the last 30 days
- Calls generateRecommendations(activities, ecoScore)
- Returns up to 6 suggestions

getForecast(req, res)
- Fetches activities from the last 30 days
- Calls forecastEmissions(activities, 7) and generateWeeklyGoals(activities, ecoScore)
- Returns: { forecast: [...], goals: [...] }

getAnomalies(req, res)
- Fetches activities from the last 30 days
- Calls detectAnomalies(activities)
- Returns: { anomalies: [...] }

chatResponse(req, res)
- Regex-based keyword dispatcher on req.body.message
- No activity data used; returns pre-written string responses
- See Section 9.5 for full keyword mapping
```

#### 7.7 SERVICES

-----

##### backend/services/emissionService.js

EMISSION\_FACTORS: hardcoded lookup table (see Section 10 for full table)

```
calculateEmission(type, category, value)
- Looks up the factor for type+category
- Returns +(factor * value).toFixed(4) as a float
- Returns 0 if type or category is not found
```

##### backend/services/aiService.js

See Section 9 for the full AI engine documentation.

#### 7.8 UTILITIES

-----

##### backend/utlis/ecoScore.js

```
calculateEcoScore(activities)
- Computes daily average emission across unique logged days
- Maps average to a base score:
 ? 2 kg/day ? 95 ? 4 ? 88 ? 6 ? 78 ? 8 ? 68
 ? 12 ? 56 ? 16 ? 44 ? 20 ? 33 ? 25 ? 22 ? 30 ? 15
 > 30 ? 10
- Adds consistency bonus (0?10 points): proportion of eco-friendly activities
 (walking, bike, train, veg/vegan meals, eco-friendly/secondhand shopping)
- Clamps final score to 0?100
```

```
getScoreLabel(score)
- score ? 80 ? "Excellent"
- score ? 50 ? "Moderate"
- else ? "Needs Improvement"
```

```
getBadges(ecoScore, streak, totalActivities)
- Activity count milestones: 1 (First Step), 10 (Getting Started), 50 (Eco Warrior)
- Score thresholds: 60 (Green Thinker), 80 (Climate Hero), 90 (Eco Legend)
- Streak milestones: 3 days (3-Day Streak), 7 days (Week Warrior),
 30 days (Monthly Master)
```

```
updateStreak(user)
- Compares lastLogDate to today (date only, no time)
- Same day ? no change (already logged today)
- Next day ? streak + 1
- Gap ? 2 days ? reset to 1 (streak broken)
- Returns { streak, lastLogDate }
```

## 7.9 SEED DATA ? backend/seed/seedData.js

- Calls sequelize.sync({ force: true }) ? DROPS all tables first
- Creates 3 demo users: alex, sam, jordan (all password: password123)
- Creates 30 activities per user randomly sampled from 18 templates:
  - car 20km, flight 1000km, bus 15km, train 50km, bike 10km,
  - electric car 30km, standard electricity 5kWh, solar electricity 10kWh,
  - beef meal, meat meal, veg meal, vegan meal, fish meal, fast food,
  - high-impact shopping, normal shopping, eco-friendly shopping, secondhand
- Activities are dated randomly across the past 30 days
- Eco scores and badges are calculated and saved after all activities are create

## 8. FRONTEND ARCHITECTURE

### 8.1 ENTRY POINT & APP SHELL

frontend/index.html

Standard Vite HTML shell. Title: "EcoBeacon AI". Root: <div id="root">.

frontend/src/main.jsx

React 18 createRoot. Mounts <App /> in StrictMode.

frontend/vite.config.js

Dev server port: 5173

Proxy: all /api/\* requests ? http://localhost:5000

This allows api.js to use baseUrl: '/api' without hardcoding the backend port.

frontend/tailwind.config.js

Custom color palette: eco.50 through eco.950 (green shades)

Custom animations:

spin-slow ? slow rotation

pulse-slow ? slow pulse

fade-in ? slides up 10px + opacity 0?1 over 0.5s

### 8.2 ROUTING & ROUTE GUARDS ? frontend/src/App.jsx

Wraps the entire app in <BrowserRouter> and <AuthProvider>.

ProtectedRoute component:

- Shows fullscreen LoadingSpinner while AuthContext is initializing
- Redirects to /login if user is null

PublicRoute component:

- Redirects to /dashboard if user is already authenticated

Route table:

```

/ ? redirect to /dashboard
/login ? <Login /> (PublicRoute)
/register ? <Register /> (PublicRoute)
/dashboard ? <Dashboard /> (ProtectedRoute)
/log ? <LogActivity /> (ProtectedRoute)
/profile ? <Profile /> (ProtectedRoute)
* ? redirect to /dashboard

```

The <AIChat /> floating chatbot is rendered globally outside the Routes block so it persists across page navigations. It is only shown when user is set.

### 8.3 AUTH CONTEXT ? frontend/src/context/AuthContext.jsx

Provides global auth state via React Context + useContext hook.

State:

```

user ? authenticated user object or null
token ? JWT string (initialized from localStorage 'eco_token' on mount)
loading ? true while the initial profile fetch is in-flight

```

## Methods:

```

login(email, password)
 POST /auth/login ? stores token in localStorage + sets user state

register(name, email, password)
 POST /auth/register ? same as login

logout()
 Clears 'eco_token' from localStorage, sets user and token to null

refreshUser()
 GET /auth/profile ? updates user state
 Called after logging an activity so the navbar eco score reflects the change

```

On mount, if a token exists in localStorage, it calls GET /auth/profile to restore the session. Sets loading = false when done.

## 8.4 API SERVICE ? frontend/src/services/api.js

-----  
 Axios instance configuration:

```

 baseURL: '/api' (proxied by Vite to http://localhost:5000)
 timeout: 10000ms

```

Request interceptor:

```

 Reads 'eco_token' from localStorage.
 If found: adds Authorization: Bearer <token> header.

```

Response interceptor (error handling):

```

 On 401: removes 'eco_token' from localStorage, redirects to /login.
 All other errors: re-thrown for the caller to handle.

```

## 8.5 PAGES

-----  
 Login.jsx

```

 Split-screen layout on large screens.
 Left: branding + 3 feature highlights.
 Right: email/password form + demo credentials shown inline.
 On submit: calls useAuth().login, navigates to /dashboard.

```

Register.jsx

```

 Centered card with name, email, password, confirm-password fields.
 Client-side validation: passwords must match, minimum 6 characters.
 On submit: calls useAuth().register.

```

Dashboard.jsx

Main logged-in view. On mount, runs 3 API calls in parallel:

```

 GET /emissions/summary/:id ? stats cards + charts
 GET /activity/user/:id?limit=10 ? recent activity list
 GET /ai/suggestions/:id ? SuggestionsPanel

```

Layout (responsive grid):

- Header: greeting + "Log Activity" button
- 4 StatsCards: Today / This Week / This Month / Eco Score
- EmissionChart (2/3 width) + EcoScoreGauge (1/3 width)
- ActivityBreakdown pie + SuggestionsPanel
- ForecastPanel + BadgesPanel
- Recent Activities list (last 6 items)

LogActivity.jsx

Activity logging form, top to bottom:

1. NLPActivityParser ? natural language input
2. AutoTracker ? GPS tracker (only shown when type = transportation)
3. Activity type selector: 4 buttons (transportation/electricity/food/shopping)
4. Category grid (context-sensitive to selected type)
5. Value input with:
  - Real-time emission preview
  - Paris Agreement budget badge (% of 2.5 kg/day target)
  - Smart pattern tip (compares value to user's average for that category)
  - Expandable ImpactContext equivalency panel

6. Date picker + notes textarea

7. Submit / Cancel buttons

On success: shows celebration screen with new eco score + streak info, then redirects to /dashboard after 2.5 seconds.

#### Profile.jsx

Shows: avatar (initial letter colored by eco score), name (inline-editable), email, score badge, streak badge, and 4 summary stats.

Left column: BadgesPanel

Right column: Activity history (15 per page, paginated)

Delete button appears on hover over each activity row.

Delete triggers a confirmation modal, recalculates stats server-side.

### 8.6 COMPONENTS

-----

#### Navbar.jsx

Fixed top bar. Logo + nav links (Dashboard / Log Activity / Profile).

Active link is highlighted. Shows user name + eco score label.

Mobile: hamburger menu with dropdown.

#### LoadingSpinner.jsx

Sizes: sm / md / lg.

fullScreen=true: renders a full-screen dark overlay with the leaf icon and "Loading EcoBeacon..." text.

#### StatsCard.jsx

Reusable metric card.

Props: title, value, unit, icon, color, subtitle, trend (optional up/down arrow).

#### EcoScoreGauge.jsx

SVG arc gauge (custom, not a chart library).

240-degree arc with colored fill proportional to score.

Tick marks at 0, 25, 50, 75, 100.

Center: score number + label.

Color: green (?80), amber (?50), red (otherwise).

Below gauge: score band legend.

#### EmissionChart.jsx

Recharts AreaChart ? 7-day daily emission breakdown.

Toggle: "Stacked" (4 areas, one per type) | "Total" (single green area).

Custom tooltip: shows per-category values + total.

#### ActivityBreakdown.jsx

Recharts PieChart (donut) ? monthly emission split by category.

Percentage labels inside segments (hidden if < 5%).

Below chart: 2-column grid of category cards with value and percentage.

#### SuggestionsPanel.jsx

Lists up to 6 AI suggestions.

Color-coded by priority: red = high, amber = medium, grey = low.

Clicking a card expands it to show full description + potential CO2 saving.

#### ForecastPanel.jsx

Calls GET /ai/forecast/:userId on mount.

Shows trend icon (? / ? / ?) + confidence level (high/medium/low).

Recharts AreaChart of 7-day predicted emissions.

Below chart: AI-generated weekly goals.

#### BadgesPanel.jsx

Earned badges shown as 3-column gradient card grid, each with a glow color.

Locked placeholder shown when fewer than 4 badges are earned.

Streak displayed as a pill in the panel header.

#### AIChat.jsx

Floating button (bottom-right) toggles a 480px-tall chat window.

Message state is local to the component (not persisted).

Quick-question chips above the input for common topics.

Sends POST /api/ai/chat with the message text.

Typing indicator: 3 bouncing dots while awaiting response.

#### NLPActivityParser.jsx

Contains 40+ regex rules organized by activity type.  
 parseActivity(text) iterates rules, returns {type, category, value} on first match.  
 Collapsible example phrases panel.  
 Parsed result shows a preview with an "Apply" button that pre-fills the form.  
 Supported intents: car, flight, bike, walk, bus, train, motorcycle, EV,  
 standard/solar/wind electricity, beef, meat, fastfood, veg, vegan, dairy,  
 fish, shopping (items/secondhand/eco-friendly/electronics).

#### AutoTracker.jsx

UI shell for the useGeoTracker hook.  
 Controls: Start / Stop / Reset.  
 Live stats: distance, speed, duration.  
 "AI Detected Mode" badge showing classified transport type.  
 On stop: "Use This Trip" button fills in distance + mode in the form.

#### ImpactContext.jsx

Given emissionKg, renders 4 equivalency cards:  
 Trees needed:  $\text{emissionKg} \div 0.058 \text{ kg/day}$   
 Phone charges:  $\text{emissionKg} \div 0.008 \text{ kg}$   
 LED TV hours:  $\text{emissionKg} \div 0.1 \text{ kg/hr}$   
 Hot showers:  $\text{emissionKg} \div 0.5 \text{ kg}$   
 Two progress bars:  
 % of world average daily budget (7.5 kg/day)  
 % of Paris Agreement target (2.5 kg/day)  
 Both bars are color-coded: green / amber / red.

### 8.7 UTILITIES & HOOKS

#### frontend/src/utlis/helpers.js

ACTIVITY\_CONFIG ? full category config with labels, icons, units for the UI  
 EMISSION\_FACTORS ? client-side copy of the backend emission factor table  
 estimateEmission(type, category, value) ? live emission preview calculation  
 formatEmission(value) ? returns string with 2 decimal places  
 getScoreColor(score) / getScoreLabel(score) / getScoreEmoji(score)  
 getCategoryColor(type) / getCategoryIcon(type)

#### frontend/src/utlis/geoTracking.js

haversineDistance(lat1, lon1, lat2, lon2)  
 Great-circle distance in km between two GPS coordinates.

classifyTransportMode(speedHistory)  
 Weighted average of rolling speed window (recent readings weighted higher).  
 Classification thresholds:  
 < 1 km/h ? stationary  
 < 7 km/h ? walking  
 < 25 km/h ? cycling  
 < 60 km/h ? bus / city driving  
 < 130 km/h ? car  
 < 350 km/h ? train  
 ? 350 km/h ? flight

formatDistance(km) / formatSpeed(kmh)

#### frontend/src/hooks/useGeoTracker.js

Custom hook wrapping navigator.geolocation.watchPosition.  
 State machine: idle ? requesting ? tracking ? stopped (or error)  
 GPS jitter filter: only accumulates distance when movement > 5 meters.  
 Rolling speed window of 8 readings for transport classifier.  
 1-second interval timer for duration display.  
 Exposed API:  
 status, distanceKm, currentSpeedKmh, detectedMode, duration, errorMsg  
 start(), stop(), reset()

### 9. AI / RECOMMENDATION ENGINE

```
=====
```

All AI logic is in backend/services/aiService.js and the chat handler in backend/controllers/aiController.js. No external ML library or API is used. All logic is rule-based JavaScript operating on the user's last 30 days of data.

#### 9.1 RECOMMENDATION GENERATOR ? generateRecommendations(activities, ecoScore)

-----

Groups activities by type using groupByType(), then applies rules:

##### Transportation Rules:

```
? 3 car trips ? "Switch to Public Transport" (HIGH)
 saving = 60% of total car emissions
Any flights ? "Reduce Air Travel" (HIGH)
 saving = 80% of flight emissions
No eco trips but
? 2 transport logs ? "Try Walking or Cycling" (MEDIUM)
```

##### Electricity Rules:

```
Total kWh > 15 ? "Reduce Peak Hour Usage" (HIGH)
 saving = 12% of electricity emissions
Any electricity ? "Explore Renewable Energy" (MEDIUM)
 saving = 90% of electricity emissions
```

##### Food Rules:

```
meat count >
veg count ? "Reduce Meat Consumption" (HIGH)
 saving = 65% of meat emissions
? 3 fast food ? "Cook More at Home" (MEDIUM)
 saving = 60% of fast food emissions
```

##### Shopping Rules:

```
Any high-impact ? "Choose Eco-Friendly Products" (MEDIUM)
 saving = 70% of high-impact emissions
? 3 normal items ? "Try Secondhand Shopping" (LOW)
 saving = 80% of normal shopping emissions
```

##### Score-based Rules:

```
ecoScore < 40 ? prepends "Start Small ? Big Impact" (HIGH)
ecoScore ? 80 ? appends "Inspire Your Community" (LOW)
```

Sorting & limit: sorted HIGH ? MEDIUM ? LOW, truncated to 6 suggestions.

No data: 3 default onboarding suggestions returned.

#### 9.2 EMISSION FORECASTER ? forecastEmissions(activities, days=7)

-----

1. Groups activities by date, builds a daily total map.
2. Calculates overall mean daily emission.
3. Splits day-series into two halves; computes average of each half.
4. Linear trend delta = (second half avg - first half avg) per day.
5. Generates N future date entries:
 

```
predicted[i] = max(0, mean + trendDelta * i)
```
6. Confidence:
 

```
? 14 data days ? "high"
? 7 data days ? "medium"
else ? "low"
```
7. Trend label:
 

```
delta > 0.1 ? "increasing"
delta < -0.1 ? "decreasing"
else ? "stable"
```

Returns [] if no activity data exists.

#### 9.3 ANOMALY DETECTOR ? detectAnomalies(activities)

-----

Requires ? 5 activities. For each activity type independently:

1. Compute mean and standard deviation of emissions in that type.
2. Flag any activity where:
 

```
emission > mean + 2*stdDev AND emission > mean * 2
```

3. Attach message: "This [category] trip was 4.2× your usual average"  
Returns maximum 5 anomalies total.

9.4 WEEKLY GOAL GENERATOR ? generateWeeklyGoals(activities, ecoScore)

-----  
Car travel:

Weekly avg km = total car km / 4  
Target = 80% of avg  
Saving = (avg - target) \* 0.21 kg/km

Meat meals:

Weekly avg count = total meat count / 4  
Target = max(0, avg - 1) (reduce by one meal per week, capped at 3)  
Saving = 25% of weekly meat emissions

Electricity:

Weekly avg kWh = total kWh / 4  
Target = 90% of avg  
Saving = (avg - target) \* 0.85 kg/kWh

ecoScore < 60:

Adds "Boost Eco Score by 10 Points" goal.

Returns maximum 3 goals.

9.5 RULE-BASED CHAT BOT ? chatResponse in aiController.js

-----  
Regex keyword dispatcher on lowercased message text:

Pattern	Topic
/biggest largest most emission/	Emission sources breakdown
/reduce.*transport car driv/	Transport reduction tips
/reduce.*food eat diet/	Food and diet tips
/electricity energy power/	Energy efficiency tips
/score eco score/	Eco Score explanation
/badge streak achievement/	Gamification explanation
/paris climate global target/	Paris Agreement context
/hello hi hey help/	Greeting
/flight fly travel air/	Aviation tips
/shopping buy purchase/	Shopping tips
(no match)	Generic "I can help with..." fallback

## =====

### 10. EMISSION FACTOR REFERENCE

## =====

Type	Category	Factor	Unit
transportation	car	0.210 kg	per km
transportation	motorcycle	0.103 kg	per km
transportation	bus	0.089 kg	per km
transportation	train	0.041 kg	per km
transportation	flight	0.255 kg	per km
transportation	bike	0.005 kg	per km
transportation	walking	0.000 kg	per km
transportation	electric_car	0.053 kg	per km
electricity	standard	0.850 kg	per kWh
electricity	solar	0.041 kg	per kWh
electricity	wind	0.011 kg	per kWh
food	beef	6.610 kg	per meal
food	meat	5.000 kg	per meal
food	fish	2.800 kg	per meal
food	fastfood	3.500 kg	per meal
food	dairy	2.100 kg	per meal
food	veg	1.500 kg	per meal
food	vegan	0.900 kg	per meal
shopping	high-impact	5.000 kg	per item

shopping	normal	2.000 kg	per item
shopping	eco-friendly	0.500 kg	per item
shopping	secondhand	0.200 kg	per item

The same table is duplicated on the frontend (frontend/src/utils/helpers.js) for live emission previews in the LogActivity form.

## 11. API ENDPOINT REFERENCE

All authenticated endpoints require: Authorization: Bearer <token>

### AUTH

----

POST /api/auth/register

Body: { name, email, password }

Returns: { user, token }

Errors: 400 (missing fields), 400 (email taken)

POST /api/auth/login

Body: { email, password }

Returns: { user, token }

Errors: 400 (invalid credentials)

GET /api/auth/profile [auth required]

Returns: { user }

PUT /api/auth/profile [auth required]

Body: { name }

Returns: { user }

### ACTIVITIES

-----

POST /api/activity/add [auth required]

Body: { type, category, value, notes?, date? }

Returns: { activity, ecoScore, totalEmission, streak, badges }

Errors: 400 (validation), 400 (unknown category)

GET /api/activity/user/:id [auth required]

Query params: page, limit, type, startDate, endDate

Returns: { activities, pagination }

Errors: 403 (accessing another user's data)

DELETE /api/activity/:id [auth required]

Returns: { message, ecoScore, totalEmission }

Errors: 404 (not found), 403 (not owner)

### EMISSIONS

-----

GET /api/emissions/summary/:id [auth required]

Returns:

summary: { daily, weekly, monthly, total }

dailyBreakdown: array[7] ? { date, total, transportation, electricity, food, shopping }

monthlyBreakdown: array[30] ? same shape

categoryBreakdown: array ? { type, total, percentage }

activityCount: { today, week, month, total }

Errors: 403 (accessing another user's data)

### AI

--

GET /api/ai/suggestions/:userId [auth required]

Returns: { suggestions: [{ title, description, priority, potentialSaving, category }] }

GET /api/ai/forecast/:userId [auth required]

Returns:

forecast: [{ date, predicted }]

goals: [{ title, target, currentAvg, unit, potentialSaving }]

```

trend: "increasing" | "decreasing" | "stable"
confidence:"high" | "medium" | "low"

```

```

GET /api/ai/anomalies/:userId [auth required]
Returns: { anomalies: [{ ...activity, message }] }

```

```

POST /api/ai/chat [auth required]
Body: { message: string }
Returns: { reply: string }

```

HEALTH

-----

```

GET /api/health
Returns: { status: "OK", timestamp }

```

## 12. ECO SCORE & GAMIFICATION

### ECO SCORE (0?100)

-----

Base score derived from average daily emission (all-time):

? 2 kg/day	? 95		? 12 ? 56		? 25 ? 22
? 4	? 88		? 16 ? 44		? 30 ? 15
? 6	? 78		? 20 ? 33		> 30 ? 10
? 8	? 68				

Consistency bonus (0?10): proportion of eco-friendly activities:

Eco-friendly types: walking, bike, train, veg/vegan meals,  
eco-friendly and secondhand shopping

Final score = base + bonus, clamped to [0, 100].

Score labels:

```

? 80 ? "Excellent"
? 50 ? "Moderate"
else ? "Needs Improvement"

```

### BADGES

-----

Activity Count:

```

1 activity ? "First Step"
10 activities ? "Getting Started"
50 activities ? "Eco Warrior"

```

Eco Score:

```

score ? 60 ? "Green Thinker"
score ? 80 ? "Climate Hero"
score ? 90 ? "Eco Legend"

```

Streak:

```

3 days ? "3-Day Streak"
7 days ? "Week Warrior"
30 days ? "Monthly Master"

```

### STREAKS

-----

A streak is the number of consecutive calendar days on which at least one activity has been logged.

- Logging on the same day does not increment the streak.
- Logging on the next calendar day increments streak by 1.
- Any gap of ? 2 days resets the streak to 1.

## 13. KNOWN NOTES & DESIGN DECISIONS

SQLite over MongoDB

Despite README.md and .env.example referencing MongoDB/Mongoose, the actual implementation uses SQLite + Sequelize. This makes the app zero-infrastructure ? no database server is needed; the .sqlite file is created automatically.

#### `_id` Virtual Fields

Both User and Activity models expose a `_id` virtual field aliasing the integer primary key. This allows frontend code written in a MongoDB-compatible style to work transparently with Sequelize integer PKs.

#### Duplicate Emission Factors

EMISSION\_FACTORS is defined in both backend/services/emissionService.js and frontend/src/utils/helpers.js. The backend is authoritative for stored values; the frontend copy enables live previews without an API roundtrip.

#### No External AI

OPENAI\_API\_KEY appears in .env.example but is never consumed. All "AI" features (NLP parser, GPS classifier, forecasting, anomaly detection, chat) are built from scratch as rule-based JavaScript logic.

#### JWT in localStorage

Token key: 'eco\_token'. The Axios response interceptor automatically clears it and redirects to /login on any 401 response.

#### User Authorization Pattern

Both activityController and emissionController explicitly check `req.user.id === parseInt(req.params.id)` before returning data. This prevents one authenticated user from accessing another user's data via URL manipulation.

#### Seed Script Warning

seedData.js uses `sequelize.sync({ force: true })`, which drops and recreates all tables. Never run `npm run seed` on a database with real production data.

=====

END OF DOCUMENTATION

=====

## 26. frontend/index.html

---

```
<!DOCTYPE html>
<html lang="en">
 <head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/svg+xml" href="/eco-icon.svg" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <meta name="description" content="EcoBeacon AI - Track and reduce your carbon footprint with AI-powered insights" />
 <title>EcoBeacon AI ?</title>
 </head>
 <body>
 <div id="root"></div>
 <script type="module" src="/src/main.jsx"></script>
 </body>
</html>
```

## 27. frontend/package.json

---

```
{
 "name": "ecobeacon-frontend",
 "version": "1.0.0",
 "description": "EcoBeacon AI Frontend - Carbon Footprint Tracker",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview"
 },
 "dependencies": {
 "axios": "^1.6.0",
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-router-dom": "^6.18.0",
 "recharts": "^2.9.0"
 },
 "devDependencies": {
 "@types/react": "^18.2.0",
 "@types/react-dom": "^18.2.0",
 "@vitejs/plugin-react": "^4.1.0",
 "autoprefixer": "^10.4.16",
 "postcss": "^8.4.31",
 "tailwindcss": "^3.3.5",
 "vite": "^4.5.0"
 }
}
```

## 28. frontend/postcss.config.js

---

```
export default {
 plugins: {
 tailwindcss: {},
 autoprefixer: {}
 }
};
```

## 29. frontend/src/App.jsx

---

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from '../context/AuthContext';
import Login from '../pages/Login';
import Register from '../pages/Register';
import Dashboard from '../pages/Dashboard';
import LogActivity from '../pages/LogActivity';
import Profile from '../pages/Profile';
import LoadingSpinner from '../components/LoadingSpinner';
import AIChat from '../components/AIChat';

const ProtectedRoute = ({ children }) => {
 const { user, loading } = useAuth();
 if (loading) return <LoadingSpinner fullScreen />;
 return user ? children : <Navigate to="/login" replace />;
};

const PublicRoute = ({ children }) => {
 const { user, loading } = useAuth();
 if (loading) return <LoadingSpinner fullScreen />;
 return !user ? children : <Navigate to="/dashboard" replace />;
};

function AppRoutes() {
 const { user } = useAuth();
 return (
 <>
 <Routes>
 <Route path="/" element={<Navigate to="/dashboard" replace />} />
 <Route path="/login" element={<PublicRoute><Login /></PublicRoute>} />
 <Route path="/register" element={<PublicRoute><Register /></PublicRoute>} />
 <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
 <Route path="/log" element={<ProtectedRoute><LogActivity /></ProtectedRoute>} />
 <Route path="/profile" element={<ProtectedRoute><Profile /></ProtectedRoute>} />
 <Route path="*" element={<Navigate to="/dashboard" replace />} />
 </Routes>
 {user && <AIChat />}
 </>
);
}

export default function App() {
 return (
 <BrowserRouter>
 <AuthProvider>
 <AppRoutes />
 </AuthProvider>
 </BrowserRouter>
);
}
```

### 30. frontend/src/components/AIChat.jsx

```
import React, { useState, useRef, useEffect } from 'react';
import api from '../services/api';

const QUICK_QUESTIONS = [
 'What is my biggest emission source?',
 'How can I reduce car emissions?',
 'What is the Paris Agreement target?',
 'How do I improve my eco score?',
 'Tips for sustainable food choices',
 'How do badges work?',
];

export default function AIChat() {
 const [open, setOpen] = useState(false);
 const [messages, setMessages] = useState([
 { role: 'assistant', text: "Hi! I'm your EcoBeacon AI assistant ? Ask me anything about reducing your carbon footprint or understanding your eco data." }
]);
 const [input, setInput] = useState('');
 const [loading, setLoading] = useState(false);
 const bottomRef = useRef(null);

 useEffect(() => {
 if (open) bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
 }, [messages, open]);

 const sendMessage = async (text) => {
 const msg = text || input.trim();
 if (!msg) return;
 setInput('');
 setMessages(m => [...m, { role: 'user', text: msg }]);
 setLoading(true);
 try {
 const { data } = await api.post('/ai/chat', { message: msg });
 setMessages(m => [...m, { role: 'assistant', text: data.reply }]);
 } catch {
 setMessages(m => [...m, { role: 'assistant', text: 'Sorry, I could not connect right now. Try again shortly.' }]);
 } finally {
 setLoading(false);
 }
 };

 return (
 <>
 </> Floating button </>
 <button
 onClick={() => setOpen(o => !o)}
 className="fixed bottom-6 right-6 z-50 w-14 h-14 rounded-full bg-eco-600 hover:bg-eco-500 text-white text-2xl shadow-lg shadow-eco-900/50 flex items-center justify-center transition-all duration-200 hover:scale-110"
 title="AI Eco Assistant"
 >
 {open ? '?' : ''}
 </button>

 </> Chat window </>
 <open && (
 <div className="fixed bottom-24 right-6 z-50 w-80 sm:w-96 rounded-2xl border border-slate-700 bg-slate-900 shadow-2xl flex flex-col overflow-hidden"
 style={{ height: '480px' }}>
 </> Header </>
 <div className="flex items-center gap-3 px-4 py-3 bg-eco-900/60 border-b border-slate-700">
```

```

?
<div>
 <p className="text-white text-sm font-semibold">EcoBeacon AI</p>
 <p className="text-eco-400 text-xs">Sustainability assistant</p>
</div>

 Online

</div>

{/* Messages */}
<div className="flex-1 overflow-y-auto px-4 py-3 space-y-3">
 {messages.map((m, i) => (
 <div key={i} className={`flex ${m.role === 'user' ? 'justify-end' : 'justify-start'}
t'}``>
 <div className={`max-w-[85%] rounded-2xl px-3.5 py-2.5 text-sm leading-relaxed
$ {
 m.role === 'user'
 ? 'bg-eco-700 text-white rounded-br-sm'
 : 'bg-slate-800 text-slate-200 rounded-bl-sm'
 }`}>
 {m.text}
 </div>
 </div>
))}
 {loading && (
 <div className="flex justify-start">
 <div className="bg-slate-800 rounded-2xl rounded-bl-sm px-4 py-3 flex gap-1">
 {[0, 1, 2].map(d => (
 <span key={d} className="w-2 h-2 rounded-full bg-slate-500 animate-bounce"
 style={{ animationDelay: `${d * 150}ms` }} />
))}
 </div>
 </div>
)}
 <div ref={bottomRef} />
</div>

{/* Quick questions */}
<div className="px-4 py-2 border-t border-slate-800 flex gap-2 overflow-x-auto scroll
bar-hide">
 {QUICK_QUESTIONS.map(q => (
 <button key={q} type="button" onClick={() => sendMessage(q)}
 className="text-xs whitespace-nowrap px-3 py-1.5 bg-slate-800 hover:bg-slate-70
0 text-slate-300 rounded-full border border-slate-700 transition-colors flex-shrink-0">
 {q}
 </button>
))}
</div>

{/* Input */}
<div className="flex gap-2 px-4 py-3 border-t border-slate-800">
 <input
 type="text"
 value={input}
 onChange={e => setInput(e.target.value)}
 onKeyDown={e => e.key === 'Enter' && !loading && sendMessage()}
 placeholder="Ask about sustainability..."
 className="flex-1 bg-slate-800 border border-slate-700 rounded-xl px-3 py-2 text-
sm text-white placeholder-slate-500 focus:outline-none focus:border-eco-500 transition-colors"
 />
 <button
 type="button"
 onClick={() => sendMessage()}
 disabled={loading || !input.trim()}
 className="px-3 py-2 bg-eco-600 hover:bg-eco-500 disabled:opacity-50 text-white r

```

```
ounded-xl transition-colors"
 >
 ?
 </button>
 </div>
</div>
)}
</>
);
}
```

## 31. frontend/src/components/ActivityBreakdown.jsx

```
import React from 'react';
import { PieChart, Pie, Cell, Tooltip, ResponsiveContainer, Legend } from 'recharts';
import { getCategoryColor, getCategoryIcon } from '../utils/helpers';

const RADIAN = Math.PI / 180;
const renderCustomLabel = ({ cx, cy, midAngle, innerRadius, outerRadius, percentage }) => {
 if (percentage < 5) return null;
 const radius = innerRadius + (outerRadius - innerRadius) * 0.5;
 const x = cx + radius * Math.cos(-midAngle * RADIAN);
 const y = cy + radius * Math.sin(-midAngle * RADIAN);
 return (
 <text x={x} y={y} fill="white" textAnchor="middle" dominantBaseline="central" fontSize="12"
 fontWeight="bold">
 {`${percentage.toFixed(0)}%`}
 </text>
);
};

const CustomTooltip = ({ active, payload }) => {
 if (!active || !payload?.length) return null;
 const d = payload[0].payload;
 return (
 <div className="bg-slate-800 border border-slate-700 rounded-xl p-3 shadow-xl">
 <div className="flex items-center gap-2">
 {getCategoryIcon(d.name)}
 {d.name}
 </div>
 <p className="text-eco-400 font-bold mt-1">{d.value} kg CO?</p>
 <p className="text-slate-400 text-xs">{d.percentage}% of total</p>
 </div>
);
};

export default function ActivityBreakdown({ data = [], loading = false }) {
 if (loading) {
 return (
 <div className="card-hover h-72 flex items-center justify-center">
 <div className="h-8 w-8 border-2 border-eco-500 border-t-transparent rounded-full animate-spin" />
 </div>
);
 }

 const hasData = data.length > 0 && data.some(d => d.value > 0);

 return (
 <div className="card-hover">
 <div className="mb-6">
 <h3 className="font-semibold text-white">Activity Breakdown</h3>
 <p className="text-slate-500 text-xs mt-0.5">Monthly emissions by category</p>
 </div>

 {!hasData ? (
 <div className="h-56 flex flex-col items-center justify-center text-slate-500 gap-3">
 ?
 <p className="text-sm">No activities logged yet</p>
 </div>
) : (
 <>
 <ResponsiveContainer width="100%" height={200}>
 <PieChart>
 <Pie
 data={data}
 cx="50%"

```

```

 cy="50%"
 innerRadius={55}
 outerRadius={85}
 paddingAngle={3}
 dataKey="value"
 labelLine={false}
 label={renderCustomLabel}
 >
 {data.map((entry) => (
 <Cell key={entry.name} fill={getCategoryColor(entry.name)}
 style={{ filter: `drop-shadow(0 0 4px ${getCategoryColor(entry.name)})50
 00/50">
))}
 </Pie>
 <Tooltip content={<CustomTooltip />} />
 </PieChart>
 </ResponsiveContainer>

 <div className="grid grid-cols-2 gap-2 mt-4">
 {data.map(item => (
 <div key={item.name} className="flex items-center gap-2 p-2 rounded-lg bg-slate-8
ame}</p>
 {getCategoryIcon(item.name)}
 <div className="flex-1 min-w-0">
 <p className="text-slate-300 text-xs capitalize font-medium truncate">{item.n
 <p className="text-white text-xs font-bold">{item.value} kg</p>
 </div>
 <span className="text-xs font-semibold px-1.5 py-0.5 rounded-md"
 style={{ background: `${getCategoryColor(item.name)}20`, color: getCatego
ryColor(item.name) }}>
 {item.percentage}%

 </div>
))}
 </div>
</>
)}
</div>
);
}

```

## 32. frontend/src/components/AutoTracker.jsx

```
import React from 'react';
import useGeoTracker from '../hooks/useGeoTracker';
import { formatDistance, formatSpeed } from '../utils/geoTracking';

const formatDuration = (secs) => {
 const m = Math.floor(secs / 60).toString().padStart(2, '0');
 const s = (secs % 60).toString().padStart(2, '0');
 return `${m}:${s}`;
};

export default function AutoTracker({ onApply }) {
 const tracker = useGeoTracker();

 const handleApply = () => {
 if (!tracker.detectedMode || tracker.distanceKm === 0) return;
 onApply({
 distanceKm: parseFloat(tracker.distanceKm.toFixed(3)),
 mode: tracker.detectedMode.mode
 });
 tracker.reset();
 };

 return (
 <div className="card mb-4 border border-blue-700/40 bg-blue-950/20">
 {/* Header */}
 <div className="flex items-center justify-between mb-3">
 <div className="flex items-center gap-2">
 ?
 <div>
 <p className="text-white font-semibold text-sm">AI Distance Tracker</p>
 <p className="text-slate-400 text-xs">Automatically detects distance & transport mo
de</p>
 </div>
 </div>
 {tracker.status === 'tracking' && (
 <span className="flex items-center gap-1.5 text-xs text-eco-400 bg-eco-500/10 px-2 py
-1 rounded-full">

 Live

)}
 </div>

 {/* Error */}
 {tracker.status === 'error' && (
 <div className="mb-3 p-3 bg-red-500/10 border border-red-700/30 rounded-xl text-red-400
text-sm">
 {tracker.errorMsg}
 </div>
)}

 {/* Tracking stats */}
 {(tracker.status === 'tracking' || tracker.status === 'stopped') && (
 <div className="grid grid-cols-3 gap-2 mb-4">
 <div className="bg-slate-800/60 rounded-xl p-3 text-center">
 <p className="text-xs text-slate-400 mb-1">Distance</p>
 <p className="text-white font-bold text-base">{formatDistance(tracker.distanceKm)}<
/p>
 </div>
 <div className="bg-slate-800/60 rounded-xl p-3 text-center">
 <p className="text-xs text-slate-400 mb-1">Speed</p>
 <p className="text-white font-bold text-base">
 {tracker.status === 'tracking' ? formatSpeed(tracker.currentSpeedKmh) : '?'}
 </p>
 </div>
 </div>
)}
 </div>
);
}
```

```

 </div>
 <div className="bg-slate-800/60 rounded-xl p-3 text-center">
 <p className="text-xs text-slate-400 mb-1">Duration</p>
 <p className="text-white font-bold text-base">{formatDuration(tracker.duration)}</p>
 </div>
 </div>
)}

{/* AI detected mode */}
{tracker.detectedMode && (
 <div className="flex items-center gap-2 mb-4 px-3 py-2 bg-slate-800/50 rounded-xl border border-slate-700/50">
 {tracker.detectedMode.icon}
 <div>
 <p className="text-xs text-slate-400">AI Detected Mode</p>
 <p className={`text-sm font-semibold ${tracker.detectedMode.color}`}>
 {tracker.detectedMode.label}
 </p>
 </div>

 auto-detected

 </div>
)}

{/* Controls */}
<div className="flex gap-2">
 {tracker.status === 'idle' || tracker.status === 'error' ? (
 <button
 type="button"
 onClick={tracker.start}
 className="flex-1 flex items-center justify-center gap-2 py-2.5 rounded-xl bg-blue-600 hover:bg-blue-500 text-white text-sm font-medium transition-all"
 >
 ? Start Tracking
 </button>
) : tracker.status === 'requesting' ? (
 <button type="button" disabled className="flex-1 py-2.5 rounded-xl bg-slate-700 text-slate-400 text-sm cursor-not-allowed">

 Requesting GPS...

 </button>
) : tracker.status === 'tracking' ? (
 <>
 <button
 type="button"
 onClick={tracker.stop}
 className="flex-1 py-2.5 rounded-xl bg-red-600 hover:bg-red-500 text-white text-sm font-medium transition-all"
 >
 ? Stop
 </button>
 <button
 type="button"
 onClick={tracker.reset}
 className="px-4 py-2.5 rounded-xl bg-slate-700 hover:bg-slate-600 text-slate-300 text-sm transition-all"
 >
 Reset
 </button>
 </>
) : tracker.status === 'stopped' ? (

```

```

<>
 <button
 type="button"
 onClick={handleApply}
 disabled={!tracker.detectedMode || tracker.distanceKm === 0}
 className="flex-1 py-2.5 rounded-xl bg-eco-600 hover:bg-eco-500 disabled:bg-slate
-700 disabled:text-slate-500 text-white text-sm font-medium transition-all"
 >
 ? Use This Trip ({formatDistance(tracker.distanceKm)})
 </button>
 <button
 type="button"
 onClick={tracker.reset}
 className="px-4 py-2.5 rounded-xl bg-slate-700 hover:bg-slate-600 text-slate-300
text-sm transition-all"
 >
 Discard
 </button>
 </>
) : null}
</div>
</div>
);
}

```

### 33. frontend/src/components/BadgesPanel.jsx

```
import React from 'react';

const BADGE_DATA = {
 'Getting Started': { icon: '?', color: 'from-slate-600 to-slate-700', glow: '#94a3b8' },
 'Active Tracker': { icon: '?', color: 'from-blue-800 to-blue-900', glow: '#3b82f6' },
 'Dedicated Tracker': { icon: '?', color: 'from-amber-700 to-amber-900', glow: '#f59e0b' },
 'Eco Aware': { icon: '?', color: 'from-eco-700 to-eco-900', glow: '#22c55e' },
 'Eco Warrior': { icon: '?', color: 'from-eco-600 to-eco-800', glow: '#16a34a' },
 'Green Hero': { icon: '?', color: 'from-emerald-500 to-emerald-800', glow: '#10b981' },
},
 '3-Day Streak': { icon: '?', color: 'from-orange-700 to-orange-900', glow: '#f97316' },
 'Week Warrior': { icon: '?', color: 'from-yellow-600 to-yellow-900', glow: '#eab308' },
 'Eco Champion': { icon: '?', color: 'from-amber-500 to-amber-800', glow: '#d97706' },
};

export default function BadgesPanel({ badges = [], streak = 0 }) {
 return (
 <div className="card-hover">
 <div className="flex items-center justify-between mb-5">
 <div>
 <h3 className="font-semibold text-white">Achievements</h3>
 <p className="text-slate-500 text-xs mt-0.5">Earn badges by staying consistent</p>
 </div>
 {streak > 0 && (
 <div className="flex items-center gap-2 px-3 py-1.5 bg-orange-500/15 border border-or
ange-700/30 rounded-xl">
 ?
 <div>
 <p className="text-orange-400 font-bold text-sm leading-none">{streak}</p>
 <p className="text-orange-500 text-xs leading-none">day streak</p>
 </div>
 </div>
)}
 </div>

 {badges.length === 0 ? (
 <div className="flex flex-col items-center justify-center py-6 text-slate-500 gap-2">
 ?
 <p className="text-sm">Log activities to earn your first badge!</p>
 </div>
) : (
 <div className="grid grid-cols-3 gap-3">
 {badges.map((badgeName) => {
 const bd = BADGE_DATA[badgeName] || { icon: '?', color: 'from-slate-700 to-slate-90
0', glow: '#64748b' };
 return (
 <div
 key={badgeName}
 className={`flex flex-col items-center gap-2 p-3 rounded-2xl bg-gradient-to-b $
{bd.color} border border-white/5 group cursor-default`}
 style={{ boxShadow: `0 0 12px ${bd.glow}20` }}
 >
 <span className="text-2xl group-hover:scale-110 transition-transform duration-2
00">{bd.icon}
 <p className="text-xs text-center text-slate-300 font-medium leading-tight">{ba
dgeName}</p>
 </div>
);
 })}
 </div>
)}

 {/* Locked badges hint */}
 {badges.length < 4 && (
```

```
 <div className="flex flex-col items-center gap-2 p-3 rounded-2xl bg-slate-800/50 border border-slate-700/30 border-dashed">
 ?
 <p className="text-xs text-center text-slate-600 leading-tight">More to unlock</p>
 >
 </div>
)}
 </div>
)}
</div>
);
}
```

### 34. frontend/src/components/EcoScoreGauge.jsx

```
import React from 'react';
import { getScoreColor, getScoreLabel, getScoreEmoji } from '../utils/helpers';

export default function EcoScoreGauge({ score = 50 }) {
 const safeScore = Math.min(100, Math.max(0, score));
 const color = getScoreColor(safeScore);
 const label = getScoreLabel(safeScore);
 const emoji = getScoreEmoji(safeScore);

 // SVG arc parameters
 const radius = 80;
 const strokeWidth = 14;
 const cx = 110;
 const cy = 100;
 const startAngle = -210;
 const endAngle = 30;
 const totalAngle = endAngle - startAngle;

 const polarToCartesian = (angle) => {
 const rad = (angle * Math.PI) / 180;
 return {
 x: cx + radius * Math.cos(rad),
 y: cy + radius * Math.sin(rad)
 };
 };

 const describeArc = (start, end) => {
 const s = polarToCartesian(start);
 const e = polarToCartesian(end);
 const largeArc = end - start > 180 ? 1 : 0;
 return `M ${s.x} ${s.y} A ${radius} ${radius} 0 ${largeArc} 1 ${e.x} ${e.y}`;
 };

 const scoreAngle = startAngle + (safeScore / 100) * totalAngle;
 const needle = polarToCartesian(scoreAngle);

 const ticks = [0, 25, 50, 75, 100];

 return (
 <div className="card-hover flex flex-col items-center">
 <h3 className="text-slate-300 font-semibold text-sm mb-4">Eco Score</h3>

 <div className="relative">
 <svg width="220" height="160" viewBox="0 0 220 160">
 /* Background arc */
 <path
 d={describeArc(startAngle, endAngle)}
 fill="none"
 stroke="#1e293b"
 strokeWidth={strokeWidth}
 strokeLinecap="round"
 />

 /* Colored fill arc */
 {safeScore > 0 && (
 <path
 d={describeArc(startAngle, scoreAngle)}
 fill="none"
 stroke={color}
 strokeWidth={strokeWidth}
 strokeLinecap="round"
 style={{ filter: `drop-shadow(0 0 6px ${color}60)` }}
 />
)}
 </svg>
 </div>
 </div>
);
}
```

```

 /* Tick marks */
 {ticks.map((tick) => {
 const angle = startAngle + (tick / 100) * totalAngle;
 const outer = polarToCartesian(angle);
 const innerR = radius - strokeWidth / 2 - 8;
 const inner = {
 x: cx + innerR * Math.cos((angle * Math.PI) / 180),
 y: cy + innerR * Math.sin((angle * Math.PI) / 180)
 };
 return (
 <g key={tick}>
 <line
 x1={inner.x} y1={inner.y}
 x2={outer.x} y2={outer.y}
 stroke="#475569" strokeWidth="1.5"
 />
 </g>
);
 })}

 /* Needle dot */
 <circle cx={needle.x} cy={needle.y} r="6" fill={color}
 style={{ filter: `drop-shadow(0 0 4px ${color})` }} />

 /* Center score */
 <text x={cx} y={cy - 4} textAnchor="middle" fill="white"
 fontSize="36" fontWeight="bold" fontFamily="sans-serif">
 {safeScore}
 </text>
 <text x={cx} y={cy + 18} textAnchor="middle" fill={color}
 fontSize="12" fontFamily="sans-serif">
 {label}
 </text>

 /* Labels */
 <text x="30" y="148" textAnchor="middle" fill="#64748b" fontSize="10">0</text>
 <text x="190" y="148" textAnchor="middle" fill="#64748b" fontSize="10">100</text>
 </svg>
</div>

<div className="text-center mt-1">
 {emoji}
 <p className="text-slate-400 text-xs mt-1">
 {safeScore >= 80 ? 'Keep up the great work!' : safeScore >= 50 ? 'Good progress, keep
going!' : 'Small changes, big impact!'}
 </p>
</div>

/* Score band indicator */
<div className="flex gap-3 mt-4 text-xs">
 [
 { range: '0?49', label: 'Improve', color: '#ef4444' },
 { range: '50?79', label: 'Moderate', color: '#f59e0b' },
 { range: '80?100', label: 'Excellent', color: '#10b981' }
].map(band => (
 <div key={band.range} className="flex items-center gap-1.5">
 <span className="w-2 h-2 rounded-full inline-block" style={{ background: band.color

 {band.range}
 </div>
))}
</div>
</div>
);
}

```

## 35. frontend/src/components/EmissionChart.jsx

```
import React, { useState } from 'react';
import {
 AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip,
 ResponsiveContainer, Legend
} from 'recharts';
import { getCategoryColor } from '../utils/helpers';

const CustomTooltip = ({ active, payload, label }) => {
 if (!active || !payload?.length) return null;
 return (
 <div className="bg-slate-800 border border-slate-700 rounded-xl p-3 shadow-xl">
 <p className="text-slate-300 text-xs font-medium mb-2">{label}</p>
 {payload.map((entry) => (
 <div key={entry.name} className="flex items-center gap-2 text-xs">

 {entry.name}
 {entry.value} kg
 </div>
))}
 <div className="border-t border-slate-700 mt-2 pt-2 flex items-center gap-2 text-xs">
 Total:

 {payload.reduce((s, p) => s + (p.value || 0), 0).toFixed(2)} kg CO?

 </div>
 </div>
);
};

export default function EmissionChart({ data = [], loading = false }) {
 const [view, setView] = useState('stacked');

 if (loading) {
 return (
 <div className="card-hover h-72 flex items-center justify-center">
 <div className="h-8 w-8 border-2 border-eco-500 border-t-transparent rounded-full anima
te-spin" />
 </div>
);
 }

 const categories = ['transportation', 'electricity', 'food', 'shopping'];
 const hasData = data.some(d => d.total > 0);

 return (
 <div className="card-hover">
 <div className="flex items-center justify-between mb-6">
 <div>
 <h3 className="font-semibold text-white">Emissions Over Time</h3>
 <p className="text-slate-500 text-xs mt-0.5">Daily CO? breakdown (kg)</p>
 </div>
 <div className="flex gap-1 bg-slate-800 p-1 rounded-xl">
 {[
 { value: 'stacked', label: 'Stacked' },
 { value: 'total', label: 'Total' }
].map(opt => (
 <button
 key={opt.value}
 onClick={() => setView(opt.value)}
 className={`px-3 py-1.5 rounded-lg text-xs font-medium transition-all ${
 view === opt.value ? 'bg-eco-600 text-white' : 'text-slate-400 hover:text-slate
-200'
 }`}
 >

```

```

 {opt.label}
 </button>
)))
 </div>
</div>

{!hasData ? (
 <div className="h-56 flex flex-col items-center justify-center text-slate-500 gap-3">
 ?
 <p className="text-sm">No emission data yet. Start logging activities!</p>
 </div>
) : (
 <ResponsiveContainer width="100%" height={220}>
 <AreaChart data={data} margin={{ top: 5, right: 5, left: -20, bottom: 5 }}>
 <defs>
 {categories.map(cat => (
 <linearGradient key={cat} id={`grad-${cat}`} x1="0" y1="0" x2="0" y2="1">
 <stop offset="5%" stopColor={getCategoryColor(cat)} stopOpacity={0.3} />
 <stop offset="95%" stopColor={getCategoryColor(cat)} stopOpacity={0.02} />
 </linearGradient>
))}
 <linearGradient id="grad-total" x1="0" y1="0" x2="0" y2="1">
 <stop offset="5%" stopColor="#10b981" stopOpacity={0.3} />
 <stop offset="95%" stopColor="#10b981" stopOpacity={0.02} />
 </linearGradient>
 </defs>
 <CartesianGrid strokeDasharray="3 3" stroke="#1e293b" />
 <XAxis dataKey="label" tick={{ fill: '#64748b', fontSize: 11 }} axisLine={false} tickLine={false} />
 <YAxis tick={{ fill: '#64748b', fontSize: 11 }} axisLine={false} tickLine={false} />
 <Tooltip content={<CustomTooltip />} />
 <Legend
 wrapperStyle={{ padding: '12px' }}
 formatter={(value) => {value}
 />

 {view === 'total' ? (
 <Area type="monotone" dataKey="total" stroke="#10b981" strokeWidth={2}
 fill="url(#grad-total)" dot={{ fill: '#10b981', r: 3 }} activeDot={{ r: 5 }}
 />
) : (
 categories.map(cat => (
 <Area key={cat} type="monotone" dataKey={cat} name={cat}
 stroke={getCategoryColor(cat)} strokeWidth={1.5}
 fill={`url(#grad-${cat})`} stackId="1" />
))
)}
 </AreaChart>
 </ResponsiveContainer>
</div>
);
}

```

## 36. frontend/src/components/ForecastPanel.jsx

```
import React, { useEffect, useState } from 'react';
import { AreaChart, Area, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';
import api from '../services/api';

const CONFIDENCE_COLOR = { high: 'text-eco-400', medium: 'text-amber-400', low: 'text-slate-500' };
const TREND_ICON = { increasing: '?', decreasing: '?', stable: '??' };

function GoalCard({ goal }) {
 return (
 <div className="flex items-center gap-3 p-3 bg-slate-800/50 rounded-xl">
 {goal.icon}
 <div className="flex-1 min-w-0">
 <p className="text-slate-200 text-sm font-medium">{goal.title}</p>
 <p className="text-slate-500 text-xs">Target: {goal.target} {goal.unit}</p>
 </div>
 {goal.saving > 0 && (

 -{goal.saving} kg CO?

)}
 </div>
);
}

const CustomTooltip = ({ active, payload, label }) => {
 if (!active || !payload?.length) return null;
 return (
 <div className="bg-slate-800 border border-slate-700 rounded-xl px-3 py-2 text-xs">
 <p className="text-slate-400 mb-1">{label}</p>
 <p className="text-eco-400 font-semibold">{payload[0].value} kg CO?</p>
 </div>
);
};

export default function ForecastPanel({ userId }) {
 const [data, setData] = useState(null);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 if (!userId) return;
 api.get(`/ai/forecast/${userId}`)
 .then(res => setData(res.data))
 .catch(() => {})
 .finally(() => setLoading(false));
 }, [userId]);

 if (loading) {
 return (
 <div className="card-hover">
 <div className="flex items-center gap-2 mb-4">
 ?
 <h3 className="font-semibold text-white">AI Emission Forecast</h3>
 AI Feature
 </div>
 <div className="h-32 flex items-center justify-center text-slate-500 text-sm">Analysing your data...</div>
 </div>
);
 }

 if (!data?.forecast?.length) return null;

 const chartData = data.forecast.map(f => ({
```

```

 date: new Date(f.date).toLocaleDateString('en-US', { weekday: 'short', month: 'short', day:
'numeric' })),
 predicted: f.predicted,
 confidence: f.confidence,
 trend: f.trend,
 }));

const firstDay = data.forecast[0];

return (
 <div className="card-hover">
 <div className="flex items-center gap-2 mb-1">
 ?
 <h3 className="font-semibold text-white">AI Emission Forecast</h3>
 <span className="ml-auto text-xs text-indigo-400 bg-indigo-950/50 px-2 py-0.5 rounded-f
ull">AI Feature
 </div>
 <div className="flex items-center gap-3 mb-4">
 {firstDay?.trend ? TREND_ICON[firstDay.trend] : '??'}
 <p className="text-slate-400 text-xs">
 Trend: {firstDay?.trend || 'stable'}
 {firstDay?.confidence}
 {firstDay?.confidence || 'low'}
 </p>
 </div>

 <ResponsiveContainer width="100%" height={140}>
 <AreaChart data={chartData}>
 <defs>
 <linearGradient id="forecastGrad" x1="0" y1="0" x2="0" y2="1">
 <stop offset="5%" stopColor="#6366f1" stopOpacity={0.3} />
 <stop offset="95%" stopColor="#6366f1" stopOpacity={0} />
 </linearGradient>
 </defs>
 <XAxis dataKey="date" tick={{ fill: '#64748b', fontSize: 10 }} axisLine={false} tickLine={false} />
 <YAxis tick={{ fill: '#64748b', fontSize: 10 }} axisLine={false} tickLine={false} width={30} />
 <Tooltip content={<CustomTooltip />} />
 <Area type="monotone" dataKey="predicted" stroke="#6366f1" fill="url(#forecastGrad)" strokeWidth={2} dot={{ fill: '#6366f1', r: 3 }} />
 </AreaChart>
 </ResponsiveContainer>

 {data.goals?.length > 0 && (
 <div className="mt-4 border-t border-slate-700/50 pt-4">
 <p className="text-slate-400 text-xs font-medium mb-3">AI-Generated Weekly Goals</p>
 <div className="space-y-2">
 {data.goals.map(g => <GoalCard key={g.id} goal={g} />)}
 </div>
 </div>
)}
 </div>
);
}

```

### 37. frontend/src/components/ImpactContext.jsx

```
import React from 'react';

const DAILY_BUDGET_KG = 7.5; // world avg per person per day
const PARIS_TARGET_KG = 2.5; // Paris Agreement per-person per-day target

const equivalents = (kg) => [
 {
 icon: '?',
 label: 'Trees needed (1 day)',
 value: (kg / 0.058).toFixed(1),
 unit: 'trees',
 description: 'A tree absorbs ~21 kg CO2/year (~0.058 kg/day)',
 },
 {
 icon: '?',
 label: 'Phone charges',
 value: Math.round(kg / 0.008),
 unit: 'charges',
 description: 'Charging a smartphone uses ~0.008 kg CO2',
 },
 {
 icon: '?',
 label: 'LED TV hours',
 value: Math.round(kg / 0.1),
 unit: 'hours',
 description: 'Watching a 32" LED TV emits ~0.1 kg CO2/hour',
 },
 {
 icon: '?',
 label: 'Hot showers',
 value: (kg / 0.5).toFixed(1),
 unit: 'showers',
 description: 'A 10-min hot shower emits ~0.5 kg CO2',
 },
];

function BudgetBar({ kg }) {
 const worldPct = Math.min((kg / DAILY_BUDGET_KG) * 100, 100);
 const parisPct = Math.min((kg / PARIS_TARGET_KG) * 100, 100);
 const worldColor = worldPct < 40 ? 'bg-eco-500' : worldPct < 75 ? 'bg-amber-500' : 'bg-red-500';
 const parisColor = parisPct < 40 ? 'bg-eco-500' : parisPct < 75 ? 'bg-amber-500' : 'bg-red-500';

 return (
 <div className="space-y-2.5">
 <div>
 <div className="flex justify-between text-xs text-slate-400 mb-1">
 World avg daily budget ({DAILY_BUDGET_KG} kg)
 = 100 ? 'text-red-400 font-medium' : 'text-slate-300'}>
 {worldPct.toFixed(0)}%

 </div>
 <div className="h-2 bg-slate-700 rounded-full overflow-hidden">
 <div className={h-full rounded-full transition-all duration-500 ${worldColor}}
 style={{ width: `${worldPct}%` }} />
 </div>
 </div>
 <div>
 <div className="flex justify-between text-xs text-slate-400 mb-1">
 Paris Agreement target ({PARIS_TARGET_KG} kg)
 = 100 ? 'text-red-400 font-medium' : 'text-slate-300'}>
 {parisPct.toFixed(0)}%

 </div>
 </div>
 </div>
);
}
```

```

 </div>
 <div className="h-2 bg-slate-700 rounded-full overflow-hidden">
 <div className={`h-full rounded-full transition-all duration-500 ${parisColor}`}
 style={{ width: `${parisPct}%` }} />
 </div>
 </div>
</div>
);
}

export default function ImpactContext({ emissionKg }) {
 if (!emissionKg || emissionKg <= 0) return null;

 const eq = equivalents(emissionKg);
 const impactLevel = emissionKg < 1 ? 'low' : emissionKg < 5 ? 'medium' : 'high';
 const impactConfig = {
 low: { color: 'border-eco-700/40 bg-eco-950/30', title: 'text-eco-300', badge: 'bg-eco-900/50 text-eco-400', label: 'Low Impact' },
 medium: { color: 'border-amber-700/40 bg-amber-950/20', title: 'text-amber-300', badge: 'bg-amber-900/50 text-amber-400', label: 'Moderate Impact' },
 high: { color: 'border-red-700/40 bg-red-950/20', title: 'text-red-300', badge: 'bg-red-900/50 text-red-400', label: 'High Impact' },
 }[impactLevel];

 return (
 <div className={`mt-3 rounded-xl border p-4 ${impactConfig.color}`}>
 <div className="flex items-center gap-2 mb-3">
 ?
 <h4 className={`text-sm font-semibold ${impactConfig.title}`}>AI Impact Analysis</h4>

 {impactConfig.label}

 </div>

 <div className="grid grid-cols-2 gap-2 mb-4">
 {eq.map(item => (
 <div key={item.label} className="bg-slate-800/50 rounded-lg p-2.5 text-center" title={item.description}>
 <div className="text-xl mb-1">{item.icon}</div>
 <div className="text-white font-bold text-sm">{item.value} {item.unit}</div>
 <div className="text-slate-500 text-xs leading-tight mt-0.5">{item.label}</div>
 </div>
))}
 </div>

 <div className="border-t border-slate-700/50 pt-3">
 <p className="text-xs text-slate-400 mb-2 font-medium">% of your daily CO? budget used by this activity:</p>
 <BudgetBar kg={emissionKg} />
 </div>
 </div>
);
}

```

### 38. frontend/src/components/LoadingSpinner.jsx

---

```
import React from 'react';

export default function LoadingSpinner({ fullscreen = false, size = 'md', text = '' }) {
 const sizes = { sm: 'h-5 w-5', md: 'h-8 w-8', lg: 'h-12 w-12' };

 const spinner = (
 <div className="flex flex-col items-center gap-3">
 <div className={` ${sizes[size]} border-3 border-slate-700 border-t-eco-500 rounded-full animate-spin`}
 style={{ borderWidth: '3px' }} />
 {text && <p className="text-slate-400 text-sm">{text}</p>}
 </div>
);

 if (fullscreen) {
 return (
 <div className="fixed inset-0 bg-slate-950 flex items-center justify-center z-50">
 <div className="flex flex-col items-center gap-4">
 ?
 <div className="h-10 w-10 border-4 border-slate-700 border-t-eco-500 rounded-full animate-spin" />
 <p className="text-slate-400">Loading EcoBeacon...</p>
 </div>
 </div>
);
 }

 return spinner;
}
```

### 39. frontend/src/components/NLPActivityParser.jsx

```
import React, { useState } from 'react';

const NLP_RULES = [
 // Transportation
 { pattern: /\b(drove?|driving|car|ride)\b.*?(\d+\.\d*)\s*(km|kilometer|mile|mi)/i, type: 'transportation', category: 'car', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|kilometer|mile|mi).*\b(drove?|driving|car)\b/i, type: 'transportation', category: 'car', valueIdx: 1 },
 { pattern: /\b(flew?|flight|airplane|plane|air)\b.*?(\d+\.\d*)\s*(km|kilometer|mile|mi)/i, type: 'transportation', category: 'flight', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|mi).*\b(flew?|flight|plane)\b/i, type: 'transportation', category: 'flight', valueIdx: 1 },
 { pattern: /\b(cycled?|cycling|bike|bicycle|biked)\b.*?(\d+\.\d*)\s*(km|kilometer|mile|mi)/i, type: 'transportation', category: 'bike', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|mi).*\b(cycled?|bike|bicycle)\b/i, type: 'transportation', category: 'bike', valueIdx: 1 },
 { pattern: /\b(walked?|walking|on foot|strolled?)\b.*?(\d+\.\d*)\s*(km|kilometer|mile|mi)/i, type: 'transportation', category: 'walking', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|mi).*\b(walked?|walking)\b/i, type: 'transportation', category: 'walking', valueIdx: 1 },
 { pattern: /\b(bus|commute by bus)\b.*?(\d+\.\d*)\s*(km|mi)/i, type: 'transportation', category: 'bus', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|mi).*\b(bus)\b/i, type: 'transportation', category: 'bus', valueIdx: 1 },
 { pattern: /\b(train|rail|subway|metro)\b.*?(\d+\.\d*)\s*(km|mi)/i, type: 'transportation', category: 'train', valueIdx: 2 },
 { pattern: /(\d+\.\d*)\s*(km|mi).*\b(train|rail|metro)\b/i, type: 'transportation', category: 'train', valueIdx: 1 },
 { pattern: /\b(motorcycle|motorbike|moped|scooter)\b.*?(\d+\.\d*)\s*(km|mi)/i, type: 'transportation', category: 'motorcycle', valueIdx: 2 },
 { pattern: /\b(electric car|ev|tesla|e-car)\b.*?(\d+\.\d*)\s*(km|mi)/i, type: 'transportation', category: 'electric_car', valueIdx: 2 },

 // Electricity
 { pattern: /(\d+\.\d*)\s*(kwh|kw-h|kilowatt|units?)\b/i, type: 'electricity', category: 'standard', valueIdx: 1 },
 { pattern: /\b(solar|solar panel)\b.*?(\d+\.\d*)\s*(kwh|kw)/i, type: 'electricity', category: 'solar', valueIdx: 2 },
 { pattern: /\b(wind|wind energy)\b.*?(\d+\.\d*)\s*(kwh|kw)/i, type: 'electricity', category: 'wind', valueIdx: 2 },
 { pattern: /\b(used?|consumed?|electricity)\b.*?(\d+\.\d*)\s*(kwh|kw|units?)/i, type: 'electricity', category: 'standard', valueIdx: 2 },

 // Food
 { pattern: /\b(ate?|eat|had|eaten)\b.*?\b(beef|steak|burger)\b/i, type: 'food', category: 'beef', valueIdx: null, defaultValue: 1 },
 { pattern: /(\d+)\s*(beef|steak)\s*(meal|portion)?/i, type: 'food', category: 'beef', valueIdx: 1 },
 { pattern: /\b(ate?|eat|had)\b.*?\b(meat|chicken|pork|lamb)\b/i, type: 'food', category: 'meat', valueIdx: null, defaultValue: 1 },
 { pattern: /\b(fast food|mcdonald|kfc|burger king|pizza|takeaway)\b/i, type: 'food', category: 'fastfood', valueIdx: null, defaultValue: 1 },
 { pattern: /\b(ate?|eat|had|eaten)\b.*?\b(veg|vegetarian|salad|veggies)\b/i, type: 'food', category: 'veg', valueIdx: null, defaultValue: 1 },
 { pattern: /\b(vegan|plant.based)\b/i, type: 'food', category: 'vegan', valueIdx: null, defaultValue: 1 },
 { pattern: /\b(dairy|milk|cheese|yogurt|butter)\b/i, type: 'food', category: 'dairy', valueIdx: null, defaultValue: 1 },
 { pattern: /\b(ate?|eat|had)\b.*?\b(fish|seafood|salmon|tuna)\b/i, type: 'food', category: 'fish', valueIdx: null, defaultValue: 1 },

 // Shopping
 { pattern: /\b(bought?|purchased?|ordered?)\b.*?(\d+)\s*(item|product|thing)/i, type: 'shopping', category: 'normal', valueIdx: 2 },

```

```

 { pattern: /\b(secondhand|second.hand|thrift)\b.*?(\d+)?/i, type: 'shopping', category: 'secondhand', valueIdx: 2, defaultValue: 1 },
 { pattern: /\b(eco.friendly|sustainable|green product)\b.*?(\d+)?/i, type: 'shopping', category: 'eco-friendly', valueIdx: 2, defaultValue: 1 },
 { pattern: /\b(electronics|gadget|appliance|device)\b.*?(\d+)?/i, type: 'shopping', category: 'high-impact', valueIdx: 2, defaultValue: 1 },
];

function parseActivity(text) {
 for (const rule of NLP_RULES) {
 const match = text.trim().match(rule.pattern);
 if (match) {
 let value = null;
 if (rule.valueIdx !== null && match[rule.valueIdx]) {
 value = parseFloat(match[rule.valueIdx]);
 } else if (rule.defaultValue) {
 value = rule.defaultValue;
 }
 if (value !== null && value > 0) {
 return { type: rule.type, category: rule.category, value };
 }
 }
 }
 return null;
}

const CATEGORY_LABELS = {
 car: 'Car', motorcycle: 'Motorcycle', bus: 'Bus', train: 'Train',
 flight: 'Flight', bike: 'Bicycle', walking: 'Walking', electric_car: 'Electric Car',
 standard: 'Standard Grid', solar: 'Solar', wind: 'Wind Energy',
 beef: 'Beef Meal', meat: 'Meat Meal', fish: 'Fish Meal', fastfood: 'Fast Food',
 dairy: 'Dairy-based', veg: 'Vegetarian', vegan: 'Vegan',
 'high-impact': 'High Impact', normal: 'Normal', 'eco-friendly': 'Eco-Friendly', secondhand: 'Secondhand'
};

const TYPE_ICONS = { transportation: '?', electricity: '?', food: '??', shopping: '??' };

const EXAMPLES = [
 'drove 25km to work',
 'ate a beef burger for lunch',
 'used 8 kWh today',
 'cycled 10km to school',
 'flew 500km for business',
 'bought 2 items online',
 'had a vegan dinner',
 'took the train 30km',
];

export default function NLPActivityParser({ onParsed }) {
 const [input, setInput] = useState('');
 const [result, setResult] = useState(null);
 const [error, setError] = useState('');
 const [showExamples, setShowExamples] = useState(false);

 const handleParse = () => {
 if (!input.trim()) return;
 const parsed = parseActivity(input);
 if (parsed) {
 setResult(parsed);
 setError('');
 } else {
 setResult(null);
 setError('Could not understand that. Try: "drove 15km" or "ate beef burger"');
 }
 };

 const handleApply = () => {

```

```

 if (result) {
 onParsed(result);
 setInput('');
 setResult(null);
 setError('');
 }
 };

 const handleExample = (ex) => {
 setInput(ex);
 setShowExamples(false);
 const parsed = parseActivity(ex);
 if (parsed) { setResult(parsed); setError(''); }
 };

 return (
 <div className="card mb-4 border border-indigo-700/40 bg-indigo-950/30">
 <div className="flex items-center gap-2 mb-3">
 ?
 <h3 className="text-indigo-300 font-semibold text-sm">AI Natural Language Logger</h3>
 <span className="ml-auto text-xs text-indigo-500 bg-indigo-950/50 px-2 py-0.5 rounded-f
ull">AI Feature
 </div>
 <p className="text-slate-400 text-xs mb-3">Describe your activity in plain English ? AI w
ill fill the form for you.</p>

 <div className="flex gap-2">
 <input
 type="text"
 value={input}
 onChange={e => { setInput(e.target.value); setResult(null); setError(''); }}
 onKeyDown={e => e.key === 'Enter' && handleParse()}
 className="flex-1 bg-slate-800/60 border border-slate-700 rounded-xl px-4 py-2.5 text
-white text-sm placeholder-slate-500 focus:outline-none focus:border-indigo-500 transition-colo
rs"
 placeholder='"drove 15km to office" or "ate beef for lunch"'
 />
 <button
 type="button"
 onClick={handleParse}
 className="px-4 py-2.5 rounded-xl bg-indigo-600 hover:bg-indigo-500 text-white text-s
m font-medium transition-colors"
 >
 Parse
 </button>
 </div>

 <button
 type="button"
 onClick={() => setShowExamples(s => !s)}
 className="text-indigo-500 text-xs mt-2 hover:text-indigo-400 transition-colors"
 >
 {showExamples ? 'Hide examples ?' : 'Show examples ?'}
 </button>

 {showExamples && (
 <div className="mt-2 flex flex-wrap gap-2">
 {EXAMPLES.map(ex => (
 <button key={ex} type="button" onClick={() => handleExample(ex)}
 className="text-xs px-3 py-1.5 bg-slate-800 hover:bg-slate-700 text-slate-300 rou
nded-full border border-slate-700 transition-colors">
 {ex}
 </button>
)}}
 </div>
)}
 {error && <p className="mt-2 text-xs text-amber-400">?? {error}</p>}
 </div>
);

```

```

 {result && (
 <div className="mt-3 p-3 bg-indigo-900/30 rounded-xl border border-indigo-700/50 flex i
tems-center justify-between">
 <div className="flex items-center gap-3 text-sm">
 {TYPE_ICONS[result.type]}
 <div>
 {CATEGORY_LABELS[result.category]}
 ({result.type})
 <p className="text-indigo-300 text-xs">Value: {result.value}</p>
 </div>
 <button type="button" onClick={handleApply}
 className="px-4 py-2 bg-indigo-600 hover:bg-indigo-500 text-white text-sm font-medi
um rounded-xl transition-colors">
 Apply ?
 </button>
 </div>
 </div>
)}
 </div>
);
}

```

## 40. frontend/src/components/Navbar.jsx

```
import React, { useState } from 'react';
import { Link, useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { getScoreColor, getScoreLabel } from '../utils/helpers';

export default function Navbar() {
 const { user, logout } = useAuth();
 const location = useLocation();
 const navigate = useNavigate();
 const [menuOpen, setMenuOpen] = useState(false);

 const handleLogout = () => {
 logout();
 navigate('/login');
 };

 const navLinks = [
 { path: '/dashboard', label: 'Dashboard', icon: '?' },
 { path: '/log', label: 'Log Activity', icon: '?' },
 { path: '/profile', label: 'Profile', icon: '?' }
];

 return (
 <nav className="fixed top-0 left-0 right-0 z-40 bg-slate-900/95 backdrop-blur-sm border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center justify-between h-16">

 {/* Logo */}
 <Link to="/dashboard" className="flex items-center gap-2.5 group">
 ?

 EcoBeacon AI

 </Link>

 {/* Desktop Nav */}
 <div className="hidden md:flex items-center gap-1">
 {navLinks.map(({ path, label, icon }) => (
 <Link
 key={path}
 to={path}
 className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-medium transition-all duration-200 ${
 location.pathname === path
 ? 'bg-eco-600/20 text-eco-400 border border-eco-700/50'
 : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800'
 }`}
 >
 {icon}
 {label}
 </Link>
))}
 </div>

 {/* User Info & Eco Score */}
 <div className="hidden md:flex items-center gap-4">
 {user && (
 <div className="flex items-center gap-3">
 <div className="text-right">
 <p className="text-sm font-medium text-slate-200">{user.name}</p>
 <p className="text-xs" style={{ color: getScoreColor(user.ecoScore) }}>
 {getScoreLabel(user.ecoScore)} ? Score: {user.ecoScore}
 </p>
 </div>

```

```

 </div>
 <div
 className="w-10 h-10 rounded-full flex items-center justify-center text-sm font-
nt-bold border-2"
 style={{ borderColor: getScoreColor(user.ecoScore), color: getScoreColor(user
.ecoScore), background: `${getScoreColor(user.ecoScore)}20` }}
 >
 {user.name.charAt(0).toUpperCase()}
 </div>
 <button onClick={handleLogout} className="text-slate-400 hover:text-red-400 tra
nsition-colors text-sm px-3 py-2 rounded-lg hover:bg-red-500/10">
 Logout
 </button>
 </div>
)}
</div>

{/* Mobile menu button */}
<button
 onClick={() => setMenuOpen(!menuOpen)}
 className="md:hidden p-2 rounded-lg text-slate-400 hover:text-white hover:bg-slate-
800"
 >
 {menuOpen ? '?' : '?'}
 </button>
</div>
</div>

{/* Mobile Menu */}
{menuOpen && (
 <div className="md:hidden bg-slate-900 border-t border-slate-800 px-4 py-4 space-y-2">
 {navLinks.map(({ path, label, icon }) => (
 <Link
 key={path}
 to={path}
 onClick={() => setMenuOpen(false)}
 className={`flex items-center gap-3 px-4 py-3 rounded-xl text-sm font-medium tran
sition-all ${
 location.pathname === path
 ? 'bg-eco-600/20 text-eco-400 border border-eco-700/50'
 : 'text-slate-400 hover:bg-slate-800'
 }`}
 >
 {icon}
 {label}
 </Link>
)}}
 <button
 onClick={() => { handleLogout(); setMenuOpen(false); }}
 className="w-full flex items-center gap-3 px-4 py-3 rounded-xl text-sm font-medium
text-red-400 hover:bg-red-500/10"
 >
 ?
 Logout
 </button>
 </div>
)}
</nav>
);
}

```

## 41. frontend/src/components/StatsCard.jsx

```
import React from 'react';

export default function StatsCard({ title, value, unit, icon, color, subtitle, trend }) {
 const colorMap = {
 green: { bg: 'bg-eco-500/10', border: 'border-eco-700/40', text: 'text-eco-400', icon: 'bg-eco-500/20' },
 blue: { bg: 'bg-blue-500/10', border: 'border-blue-700/40', text: 'text-blue-400', icon: 'bg-blue-500/20' },
 yellow: { bg: 'bg-amber-500/10', border: 'border-amber-700/40', text: 'text-amber-400', icon: 'bg-amber-500/20' },
 red: { bg: 'bg-red-500/10', border: 'border-red-700/40', text: 'text-red-400', icon: 'bg-red-500/20' },
 purple: { bg: 'bg-violet-500/10', border: 'border-violet-700/40', text: 'text-violet-400', icon: 'bg-violet-500/20' }
 };

 const c = colorMap[color] || colorMap.green;

 return (
 <div className={`card-hover ${c.bg} border ${c.border} animate-fade-in`} >
 <div className="flex items-start justify-between">
 <div className="flex-1">
 <p className="text-slate-400 text-sm font-medium">{title}</p>
 <div className="flex items-baseline gap-2 mt-2">
 {value}
 {unit && {unit}}
 </div>
 {subtitle && <p className={`text-xs mt-1.5 ${c.text}`}>{subtitle}</p>}
 {trend !== undefined && (
 <div className={`flex items-center gap-1 mt-2 text-xs ${trend <= 0 ? 'text-eco-400' : 'text-red-400'}`} >
 {trend <= 0 ? '?' : '?'}
 {Math.abs(trend).toFixed(1)} kg vs last week
 </div>
)}
 </div>
 <div className={`w-12 h-12 ${c.icon} rounded-2xl flex items-center justify-center text-2xl flex-shrink-0`} >
 {icon}
 </div>
 </div>
 </div>
);
}
```

## 42. frontend/src/components/SuggestionsPanel.jsx

```
import React, { useState } from 'react';
import { getCategoryColor } from '../utils/helpers';

const priorityConfig = {
 high: { label: 'High Impact', bg: 'bg-red-500/10', text: 'text-red-400', border: 'border-red-700/30' },
 medium: { label: 'Medium', bg: 'bg-amber-500/10', text: 'text-amber-400', border: 'border-amber-700/30' },
 low: { label: 'Low', bg: 'bg-slate-500/10', text: 'text-slate-400', border: 'border-slate-700/30' }
};

export default function SuggestionsPanel({ suggestions = [], loading = false, activitiesAnalyzed = 0 }) {
 const [expanded, setExpanded] = useState(null);

 if (loading) {
 return (
 <div className="card-hover">
 <div className="flex items-center gap-3 mb-4">
 ?
 <h3 className="font-semibold text-white">AI Recommendations</h3>
 </div>
 <div className="space-y-3">
 {[1, 2, 3].map(i => (
 <div key={i} className="h-16 bg-slate-800 rounded-xl animate-pulse" />
))}
 </div>
 </div>
);
 }

 return (
 <div className="card-hover">
 <div className="flex items-center justify-between mb-5">
 <div className="flex items-center gap-3">
 <div className="w-9 h-9 bg-eco-500/15 rounded-xl flex items-center justify-center">
 ?
 </div>
 <div>
 <h3 className="font-semibold text-white">AI Recommendations</h3>
 <p className="text-slate-500 text-xs">
 {activitiesAnalyzed > 0 ? `Based on ${activitiesAnalyzed} activities` : 'Personalized suggestions'}
 </p>
 </div>
 </div>

 {suggestions.length} tips

 </div>

 {suggestions.length === 0 ? (
 <div className="flex flex-col items-center justify-center py-8 text-slate-500 gap-3">
 ?
 <p className="text-sm text-center">Log some activities to get personalized AI recommendations!</p>
 </div>
) : (
 <div className="space-y-3">
 {suggestions.map((s, idx) => {
 const pc = priorityConfig[s.priority] || priorityConfig.low;
 const isOpen = expanded === s.id;

```



## 43. frontend/src/context/AuthContext.jsx

---

```
import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
import api from '../services/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
 const [user, setUser] = useState(null);
 const [token, setToken] = useState(() => localStorage.getItem('eco_token'));
 const [loading, setLoading] = useState(true);

 const loadUser = useCallback(async () => {
 if (!token) { setLoading(false); return; }
 try {
 const res = await api.get('/auth/profile');
 setUser(res.data.user);
 } catch {
 localStorage.removeItem('eco_token');
 setToken(null);
 setUser(null);
 } finally {
 setLoading(false);
 }
 }, [token]);

 useEffect(() => { loadUser(); }, [loadUser]);

 const login = async (email, password) => {
 const res = await api.post('/auth/login', { email, password });
 const { user: userData, token: newToken } = res.data;
 localStorage.setItem('eco_token', newToken);
 setToken(newToken);
 setUser(userData);
 return userData;
 };

 const register = async (name, email, password) => {
 const res = await api.post('/auth/register', { name, email, password });
 const { user: userData, token: newToken } = res.data;
 localStorage.setItem('eco_token', newToken);
 setToken(newToken);
 setUser(userData);
 return userData;
 };

 const logout = () => {
 localStorage.removeItem('eco_token');
 setToken(null);
 setUser(null);
 };

 const refreshUser = async () => {
 try {
 const res = await api.get('/auth/profile');
 setUser(res.data.user);
 } catch (err) {
 console.error('Failed to refresh user', err);
 }
 };

 return (
 <AuthContext.Provider value={{ user, token, loading, login, register, logout, refreshUser }}>
 {children}
 </AuthContext.Provider>
);
};
```

```
};

export const useAuth = () => {
 const ctx = useContext(AuthContext);
 if (!ctx) throw new Error('useAuth must be used within AuthProvider');
 return ctx;
};
```

## 44. frontend/src/hooks/useGeoTracker.js

---

```
import { useState, useRef, useCallback } from 'react';
import { haversineDistance, classifyTransportMode } from '../utils/geoTracking';

const SPEED_HISTORY_SIZE = 8; // rolling window for AI classifier
const MIN_MOVE_METERS = 5; // ignore GPS jitter below this threshold

export default function useGeoTracker() {
 const [state, setState] = useState({
 status: 'idle', // idle | requesting | tracking | stopped | error
 distanceKm: 0,
 currentSpeedKmh: 0,
 detectedMode: null,
 positions: [],
 errorMsg: '',
 duration: 0
 });

 const watchIdRef = useRef(null);
 const lastPosRef = useRef(null);
 const speedHistRef = useRef([]);
 const startTimeRef = useRef(null);
 const timerRef = useRef(null);

 const start = useCallback(() => {
 if (!navigator.geolocation) {
 setState(s => ({ ...s, status: 'error', errorMsg: 'Geolocation is not supported by your browser.' }));
 return;
 }

 setState(s => ({ ...s, status: 'requesting', errorMsg: '' }));

 watchIdRef.current = navigator.geolocation.watchPosition(
 (pos) => {
 const { latitude: lat, longitude: lon, speed } = pos.coords;
 const now = Date.now();

 if (!startTimeRef.current) {
 startTimeRef.current = now;
 // Start duration timer
 timerRef.current = setInterval(() => {
 setState(s => ({ ...s, duration: Math.floor((Date.now() - startTimeRef.current) / 1000) }));
 }, 1000);
 }

 setState(prev => {
 let newDistanceKm = prev.distanceKm;

 if (lastPosRef.current) {
 const d = haversineDistance(
 lastPosRef.current.lat, lastPosRef.current.lon, lat, lon
);
 // Only count if moved more than MIN_MOVE_METERS (filters GPS noise)
 if (d * 1000 >= MIN_MOVE_METERS) {
 newDistanceKm += d;
 lastPosRef.current = { lat, lon };
 }
 } else {
 lastPosRef.current = { lat, lon };
 }

 // Convert m/s ? km/h; fallback to 0 if GPS doesn't provide speed
 const speedKmh = speed != null ? speed * 3.6 : 0;

```

```

 // Maintain rolling speed history for AI classifier
 speedHistRef.current = [
 ...speedHistRef.current.slice(-(SPEED_HISTORY_SIZE - 1)),
 speedKmh
];

 const detectedMode = classifyTransportMode(speedHistRef.current);

 return {
 ...prev,
 status: 'tracking',
 distanceKm: newDistanceKm,
 currentSpeedKmh: speedKmh,
 detectedMode,
 positions: [...prev.positions, { lat, lon, t: now }]
 };
 });
},
(err) => {
 const messages = {
 1: 'Location permission denied. Please allow location access.',
 2: 'Location unavailable. Check GPS signal.',
 3: 'Location request timed out.'
 };
 setState(s => ({
 ...s,
 status: 'error',
 errorMsg: messages[err.code] || 'Failed to get location.'
 }));
},
{ enableHighAccuracy: true, timeout: 15000, maximumAge: 0 }
);
}, []);

const stop = useCallback(() => {
 if (watchIdRef.current !== null) {
 navigator.geolocation.clearWatch(watchIdRef.current);
 watchIdRef.current = null;
 }
 if (timerRef.current) {
 clearInterval(timerRef.current);
 timerRef.current = null;
 }
 lastPosRef.current = null;
 speedHistRef.current = [];
 startTimeRef.current = null;
 setState(s => ({ ...s, status: 'stopped' }));
}, []);

const reset = useCallback(() => {
 stop();
 setState({
 status: 'idle',
 distanceKm: 0,
 currentSpeedKmh: 0,
 detectedMode: null,
 positions: [],
 errorMsg: '',
 duration: 0
 });
}, [stop]);

return { ...state, start, stop, reset };
}

```

## 45. frontend/src/index.css

---

```

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
 * {
 box-sizing: border-box;
 }

 body {
 @apply bg-slate-950 text-slate-100 font-sans antialiased;
 min-height: 100vh;
 }

 ::-webkit-scrollbar {
 width: 6px;
 height: 6px;
 }
 ::-webkit-scrollbar-track {
 @apply bg-slate-900;
 }
 ::-webkit-scrollbar-thumb {
 @apply bg-slate-700 rounded-full;
 }
 ::-webkit-scrollbar-thumb:hover {
 @apply bg-slate-600;
 }
}

@layer components {
 .card {
 @apply bg-slate-900 border border-slate-800 rounded-2xl p-6;
 }

 .card-hover {
 @apply card transition-all duration-200 hover:border-eco-700 hover:shadow-lg hover:shadow-eco-900/20;
 }

 .btn-primary {
 @apply bg-eco-600 hover:bg-eco-500 text-white font-semibold py-2.5 px-5 rounded-xl
 transition-all duration-200 focus:outline-none focus:ring-2 focus:ring-eco-500 focus
:ring-offset-2
 focus:ring-offset-slate-950 disabled:opacity-50 disabled:cursor-not-allowed;
 }

 .btn-secondary {
 @apply bg-slate-800 hover:bg-slate-700 text-slate-200 font-semibold py-2.5 px-5 rounded-xl
 border border-slate-700 transition-all duration-200 focus:outline-none focus:ring-2
 focus:ring-slate-500 focus:ring-offset-2 focus:ring-offset-slate-950;
 }

 .input-field {
 @apply w-full bg-slate-800 border border-slate-700 text-slate-100 placeholder-slate-500
 rounded-xl px-4 py-3 focus:outline-none focus:ring-2 focus:ring-eco-500
 focus:border-eco-500 transition-colors duration-200;
 }

 .label {
 @apply block text-sm font-medium text-slate-300 mb-1.5;
 }

 .badge {
 @apply inline-flex items-center gap-1.5 px-3 py-1 rounded-full text-xs font-semibold;
 }
}

```

```

.stat-value {
 @apply text-3xl font-bold text-white;
}

.stat-label {
 @apply text-sm text-slate-400 mt-1;
}
}

@layer utilities {
 .text-gradient {
 @apply bg-gradient-to-r from-eco-400 to-eco-600 bg-clip-text text-transparent;
 }

 .glow-eco {
 box-shadow: 0 0 20px rgba(34, 197, 94, 0.15);
 }
}

/* Chart tooltip styling */
.recharts-tooltip-wrapper .recharts-default-tooltip {
 background-color: #1e293b !important;
 border: 1px solid #334155 !important;
 border-radius: 12px !important;
 color: #f1f5f9 !important;
}

```

## 46. frontend/src/main.jsx

---

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
 <React.StrictMode>
 <App />
 </React.StrictMode>
);
```

## 47. frontend/src/pages/Dashboard.jsx

---

```
import React, { useState, useEffect, useCallback } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../services/api';
import { formatEmission } from '../utils/helpers';
import Navbar from '../components/Navbar';
import StatsCard from '../components/StatsCard';
import EcoScoreGauge from '../components/EcoScoreGauge';
import EmissionChart from '../components/EmissionChart';
import ActivityBreakdown from '../components/ActivityBreakdown';
import SuggestionsPanel from '../components/SuggestionsPanel';
import BadgesPanel from '../components/BadgesPanel';
import ForecastPanel from '../components/ForecastPanel';

export default function Dashboard() {
 const { user, refreshUser } = useAuth();
 const [summary, setSummary] = useState(null);
 const [suggestions, setSuggestions] = useState([]);
 const [activities, setActivities] = useState([]);
 const [loading, setLoading] = useState(true);
 const [aiLoading, setAiLoading] = useState(true);
 const [activitiesAnalyzed, setActivitiesAnalyzed] = useState(0);

 const fetchDashboardData = useCallback(async () => {
 if (!user) return;
 try {
 const [summaryRes, activityRes] = await Promise.all([
 api.get(`/emissions/summary/${user._id}`),
 api.get(`/activity/user/${user._id}?limit=10`)
]);
 setSummary(summaryRes.data);
 setActivities(activityRes.data.activities || []);
 } catch (err) {
 console.error('Dashboard data error:', err);
 } finally {
 setLoading(false);
 }
 }, [user]);

 const fetchSuggestions = useCallback(async () => {
 if (!user) return;
 setAiLoading(true);
 try {
 const res = await api.get(`/ai/suggestions/${user._id}`);
 setSuggestions(res.data.suggestions || []);
 setActivitiesAnalyzed(res.data.activitiesAnalyzed || 0);
 } catch (err) {
 console.error('AI suggestions error:', err);
 } finally {
 setAiLoading(false);
 }
 }, [user]);

 useEffect(() => {
 refreshUser();
 fetchDashboardData();
 fetchSuggestions();
 }, []);

 const recentActivity = activities[0];

 return (
 <div className="min-h-screen bg-slate-950">
 <Navbar />
```

```

<main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 pt-24 pb-12">

 {/ * Header */}
 <div className="flex items-center justify-between mb-8">
 <div>
 <h1 className="text-2xl font-bold text-white">
 Welcome back, {user?.name?.split(' ')[0]}

 </h1>
 <p className="text-slate-400 text-sm mt-1">
 {recentActivity
 ? `Last activity: ${new Date(recentActivity.date).toLocaleDateString('en-US', {
 weekday: 'short', month: 'short', day: 'numeric' })}`
 : 'Start tracking your carbon footprint today'}
 </p>
 </div>
 <Link to="/log" className="btn-primary flex items-center gap-2">
 ?
 Log Activity
 </Link>
 </div>

 {/ * Stats Row */}
 <div className="grid grid-cols-2 lg:grid-cols-4 gap-4 mb-6">
 <StatsCard
 title="Today's Emissions"
 value={loading ? '?' : formatEmission(summary?.summary?.daily)}
 unit="kg CO2"
 icon="?"
 color="blue"
 subtitle="Current day"
 />
 <StatsCard
 title="This Week"
 value={loading ? '?' : formatEmission(summary?.summary?.weekly)}
 unit="kg CO2"
 icon="?"
 color="yellow"
 subtitle="Last 7 days"
 />
 <StatsCard
 title="This Month"
 value={loading ? '?' : formatEmission(summary?.summary?.monthly)}
 unit="kg CO2"
 icon="?"
 color="purple"
 subtitle="Current month"
 />
 <StatsCard
 title="Eco Score"
 value={user?.ecoScore ?? 50}
 unit="/ 100"
 icon="?"
 color="green"
 subtitle={user?.ecoScore >= 80 ? 'Excellent!' : user?.ecoScore >= 50 ? 'Good progre
ss' : 'Keep improving'}
 />
 </div>

 {/ * Middle Row: Chart + Gauge */}
 <div className="grid grid-cols-1 lg:grid-cols-3 gap-4 mb-6">
 <div className="lg:col-span-2">
 <EmissionChart
 data={summary?.dailyBreakdown || []}
 loading={loading}
 />
 </div>
 </div>

```

```

<div>
 <EcoScoreGauge score={user?.ecoScore ?? 50} />
</div>
</div>

{/* Bottom Row: Breakdown + Suggestions + Badges */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4 mb-6">
 <ActivityBreakdown
 data={summary?.categoryBreakdown || []}
 loading={loading}
 />
 <div className="lg:col-span-2">
 <SuggestionsPanel
 suggestions={suggestions}
 loading={aiLoading}
 activitiesAnalyzed={activitiesAnalyzed}
 />
 </div>
</div>

{/* AI Forecast + Badges + Recent Activity */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-4 mb-6">
 <ForecastPanel userId={user?._id} />
 <BadgesPanel badges={user?.badges || []} streak={user?.streak || 0} />
</div>

{/* Recent Activity */}
<div className="card-hover">
 <div className="flex items-center justify-between mb-5">
 <h3 className="font-semibold text-white">Recent Activities</h3>
 <Link to="/log" className="text-eco-400 hover:text-eco-300 text-sm">+ Add new</Li
nk>
 </div>
 {activities.length === 0 ? (
 <div className="flex flex-col items-center justify-center py-8 text-slate-500 gap
-3">
 ?
 <p className="text-sm">No activities logged yet</p>
 <Link to="/log" className="btn-primary text-sm py-2">Log your first activity</L
ink>
 </div>
) : (
 <div className="space-y-2">
 {activities.slice(0, 6).map(a => (
 <div key={a._id} className="flex items-center gap-3 p-3 bg-slate-800/50 round
ed-xl hover:bg-slate-800 transition-colors">
 <div className="w-9 h-9 rounded-xl bg-slate-700 flex items-center justify-c
enter text-lg flex-shrink-0">
 {a.type === 'transportation' ? '?' : a.type === 'electricity' ? '?' : a.t
ype === 'food' ? '??' : '??'}
 </div>
 <div className="flex-1 min-w-0">
 <p className="text-slate-200 text-sm font-medium capitalize">
 {a.category.replace('-', ' ')} <span className="text-slate-500 font-nor
mal capitalize">({a.type})
 </p>
 <p className="text-slate-500 text-xs">{new Date(a.date).toLocaleDateStrin
g('en-US', { month: 'short', day: 'numeric', hour: '2-digit', minute: '2-digit' })}</p>
 </div>
 <div className="text-right flex-shrink-0">
 <p className="text-red-400 text-sm font-semibold">{a.emission.toFixed(2)}
</p>
 <p className="text-slate-500 text-xs">kg CO?</p>
 </div>
 </div>
)
 </div>
)}
</div>

```

```
 })
 </div>
</main>
</div>
);
}
```

## 48. frontend/src/pages/LogActivity.jsx

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../services/api';
import { ACTIVITY_CONFIG, EMISSION_FACTORS, estimateEmission } from '../utils/helpers';
import Navbar from '../components/Navbar';
import AutoTracker from '../components/AutoTracker';
import NLPActivityParser from '../components/NLPActivityParser';
import ImpactContext from '../components/ImpactContext';

// ?? Smart pattern tip: compare current input to user's history ??????????????
function useSmartTip(type, category, value, userId) {
 const [tip, setTip] = useState(null);

 useEffect(() => {
 if (!type || !category || !value || !userId) { setTip(null); return; }
 api.get(`/activity/user/${userId}`)
 .then(({ data }) => {
 const acts = (data.activities || []).filter(a => a.type === type && a.category === cate
gory);
 if (acts.length < 2) { setTip(null); return; }
 const avg = acts.reduce((s, a) => s + a.value, 0) / acts.length;
 const diff = parseFloat(value) - avg;
 const pct = Math.round(Math.abs(diff / avg) * 100);
 if (pct < 10) { setTip(null); return; }
 setTip(diff > 0
 ? { type: 'warn', text: `${pct}% more than your usual ${avg.toFixed(1)} for this cate
gory.` }
 : { type: 'good', text: `${pct}% less than your usual ${avg.toFixed(1)} ? great job!` }
);
 })
 .catch(() => setTip(null));
 }, [type, category, value, userId]);

 return tip;
}

// ?? Carbon budget indicator ???
function CarbonBudgetBadge({ emission }) {
 if (!emission) return null;
 const budget = 2.5;
 const pct = Math.round((emission / budget) * 100);
 const color = pct < 30 ? 'text-eco-400' : pct < 70 ? 'text-amber-400' : 'text-red-400';
 return (

 {pct}% of Paris target daily budget

);
}

export default function LogActivity() {
 const { user, refreshUser } = useAuth();
 const navigate = useNavigate();

 const [form, setForm] = useState({
 type: 'transportation',
 category: '',
 value: '',
 notes: '',
 date: new Date().toISOString().split('T')[0]
 });
 const [loading, setLoading] = useState(false);
 const [success, setSuccess] = useState(null);
```

```

const [error, setError] = useState('');
const [showImpact, setShowImpact] = useState(false);

const currentConfig = ACTIVITY_CONFIG[form.type];
const selectedCategory = currentConfig?.categories.find(c => c.value === form.category);
const estimatedEmission = form.value && form.category
 ? estimateEmission(form.type, form.category, parseFloat(form.value))
 : null;

const smartTip = useSmartTip(form.type, form.category, form.value, user?.id);

// Auto-show impact panel when emission is calculated
useEffect(() => {
 if (estimatedEmission !== null && estimatedEmission > 0) setShowImpact(true);
}, [estimatedEmission]);

const handleNLPParsed = ({ type, category, value }) => {
 setForm(f => ({ ...f, type, category, value: String(value) }));
 setError('');
};

const handleAutoTrack = ({ distanceKm, mode }) => {
 setForm(f => ({ ...f, type: 'transportation', category: mode, value: distanceKm.toFixed(2)
}));
 setError('');
};

const handleTypeChange = (type) => {
 setForm(f => ({ ...f, type, category: '' }));
 setError('');
};

const handleChange = (e) => {
 setForm(f => ({ ...f, [e.target.name]: e.target.value }));
 setError('');
};

const handleSubmit = async (e) => {
 e.preventDefault();
 if (!form.category) { setError('Please select a category'); return; }
 if (!form.value || parseFloat(form.value) <= 0) { setError('Please enter a valid value greater than 0'); return; }

 setLoading(true);
 setError('');
 try {
 const res = await api.post('/activity/add', {
 type: form.type,
 category: form.category,
 value: parseFloat(form.value),
 notes: form.notes,
 date: form.date
 });
 await refreshUser();
 setSuccess(res.data);
 setTimeout(() => navigate('/dashboard'), 2500);
 } catch (err) {
 setError(err.response?.data?.message || 'Failed to log activity. Please try again.');
```

```

 } finally {
 setLoading(false);
 }
};

if (success) {
 return (
 <div className="min-h-screen bg-slate-950">
 <Navbar />
 <div className="max-w-lg mx-auto px-4 pt-32 pb-12 text-center">
```

```

<div className="card animate-fade-in">
 <div className="text-6xl mb-4">?</div>
 <h2 className="text-2xl font-bold text-white mb-2">Activity Logged!</h2>
 <p className="text-eco-400 text-lg font-semibold mb-1">
 +{success.activity?.emission?.toFixed(2)} kg CO?
 </p>
 <p className="text-slate-400 text-sm mb-4">
 Your eco score is now <strong className="text-white">{success.stats?.newEcoScore}

 </p>
 {success.stats?.streak > 1 && (
 <p className="text-orange-400 text-sm">? {success.stats.streak}-day streak!</p>
)}
 {/* AI impact context on success */}
 {success.activity?.emission > 0 && (
 <div className="mt-4 text-left">
 <ImpactContext emissionKg={success.activity.emission} />
 </div>
)}
 <p className="text-slate-500 text-xs mt-4">Redirecting to dashboard...</p>
</div>
</div>
</div>
);
}

return (
 <div className="min-h-screen bg-slate-950">
 <Navbar />
 <main className="max-w-2xl mx-auto px-4 pt-24 pb-12">
 <div className="mb-6">
 <h1 className="text-2xl font-bold text-white">Log Activity</h1>
 <p className="text-slate-400 text-sm mt-1">Track your daily carbon-emitting activitie
s</p>
 </div>

 {/* ?? AI Feature 1: Natural Language Parser ?? */}
 <NLPAActivityParser onParsed={handleNLPParsed} />

 {/* ?? AI Auto GPS Tracker (transportation only) ?? */}
 {form.type === 'transportation' && (
 <AutoTracker onApply={handleAutoTrack} />
)}

 {/* Activity Type Selector */}
 <div className="card mb-4">
 <h3 className="text-slate-300 font-medium text-sm mb-3">Activity Type</h3>
 <div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
 {Object.entries(ACTIVITY_CONFIG).map(([key, cfg]) => (
 <button
 key={key}
 onClick={() => handleTypeChange(key)}
 className={`flex flex-col items-center gap-2 p-4 rounded-xl border transition-a
ll duration-200 ${
 form.type === key
 ? 'border-eco-600 bg-eco-600/15 text-eco-400'
 : 'border-slate-700 bg-slate-800/50 text-slate-400 hover:border-slate-600 h
over:bg-slate-800'
 }`}
 >
 {cfg.icon}
 {cfg.label}
 </button>
)})}
 </div>
 </div>

 {/* Form */}

```

```

<form onSubmit={handleSubmit}>
 <div className="card space-y-5">

 {/* Category */}
 <div>
 <label className="label">Category *</label>
 <div className="grid grid-cols-2 sm:grid-cols-3 gap-2">
 {currentConfig?.categories.map(cat => (
 <button
 key={cat.value}
 type="button"
 onClick={() => setForm(f => ({ ...f, category: cat.value })))}
 className={`flex items-center gap-2.5 p-3 rounded-xl border text-left trans
ition-all duration-200 ${
 form.category === cat.value
 ? 'border-eco-600 bg-eco-600/15 text-eco-300'
 : 'border-slate-700 bg-slate-800/50 text-slate-400 hover:border-slate-6
00'
 }`}
 >
 {cat.icon}
 <div>
 <p className="text-xs font-medium">{cat.label}</p>
 <p className="text-xs opacity-60">{cat.unit}</p>
 </div>
 </button>
)})}
 </div>
 </div>

 {/* Value */}
 <div>
 <label className="label">
 Amount {selectedCategory ? `(${selectedCategory.unit})` : ''} *
 </label>
 <div className="relative">
 <input
 type="number"
 name="value"
 value={form.value}
 onChange={handleChange}
 className="input-field pr-20"
 placeholder={`Enter ${selectedCategory?.unit} || 'amount'`}
 min="0"
 step="0.1"
 />
 {selectedCategory && (
 <span className="absolute right-4 top-1/2 -translate-y-1/2 text-slate-500 tex
t-sm">
 {selectedCategory.unit}

)}
 </div>
 </div>

 {/* ?? AI Feature 2: Emission preview + carbon budget badge ?? */}
 {estimatedEmission !== null && (
 <div className={`mt-2 flex items-center justify-between gap-2 px-3 py-2 rounded
-lg text-sm ${
 estimatedEmission === 0
 ? 'bg-eco-500/10 border border-eco-700/30 text-eco-400'
 : estimatedEmission < 2
 ? 'bg-eco-500/10 border border-eco-700/30 text-eco-400'
 : estimatedEmission < 5
 ? 'bg-amber-500/10 border border-amber-700/30 text-amber-400'
 : 'bg-red-500/10 border border-red-700/30 text-red-400'
 }`}
 >


```

```

 {estimatedEmission === 0 ? '?' : estimatedEmission < 2 ? '?' : estima
tedEmission < 5 ? '??' : '?'}
 Est. {estimatedEmission.toFixed(2)} kg CO

 <CarbonBudgetBadge emission={estimatedEmission} />
</div>
)}

{/* ?? AI Feature 3: Smart pattern tip ?? */}
{smartTip && (
 <div className={`mt-2 flex items-center gap-2 px-3 py-2 rounded-lg text-xs ${
 smartTip.type === 'warn'
 ? 'bg-amber-500/10 border border-amber-700/30 text-amber-300'
 : 'bg-eco-500/10 border border-eco-700/30 text-eco-300'
 }`}>
 {smartTip.type === 'warn' ? '?' : '?'}
 AI Insight: {smartTip.text}
 </div>
)}

{/* ?? AI Feature 4: Impact Context (expandable) ?? */}
{estimatedEmission !== null && estimatedEmission > 0 && (
 <div>
 <button
 type="button"
 onClick={() => setShowImpact(s => !s)}
 className="mt-2 text-xs text-indigo-400 hover:text-indigo-300 transition-co
lors flex items-center gap-1"
 >
 {showImpact ? '?' : '?'}
 {showImpact ? 'Hide' : 'Show'} AI impact analysis
 </button>
 {showImpact && <ImpactContext emissionKg={estimatedEmission} />}
 </div>
)}
</div>

{/* Date */}
<div>
 <label className="label">Date</label>
 <input
 type="date"
 name="date"
 value={form.date}
 onChange={handleChange}
 className="input-field"
 max={new Date().toISOString().split('T')[0]}
 />
</div>

{/* Notes */}
<div>
 <label className="label">Notes (optional)
</label>

 <textarea
 name="notes"
 value={form.notes}
 onChange={handleChange}
 className="input-field resize-none"
 rows={3}
 placeholder="Add any additional details..."
 maxLength={200}
 />
 <p className="text-slate-600 text-xs mt-1 text-right">{form.notes.length}/200</p>
</div>

{error && (
 <div className="p-3 bg-red-500/10 border border-red-700/40 rounded-xl text-red-40

```

```

0 text-sm">
 {error}
 </div>
 })

 <div className="flex gap-3">
 <button type="button" onClick={() => navigate('/dashboard')} className="btn-secon
dary flex-1">
 Cancel
 </button>
 <button type="submit" disabled={loading} className="btn-primary flex-1">
 {loading ? (

 <span className="h-4 w-4 border-2 border-white/30 border-t-white rounded-fu
ll animate-spin" />
 Logging...

) : 'Log Activity'}
 </button>
 </div>
</div>
</form>
</main>
</div>
);
}

```



```

 <h1 className="text-2xl font-bold text-white mt-3">EcoBeacon AI</h1>
 </div>

 <div className="card">
 <h2 className="text-2xl font-bold text-white mb-1">Welcome back</h2>
 <p className="text-slate-400 text-sm mb-6">Sign in to continue your eco journey</p>

 {error && (
 <div className="mb-4 p-3 bg-red-500/10 border border-red-700/40 rounded-xl text-red-400 text-sm">
 {error}
 </div>
)}

 <form onSubmit={handleSubmit} className="space-y-4">
 <div>
 <label className="label">Email address</label>
 <input
 type="email"
 name="email"
 value={form.email}
 onChange={handleChange}
 className="input-field"
 placeholder="you@example.com"
 required
 />
 </div>
 <div>
 <label className="label">Password</label>
 <input
 type="password"
 name="password"
 value={form.password}
 onChange={handleChange}
 className="input-field"
 placeholder="???????"
 required
 />
 </div>
 <button type="submit" disabled={loading} className="btn-primary w-full mt-2">
 {loading ? (

 Signing in...

) : 'Sign In'}
 </button>
 </form>

 <div className="mt-5 p-3 bg-slate-800/50 rounded-xl border border-slate-700/50">
 <p className="text-slate-400 text-xs font-medium mb-1">Demo credentials:</p>
 <p className="text-slate-300 text-xs">alex@ecobeacon.com / password123</p>
 </div>

 <p className="text-center text-slate-400 text-sm mt-5">
 Don't have an account?{' '}
 <Link to="/register" className="text-eco-400 hover:text-eco-300 font-medium">
 Create one free
 </Link>
 </p>
 </div>
</div>
</div>
);
}

```

## 50. frontend/src/pages/Profile.jsx

---

```
import React, { useState, useEffect, useCallback } from 'react';
import { useAuth } from '../../context/AuthContext';
import api from '../../services/api';
import { formatEmission, getScoreColor, getScoreLabel } from '../../utils/helpers';
import Navbar from '../../components/Navbar';
import BadgesPanel from '../../components/BadgesPanel';

export default function Profile() {
 const { user, refreshUser } = useAuth();
 const [activities, setActivities] = useState([]);
 const [summary, setSummary] = useState(null);
 const [loading, setLoading] = useState(true);
 const [editName, setEditName] = useState(false);
 const [newName, setNewName] = useState(user?.name || '');
 const [saving, setSaving] = useState(false);
 const [deleteTarget, setDeleteTarget] = useState(null);
 const [page, setPage] = useState(1);
 const [pagination, setPagination] = useState({});

 const fetchData = useCallback(async () => {
 if (!user) return;
 setLoading(true);
 try {
 const [actRes, sumRes] = await Promise.all([
 api.get(`/activity/user/${user._id}?limit=15&page=${page}`),
 api.get(`/emissions/summary/${user._id}`)
]);
 setActivities(actRes.data.activities || []);
 setPagination(actRes.data.pagination || {});
 setSummary(sumRes.data);
 } catch (err) {
 console.error(err);
 } finally {
 setLoading(false);
 }
 }, [user, page]);

 useEffect(() => { fetchData(); }, [fetchData]);

 const handleSaveName = async () => {
 if (!newName.trim()) return;
 setSaving(true);
 try {
 await api.put('/auth/profile', { name: newName.trim() });
 await refreshUser();
 setEditName(false);
 } catch (err) {
 console.error(err);
 } finally {
 setSaving(false);
 }
 };

 const handleDelete = async (id) => {
 try {
 await api.delete(`/activity/${id}`);
 await Promise.all([fetchData(), refreshUser()]);
 setDeleteTarget(null);
 } catch (err) {
 console.error(err);
 }
 };

 const typeIcons = { transportation: '?', electricity: '?', food: '??', shopping: '??' };
```

```

return (
 <div className="min-h-screen bg-slate-950">
 <Navbar />
 <main className="max-w-5xl mx-auto px-4 pt-24 pb-12">

 {/ * Profile Header */}
 <div className="card mb-6">
 <div className="flex flex-col sm:flex-row items-start sm:items-center gap-5">
 <div
 className="w-20 h-20 rounded-2xl flex items-center justify-center text-3xl font-b
old border-2 flex-shrink-0"
 style={{
 borderColor: getScoreColor(user?.ecoScore || 50),
 background: ` ${getScoreColor(user?.ecoScore || 50)}15`,
 color: getScoreColor(user?.ecoScore || 50)
 }}
 >
 {user?.name?.charAt(0).toUpperCase()}
 </div>
 <div className="flex-1">
 {editName ? (
 <div className="flex items-center gap-3">
 <input
 value={newName}
 onChange={e => setNewName(e.target.value)}
 className="input-field max-w-xs py-2"
 autoFocus
 />
 <button onClick={handleSaveName} disabled={saving} className="btn-primary py-
2 px-4 text-sm">
 {saving ? '...' : 'Save'}
 </button>
 <button onClick={() => { setEditName(false); setNewName(user?.name); }} class
Name="btn-secondary py-2 px-4 text-sm">
 Cancel
 </button>
 </div>
) : (
 <div className="flex items-center gap-3">
 <h1 className="text-2xl font-bold text-white">{user?.name}</h1>
 <button onClick={() => setEditName(true)} className="text-slate-500 hover:tex
t-slate-300 text-sm">??</button>
 </div>
)}
 <p className="text-slate-400 text-sm mt-1">{user?.email}</p>
 <div className="flex items-center gap-3 mt-2">
 <span className="badge" style={{ background: ` ${getScoreColor(user?.ecoScore ||
50)}15`, color: getScoreColor(user?.ecoScore || 50), border: `1px solid ${getScoreColor(user?.
ecoScore || 50)}40` }}>
 {getScoreLabel(user?.ecoScore || 50)} ? Score: {user?.ecoScore}

 {user?.streak > 0 && (
 <span className="badge bg-orange-500/15 text-orange-400 border border-orange-
700/30">
 ? {user.streak}-day streak

)}
 </div>
 </div>
 </div>

 {/ * Stats mini grid */}
 <div className="grid grid-cols-2 gap-3 min-w-fit">
 {[
 { label: 'Total CO?', value: ` ${formatEmission(summary?.summary?.total)} kg` },
 { label: 'This Month', value: ` ${formatEmission(summary?.summary?.monthly)} kg`
 },
 { label: 'Activities', value: summary?.activityCount?.total ?? '?' },

```

```

 { label: 'Badges', value: user?.badges?.length ?? 0 }
].map(s => (
 <div key={s.label} className="bg-slate-800/70 rounded-xl p-3 text-center min-w-
[90px]">
 <p className="text-white font-bold">{s.value}</p>
 <p className="text-slate-500 text-xs mt-0.5">{s.label}</p>
 </div>
))}
</div>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
 { /* Badges */ }
 <BadgesPanel badges={user?.badges || []} streak={user?.streak || 0} />

 { /* Activity History */ }
 <div className="lg:col-span-2 card-hover">
 <div className="flex items-center justify-between mb-5">
 <h3 className="font-semibold text-white">Activity History</h3>
 {pagination.total || 0} total
 </div>

 {loading ? (
 <div className="space-y-2">
 {[1,2,3,4,5].map(i => (
 <div key={i} className="h-14 bg-slate-800 rounded-xl animate-pulse" />
))}
 </div>
) : activities.length === 0 ? (
 <div className="flex flex-col items-center justify-center py-12 text-slate-500 ga
p-3">
 ?
 <p className="text-sm">No activities logged yet</p>
 </div>
) : (
 <>
 <div className="space-y-2">
 {activities.map(a => (
 <div key={a._id} className="flex items-center gap-3 p-3 bg-slate-800/50 rou
nded-xl hover:bg-slate-800 transition-colors group">
 <div className="w-9 h-9 rounded-xl bg-slate-700 flex items-center justify
-center text-base flex-shrink-0">
 {typeIcons[a.type] || '?'}
 </div>
 <div className="flex-1 min-w-0">
 <p className="text-slate-200 text-sm font-medium capitalize truncate">
 {a.category.replace(/-/g, ' ')}
 · {a.value}
{a.unit || a.type === 'food' ? 'meals' : a.type === 'shopping' ? 'items' : a.type === 'electri
city' ? 'kWh' : 'km'}
 </p>
 <p className="text-slate-500 text-xs">{new Date(a.date).toLocaleDateStr
ing('en-US', { weekday: 'short', month: 'short', day: 'numeric', year: 'numeric' })}</p>
 </div>
 <div className="text-right flex-shrink-0">
 <p className={`text-sm font-bold ${a.emission === 0 ? 'text-eco-400' :
a.emission < 2 ? 'text-eco-300' : a.emission < 5 ? 'text-amber-400' : 'text-red-400'}`}>
 {a.emission === 0 ? '0.00' : `+${a.emission.toFixed(2)}`}
 </p>
 <p className="text-slate-500 text-xs">kg CO₂</p>
 </div>
 <button
 onClick={() => setDeleteTarget(a._id)}
 className="opacity-0 group-hover:opacity-100 ml-2 p-1.5 text-slate-600
hover:text-red-400 hover:bg-red-500/10 rounded-lg transition-all"
 >

```

```

 ??
 </button>
 </div>
)}}
</div>

 { /* Pagination */
 {pagination.pages > 1 && (
 <div className="flex justify-center gap-2 mt-4">
 <button disabled={page === 1} onClick={() => setPage(p => p - 1)} className
="btn-secondary py-1.5 px-3 text-sm disabled:opacity-40">? Prev</button>
 {page} / {p
agination.pages}
 <button disabled={page === pagination.pages} onClick={() => setPage(p => p
+ 1)} className="btn-secondary py-1.5 px-3 text-sm disabled:opacity-40">Next ?</button>
 </div>
)}
</>
)}
</div>
</div>

 { /* Delete Confirmation Modal */
 {deleteTarget && (
 <div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-50 flex items-center jus
tify-center p-4">
 <div className="card max-w-sm w-full">
 <h3 className="font-bold text-white text-lg mb-2">Delete Activity?</h3>
 <p className="text-slate-400 text-sm mb-5">This will permanently remove the activ
ity and recalculate your eco score.</p>
 <div className="flex gap-3">
 <button onClick={() => setDeleteTarget(null)} className="btn-secondary flex-1">
Cancel</button>
 <button onClick={() => handleDelete(deleteTarget)} className="flex-1 bg-red-600
hover:bg-red-500 text-white font-semibold py-2.5 px-5 rounded-xl transition-colors">Delete</bu
tton>
 </div>
 </div>
 </div>
)}
</main>
</div>
);
}

```

## 51. frontend/src/pages/Register.jsx

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Register() {
 const [form, setForm] = useState({ name: '', email: '', password: '', confirmPassword: '' });
 const [error, setError] = useState('');
 const [loading, setLoading] = useState(false);
 const { register } = useAuth();
 const navigate = useNavigate();

 const handleChange = (e) => setForm(f => ({ ...f, [e.target.name]: e.target.value }));

 const handleSubmit = async (e) => {
 e.preventDefault();
 setError('');

 if (form.password !== form.confirmPassword) {
 setError('Passwords do not match');
 return;
 }
 if (form.password.length < 6) {
 setError('Password must be at least 6 characters');
 return;
 }

 setLoading(true);
 try {
 await register(form.name, form.email, form.password);
 navigate('/dashboard');
 } catch (err) {
 setError(err.response?.data?.message || 'Registration failed. Please try again.');
```

} finally {
 setLoading(false);
 }
 };

 return (
 <div className="min-h-screen bg-slate-950 flex items-center justify-center p-6">
 <div className="w-full max-w-md">
 <div className="text-center mb-6">
 <span className="text-5xl">?</span>
 <h1 className="text-2xl font-bold text-white mt-3">Join EcoBeacon AI</h1>
 <p className="text-slate-400 text-sm mt-1">Start your sustainability journey today</p>
 </div>

 <div className="card">
 <h2 className="text-xl font-bold text-white mb-5">Create your account</h2>

 {error && (
 <div className="mb-4 p-3 bg-red-500/10 border border-red-700/40 rounded-xl text-red-400 text-sm">
 {error}
 </div>
 )}

 <form onSubmit={handleSubmit} className="space-y-4">
 <div>
 <label className="label">Full Name</label>
 <input
 type="text"
 name="name"
 value={form.name}
 onChange={handleChange}

```

 className="input-field"
 placeholder="Jane Doe"
 required
 />
 </div>
 <div>
 <label className="label">Email address</label>
 <input
 type="email"
 name="email"
 value={form.email}
 onChange={handleChange}
 className="input-field"
 placeholder="you@example.com"
 required
 />
 </div>
 <div>
 <label className="label">Password</label>
 <input
 type="password"
 name="password"
 value={form.password}
 onChange={handleChange}
 className="input-field"
 placeholder="Min. 6 characters"
 required
 />
 </div>
 <div>
 <label className="label">Confirm Password</label>
 <input
 type="password"
 name="confirmPassword"
 value={form.confirmPassword}
 onChange={handleChange}
 className="input-field"
 placeholder="Repeat your password"
 required
 />
 </div>
 <button type="submit" disabled={loading} className="btn-primary w-full">
 {loading ? (

 <span className="h-4 w-4 border-2 border-white/30 border-t-white rounded-full
animate-spin" />
 Creating account...

) : 'Create Account'}
 </button>
 </form>

 <p className="text-center text-slate-400 text-sm mt-5">
 Already have an account?{' '}
 <Link to="/login" className="text-eco-400 hover:text-eco-300 font-medium">
 Sign in
 </Link>
 </p>
</div>
</div>
</div>
);
}

```

## 52. frontend/src/services/api.js

---

```
import axios from 'axios';

const api = axios.create({
 baseURL: '/api',
 timeout: 10000,
 headers: { 'Content-Type': 'application/json' }
});

api.interceptors.request.use(
 (config) => {
 const token = localStorage.getItem('eco_token');
 if (token) config.headers.Authorization = `Bearer ${token}`;
 return config;
 },
 (error) => Promise.reject(error)
);

api.interceptors.response.use(
 (response) => response,
 (error) => {
 if (error.response?.status === 401) {
 localStorage.removeItem('eco_token');
 window.location.href = '/login';
 }
 return Promise.reject(error);
 }
);

export default api;
```

## 53. frontend/src/utils/geoTracking.js

---

```
// Haversine formula ? distance between two GPS points in km
export const haversineDistance = (lat1, lon1, lat2, lon2) => {
 const R = 6371;
 const dLat = (lat2 - lat1) * (Math.PI / 180);
 const dLon = (lon2 - lon1) * (Math.PI / 180);
 const a =
 Math.sin(dLat / 2) * Math.sin(dLat / 2) +
 Math.cos(lat1 * (Math.PI / 180)) *
 Math.cos(lat2 * (Math.PI / 180)) *
 Math.sin(dLon / 2) *
 Math.sin(dLon / 2);
 return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
};

// AI speed-based transport mode classifier
// Uses a weighted average of recent speeds to reduce GPS noise
export const classifyTransportMode = (speedHistory = []) => {
 if (speedHistory.length === 0) return null;

 // Weight recent readings more heavily
 const weighted = speedHistory.map((s, i) => s * (i + 1));
 const avgSpeed = weighted.reduce((a, b) => a + b, 0) /
 speedHistory.map((_, i) => i + 1).reduce((a, b) => a + b, 0);

 if (avgSpeed < 1) return { mode: 'walking', label: 'Stationary', icon: '?', color:
'text-slate-400' };
 if (avgSpeed < 7) return { mode: 'walking', label: 'Walking', icon: '?', color
: 'text-eco-400' };
 if (avgSpeed < 25) return { mode: 'bike', label: 'Cycling', icon: '?', color
: 'text-blue-400' };
 if (avgSpeed < 60) return { mode: 'bus', label: 'Bus / City Drive', icon: '?', colo
r: 'text-amber-400' };
 if (avgSpeed < 130) return { mode: 'car', label: 'Car', icon: '?', color
: 'text-orange-400' };
 if (avgSpeed < 350) return { mode: 'train', label: 'Train', icon: '?', color
: 'text-purple-400' };
 return { mode: 'flight', label: 'Flight', icon: '??', colo
r: 'text-red-400' };
};

export const formatDistance = (km) => {
 if (km < 1) return `${(km * 1000).toFixed(0)} m`;
 return `${km.toFixed(2)} km`;
};

export const formatSpeed = (kmh) => `${kmh.toFixed(1)} km/h`;
```

## 54. frontend/src/utils/helpers.js

---

```

export const formatEmission = (value) => {
 if (value === null || value === undefined) return '0.00';
 return Number(value).toFixed(2);
};

export const getScoreColor = (score) => {
 if (score >= 80) return '#10b981';
 if (score >= 50) return '#f59e0b';
 return '#ef4444';
};

export const getScoreLabel = (score) => {
 if (score >= 80) return 'Excellent';
 if (score >= 50) return 'Moderate';
 return 'Needs Improvement';
};

export const getScoreEmoji = (score) => {
 if (score >= 80) return '?';
 if (score >= 50) return '?';
 return '??';
};

export const getCategoryColor = (category) => {
 const colors = {
 transportation: '#3b82f6',
 electricity: '#f59e0b',
 food: '#10b981',
 shopping: '#8b5cf6'
 };
 return colors[category] || '#64748b';
};

export const getCategoryIcon = (category) => {
 const icons = {
 transportation: '?',
 electricity: '?',
 food: '??',
 shopping: '??',
 general: '?'
 };
 return icons[category] || '?';
};

export const ACTIVITY_CONFIG = {
 transportation: {
 label: 'Transportation',
 icon: '?',
 color: '#3b82f6',
 categories: [
 { value: 'car', label: 'Car', unit: 'km', icon: '?' },
 { value: 'motorcycle', label: 'Motorcycle', unit: 'km', icon: '??' },
 { value: 'bus', label: 'Bus', unit: 'km', icon: '?' },
 { value: 'train', label: 'Train', unit: 'km', icon: '?' },
 { value: 'flight', label: 'Flight', unit: 'km', icon: '??' },
 { value: 'bike', label: 'Bicycle', unit: 'km', icon: '?' },
 { value: 'walking', label: 'Walking', unit: 'km', icon: '?' },
 { value: 'electric_car', label: 'Electric Car', unit: 'km', icon: '??' }
]
 },
 electricity: {
 label: 'Electricity',
 icon: '?',
 color: '#f59e0b',

```

```

categories: [
 { value: 'standard', label: 'Standard Grid', unit: 'kWh', icon: '?' },
 { value: 'solar', label: 'Solar', unit: 'kWh', icon: '??' },
 { value: 'wind', label: 'Wind Energy', unit: 'kWh', icon: '?' }
]
},
food: {
 label: 'Food',
 icon: '??',
 color: '#10b981',
 categories: [
 { value: 'beef', label: 'Beef Meal', unit: 'meals', icon: '?' },
 { value: 'meat', label: 'Meat Meal', unit: 'meals', icon: '?' },
 { value: 'fish', label: 'Fish Meal', unit: 'meals', icon: '?' },
 { value: 'fastfood', label: 'Fast Food', unit: 'meals', icon: '?' },
 { value: 'dairy', label: 'Dairy-based', unit: 'meals', icon: '?' },
 { value: 'veg', label: 'Vegetarian', unit: 'meals', icon: '?' },
 { value: 'vegan', label: 'Vegan', unit: 'meals', icon: '?' }
]
},
shopping: {
 label: 'Shopping',
 icon: '??',
 color: '#8b5cf6',
 categories: [
 { value: 'high-impact', label: 'High Impact', unit: 'items', icon: '?' },
 { value: 'normal', label: 'Normal', unit: 'items', icon: '?' },
 { value: 'eco-friendly', label: 'Eco-Friendly', unit: 'items', icon: '??' },
 { value: 'secondhand', label: 'Secondhand', unit: 'items', icon: '?' }
]
}
};

export const EMISSION_FACTORS = {
 transportation: { car: 0.21, motorcycle: 0.103, bus: 0.089, train: 0.041, flight: 0.255, bike: 0.005, walking: 0, electric_car: 0.053 },
 electricity: { standard: 0.85, solar: 0.041, wind: 0.011 },
 food: { beef: 6.61, meat: 5.0, fish: 2.8, fastfood: 3.5, dairy: 2.1, veg: 1.5, vegan: 0.9 },
 shopping: { 'high-impact': 5.0, normal: 2.0, 'eco-friendly': 0.5, secondhand: 0.2 }
};

export const estimateEmission = (type, category, value) => {
 const factor = EMISSION_FACTORS[type]?.[category] ?? 0;
 return parseFloat((factor * value).toFixed(4));
};

export const formatDate = (date) => {
 return new Date(date).toLocaleDateString('en-US', { year: 'numeric', month: 'short', day: 'numeric' });
};

export const formatDateShort = (date) => {
 return new Date(date).toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
};

```

## 55. frontend/tailwind.config.js

---

```

/** @type {import('tailwindcss').Config} */
export default {
 content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
 darkMode: 'class',
 theme: {
 extend: {
 colors: {
 eco: {
 50: '#f0fdf4',
 100: '#dcfce7',
 200: '#bbf7d0',
 300: '#86efac',
 400: '#4ade80',
 500: '#22c55e',
 600: '#16a34a',
 700: '#15803d',
 800: '#166534',
 900: '#14532d',
 950: '#052e16'
 }
 },
 animation: {
 'spin-slow': 'spin 3s linear infinite',
 'pulse-slow': 'pulse 3s cubic-bezier(0.4, 0, 0.6, 1) infinite',
 'fade-in': 'fadeIn 0.5s ease-in-out'
 },
 keyframes: {
 fadeIn: {
 '0%': { opacity: '0', transform: 'translateY(10px)' },
 '100%': { opacity: '1', transform: 'translateY(0)' }
 }
 }
 }
 },
 plugins: []
};

```

## 56. frontend/vite.config.js

---

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
 plugins: [react()],
 server: {
 port: 5173,
 proxy: {
 '/api': {
 target: 'http://localhost:5000',
 changeOrigin: true
 }
 }
 }
});
```

## 57. remove\_blank\_pdf\_pages.py

---

*[Binary or unsupported file type: .py] - content not embedded.*